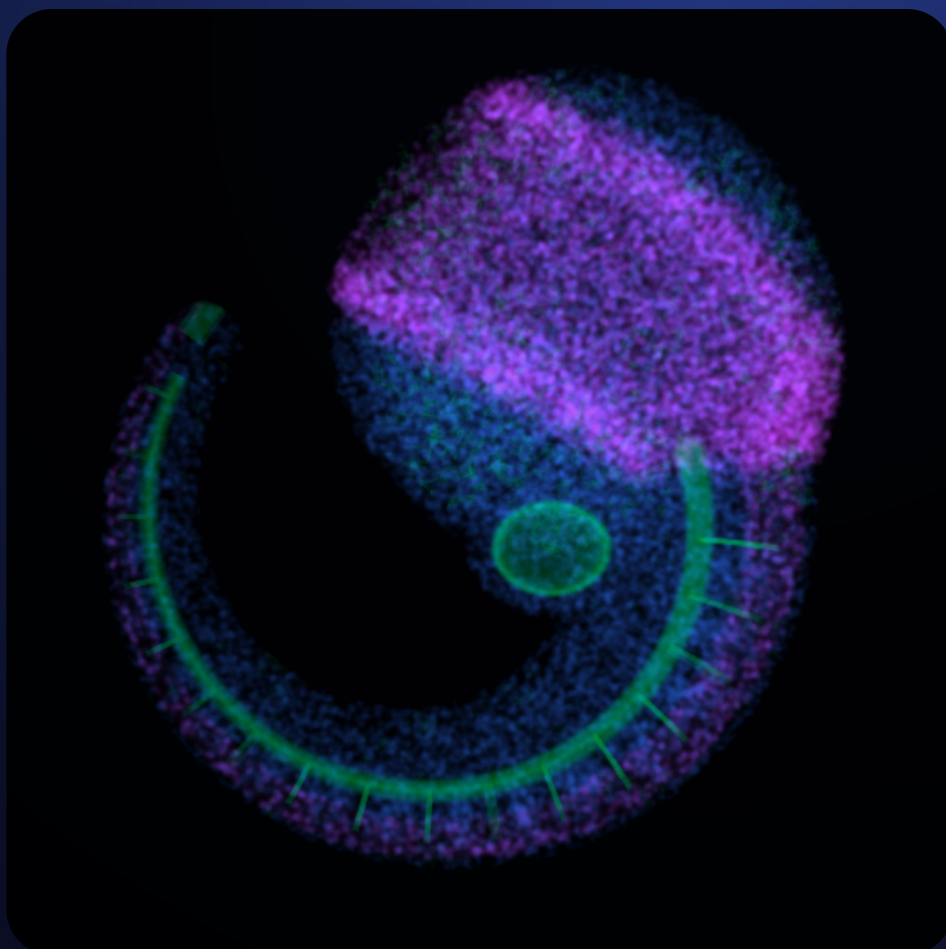




IRIBHM · ULB — LUMEN3D

Lumen3D, de A à Z

Documentation complète de la plateforme

Comment un fichier Imaris de plusieurs gigaoctets devient un embryon que l'on fait tourner dans un navigateur : les langages, les pages, le nettoyage de l'image, la compression, les briques, le rendu 3D, les outils de mesure et le panneau d'administration. Écrit pour des biologistes, illustré à chaque page.

Sommaire

Comment lire ce document	5
Trois façons de le lire	5
Les encadrés	5
Les conventions	5
1. La plateforme en un coup d'œil	7
1.1 Qu'est-ce que Lumen3D, et pour qui ?	7
1.2 Le problème : un fichier trop gros pour un navigateur	7
1.3 Les trois types de jeux de données	8
1.4 Le voyage d'une acquisition	9
1.5 Chiffres clés	10
1.6 Carte de la documentation	11
2. Les technologies : qui fait quoi	12
2.1 Une page web, c'est un corps humain	12
2.2 Le client : ce qui tourne dans votre navigateur	12
2.3 GLSL : le programme minuscule exécuté des millions de fois	13
2.4 Le serveur : un guichet, pas un cerveau	
2.5 Python : l'établi du laboratoire	15
2.6 Les formats de fichiers : des contenants	15
2.7 Pourquoi tout est « auto-hébergé »	16
2.8 Ce qu'il faut côté navigateur	17
2.9 Le dossier du projet	17
3. Le tour des pages	19
3.1 La carte des pages	19
3.2 L'Accueil	20
3.3 L'Explorateur	21
3.4 Le viewer 3D	22
3.5 Le viewer Live : la ligne de temps	23
3.6 La page 2D	24
3.7 Comparer	25
3.8 À propos et Mentions légales	26
3.9 L'administration : un aperçu	26
3.10 Les adresses à connaître	27
4. Du microscope au fichier Imaris	29
4.1 Du pixel au voxel	29
4.2 Une pile de coupes, plusieurs canaux	30
4.3 Combien de niveaux de gris ? 8 ou 16 bits	31
4.4 Un .ims est un « disque dur dans un fichier »	31
4.5 La pyramide d'Imaris : ignorée	32
4.6 L'étape 1 : lire les métadonnées	33
4.7 La calibration : la taille du voxel	33
4.8 L'horloge d'une séquence temporelle	34
5. Nettoyer l'image : retirer le bruit de fond	36

5.1 Pourquoi nettoyer ?	36
5.2 Étape A : mesurer le fond (<code>bg_floor</code>)	36
5.3 Étape A (suite) : mesurer le blanc (<code>sig_max</code>)	38
5.4 Le masque : protéger le vrai signal	38
5.5 La médiane, hors du masque seulement	40
5.6 La fenêtre : de 16 à 8 bits	41
5.7 Avant et après	
5.8 Ce qui a été essayé, puis abandonné	43
5.9 Séries temporelles : une seule fenêtre	43
5.10 Gros volumes : par tuiles, sans changer le résultat	44
5.11 Ce que cela change pour vous	45
6. Réduire la taille sans rien perdre d'important	
6.1 Le budget de taille en chiffres	47
6.2 De 16 bits à 8 bits : diviser par deux	48
6.3 La pyramide de résolutions	48
6.4 Compresser sans rien perdre	
6.5 Ne pas stocker le vide	53
6.6 Taille finale : du .ims au dossier publié	54
7. Le découpage en briques	56
7.1 Pourquoi découper un volume ?	56
7.2 Une brique : 64 voxels utiles et une bordure	57
7.3 D'un cube à une image : la mosaïque	58
7.4 La grille de briques de chaque niveau	
7.5 Les paquets : des boîtes de 64 briques	61
7.6 L'index et le manifeste	
7.7 L'arbre d'un jeu de données publié	64
7.8 <code>planes/</code> : une coupe XY sans lire 64 plans	64
7.9 <code>mips/</code> : le maximum d'une couche de 64 plans	65
7.10 Les formats de données 1 à 4	66
8. La fin du pipeline : métadonnées, suivi cellulaire, publication	
8.1 La vignette	68
8.2 Les histogrammes	
8.3 <code>metadata.json</code> : la carte d'identité du jeu de données	69
8.4 Le suivi cellulaire (séries temporelles)	
8.5 Publier « tout ou rien »	73
8.6 Ce que le pipeline conserve quand on retire un jeu de données	
8.7 Le dossier <code>download/</code>	74
8.8 Les photographies 2D, en une demi-page	
8.9 Lancer tout cela : <code>RUN.bat</code>	75
9. Le viewer 3D : Three.js et le lancer de rayons	
9.1 Deux cerveaux : le processeur et la carte graphique	78
9.2 WebGL2 et Three.js : qui fait quoi	
9.3 Le lancer de rayons, pas à pas	80
9.4 Le grain aléatoire contre les bandes	81
9.5 Sauter le vide	81
9.6 Les trois modes de rendu	82
9.7 Lisser entre les voxels	85

9.8	Fluide quand vous bougez, net quand vous vous arrêtez	85
9.9	Le garde-fou contre les plantages de la carte graphique	86
9.10	Les vraies proportions : calibration et étirement en Z	86
10.	Afficher des gigaoctets : le streaming	88
10.1	La séquence de chargement	88
10.2	Le menu de qualité : 512, 1024, Natif	89
10.3	L'atlas GPU et la table des pages	91
10.4	Le budget de mémoire vidéo	92
10.5	Télécharger et décoder sans bloquer la page	93
10.6	Les paquets : un téléchargement ordonné	94
10.7	Les séries temporelles	94
10.8	Le mode « Détail au zoom »	96
10.9	Quand la carte graphique plante	97
11.	Couleurs, contrastes et histogrammes	98
11.1	La chaîne de traitement d'un canal	98
11.2	L'étape cachée : le plancher du fond	99
11.3	La fenêtre min / max et le gamma	99
11.4	Le panneau d'un canal	100
11.5	Les histogrammes	102
11.6	Les boutons de réglage rapide	103
11.7	Couleurs, opacité, exposition, solo	103
11.8	Le flou gaussien	104
11.9	Garde-fou : ce que les niveaux de gris veulent dire	106
13.	Le panneau d'administration en bref	107
13.1	À quoi sert le panneau, et comment y accéder	107
13.2	Les 15 onglets, en 4 groupes	107
13.4	Mettre un dataset en ligne	109
13.5	Éditer un dataset	
13.6	Mises à jour des données	111
13.7	Plugins et catalogue signé	113
13.8	Mettre à jour la plateforme	113
13.9	Personnaliser le site (marque blanche)	114
14.	Sécurité et fiabilité	116
14.1	Le mot de passe administrateur	116
14.2	La politique de sécurité du contenu (CSP)	
14.3	Les fichiers importés ne sont jamais servis	117
14.4	Les plugins : confiance et cage	
14.5	Versions et catalogue signés	118
14.6	Vos données restent chez vous	119
14.7	Fiabilité	119
15.	Glossaire	120

Comment lire ce document

Ce document explique **tout** ce que fait Lumen3D, sans supposer que vous savez programmer. Chaque page a au moins une illustration ; le texte reste court.

Trois façons de le lire

En 20 minutes

Lisez seulement les encadrés verts « **En 30 secondes** » de chaque chapitre et regardez les figures.

« Mon dataset »

Chapitres **4 à 8** : ce qui arrive à votre fichier Imaris, étape par étape, avec de vrais chiffres.

« L'écran »

Chapitres **9 à 12** : comment le viewer dessine le volume et comment fonctionnent les outils.

Les encadrés

EN 30 SECONDES

Le résumé du chapitre ou de la section, en trois points.

UNE IMAGE POUR COMPRENDRE

Une comparaison avec la vie de tous les jours.

POUR LES CURIEUX

Le détail technique (formules, noms de fichiers). Vous pouvez le sauter.

MICRO-EXEMPLE

Un calcul fait à la main avec de vrais nombres.

ATTENTION

Une limite à connaître avant d'interpréter vos images.

Les conventions

VOUS VOYEZ...	CELA SIGNIFIE...
 Enregistrer	un bouton ou un libellé tel qu'il apparaît à l'écran
 3	le repère numéroté n° 3 sur la capture d'écran juste au-dessus
<code>metadata.json</code>	un nom de fichier ou une valeur technique
→ chapitre 7	la notion est expliquée en détail ailleurs

À SAVOIR

Les images de volumes de ce document proviennent d'**embryons de démonstration synthétiques**, fabriqués pour cette documentation puis traités par le vrai pipeline de la plateforme. Ce ne sont pas des données du laboratoire. Les chiffres cités (seuils, tailles, formules) sont lus dans le code de la version 1.59.2 de la plateforme et 0.21.0 du pipeline.

POUR ALLER PLUS LOIN

Une version courte de ce document, « **L'essentiel pour le biologiste** » (10 pages), et le **Guide de l'administrateur** sont publiés au même endroit, dans l'onglet *Documentation* du panneau d'administration.

1. La plateforme en un coup d'œil

- **Lumen3D** est un site web qui permet d'explorer, dans un navigateur ordinaire, des images de microscopie confocale pesant plusieurs gigaoctets : rien à installer.
- Une pile de ~10 Go ne tient ni dans la mémoire d'un navigateur ni dans celle d'une carte graphique : toute la plateforme est conçue pour ne charger que ce que l'écran montre.
- Il existe **trois types** de jeux de données (3D, Live, 2D) et un chemin unique du microscope à l'écran, que ce chapitre vous fait parcourir.

1.1 Qu'est-ce que Lumen3D, et pour qui ?

Lumen3D (*Light-based Unified Microscopy Exploration in 3D*) est né à l'IRIBHM (ULB) pour regarder des embryons de souris. Il est ensuite devenu une plateforme « marque blanche » : nom, couleurs, pages et vocabulaire se changent depuis le panneau d'administration, sans écrire une ligne de code.

Quatre personnes l'utilisent, chacune avec une part différente de ce document :

Le biologiste

Ouvre un embryon, règle les couleurs, mesure, coupe, compare, fabrique une figure.

chapitres 3, 11, 12

Le technicien

Lance le **pipeline** Python qui transforme le fichier Imaris en dossier web.

chapitres 4 à 8

L'administrateur

Importe, publie, personnalise le site, met à jour la plateforme.

chapitres 13 et 14

Le visiteur public

Parcourt le catalogue publié et regarde, sans compte ni mot de passe.

chapitre 3

i À SAVOIR

Il n'y a **aucun compte utilisateur**. Seule l'administration est protégée par un mot de passe ; tout ce qui est publié est visible par quiconque a l'adresse du site.

1.2 Le problème : un fichier trop gros pour un navigateur

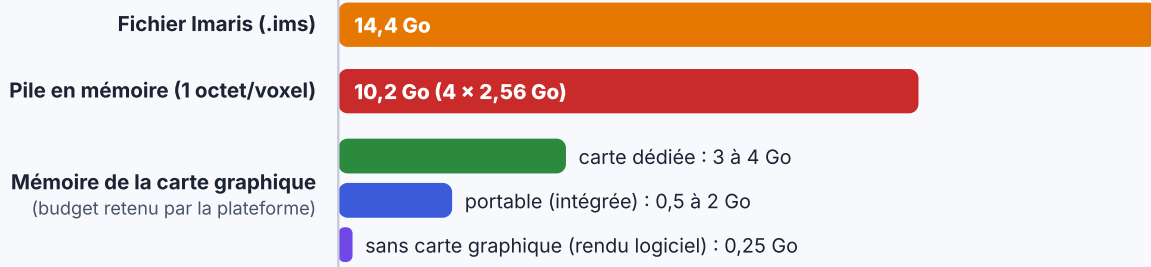
Prenons le plus gros jeu de référence du pipeline : un embryon de **3789 × 3789 × 178 voxels** sur **4 canaux**. Le fichier Imaris d'origine pèse **14,4 Go**.

1 2 3 4 MICRO-EXEMPLE

Le calcul. $3789 \times 3789 \times 178 = 2\,555\,460\,738$ voxels par canal. À 1 octet par voxel : **≈ 2,56 Go par canal**, donc **≈ 10,2 Go** pour 4 canaux. La carte graphique, elle, n'offre à la plateforme que 0,25 à 4 Go selon sa classe.

Pourquoi un navigateur ne peut pas « simplement ouvrir » la pile

Exemple réel : embryon 3789 × 3789 × 178 voxels, 4 canaux (jeu de référence du pipeline)



Conclusion : il faut découper, réduire et ne charger que ce que l'écran montre.

À ~5 Mo/s (débit mesuré sur un hébergeur), télécharger 10 Go d'un coup prendrait plus d'une demi-heure.

▲ Même en le réduisant à 1 octet par voxel, la pile dépasse la mémoire de la plupart des cartes graphiques.

Trois obstacles se cumulent :

- **le réseau** : à environ 5 Mo/s, 10 Go demandent plus d'une demi-heure ;
- **la mémoire du navigateur** : il n'est pas fait pour garder des gigaoctets d'image ;
- **la mémoire de la carte graphique (VRAM)** : c'est elle qui dessine, et elle est petite.

💡 UNE IMAGE POUR COMPRENDRE

Une encyclopédie de 50 tomes. Personne ne pose les 50 tomes ouverts sur son bureau pour lire un paragraphe. On prend l'index, on ouvre le bon tome à la bonne page. Lumen3D fait pareil avec l'image : un index (le *manifeste*) et des petits morceaux (les *briques*) que l'on va chercher à la demande.

La réponse tient en trois idées, détaillées aux chapitres 5 à 10 :

- 1 **Réduire** : retirer le bruit de fond, passer en 8 bits, fabriquer des versions plus petites de l'image (les niveaux de détail).
- 2 **Découper** : cuber l'image en briques de 64 voxels de côté et les ranger dans quelques gros fichiers.
- 3 **Streamer** : le navigateur télécharge d'abord la version grossière (image instantanée), puis affine seulement la zone regardée.

1.3 Les trois types de jeux de données

Tout jeu de données publié est d'un de ces trois types. Le même mot sert de nom de dossier, de début d'identifiant (par exemple `3d/Embryo-E95-Em2-Pecam1-Sox2`) et de filtre dans l'Explorateur.

Les trois types de jeux de données

3D · « Imagerie 3D »



Un volume figé

piles confocales multicanaux
page : viewer.html

[dossier DATA_WEB/3d/](#)

Live · « Imagerie en direct »



Un volume qui évolue

time-lapse : 1 volume par instant
page : viewer.html + ligne de temps

[DATA_WEB/live/ \(+ suivi cellulaire\)](#)

2D · « Imagerie 2D »



Une photographie

stéréomicroscope, calibrée en μm
page : 2d.html

[dossier DATA_WEB/2d/](#)

Le suivi cellulaire n'est pas un type : c'est une couche d'un jeu « Live »
(points, trajectoires, surface), affichée dans le viewer et analysée par cinq outils dédiés.

▲ Trois types, trois pages d'affichage. Les noms entre guillemets sont ceux que le public lit par défaut.

À SAVOIR

Les noms affichés (« 3D », « Live », « 2D ») appartiennent à l'administrateur : il peut les renommer dans l'onglet **Types de données** sans toucher aux dossiers. Dans ce document, on utilise les noms par défaut.

TYPE	CE QUE CONTIENT LE JEU	EXEMPLE DU JEU DE DÉMONSTRATION
3D	un volume, 1 à 4 canaux	Embryo-E95-Em2-Pecam1-Sox2 (768 × 576 × 112, 3 canaux)
Live	un volume par instant, avec ligne de temps	Demo-Lumen3D-E85-Em1-30min-2ch-4tp (4 instants)
2D	une photographie calibrée en μm	DLL4xCD1-E95-x3.2-240913-1 (coloration X-gal)

? POURQUOI CE CHOIX ?

Pourquoi le suivi cellulaire n'est-il pas un type ? Parce qu'il décrit un jeu Live, il ne le remplace pas. Les positions des cellules, leurs trajectoires et la surface sont des **couches** dessinées par-dessus le volume, et cinq outils dédiés les analysent (chapitre 12). Un jeu Live sans suivi reste un jeu Live.

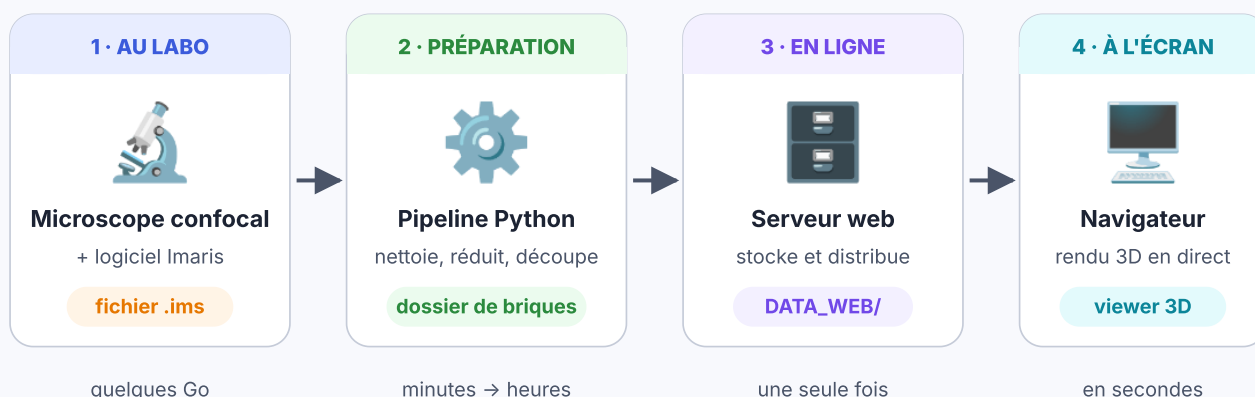
⚠ ATTENTION

Les jeux de ce document sont des **jeux de démonstration synthétiques**, fabriqués pour l'occasion : ce ne sont pas des données réelles du laboratoire.

1.4 Le voyage d'une acquisition

De l'échantillon à l'écran, les données traversent quatre lieux.

Le voyage d'une acquisition : du microscope à l'écran



Le fichier d'origine n'est jamais modifié : le pipeline en fabrique une copie optimisée pour le web.

▲ Du microscope à l'écran, en quatre étapes.

ÉTAPE	QUI	DURÉE TYPIQUE	CHAPITRE
1. Acquisition	microscope confocal + logiciel Imaris	selon l'échantillon	4
2. Préparation	technicien, pipeline Python sur son ordinateur	minutes à heures, une seule fois	5 à 8
3. Mise en ligne	administrateur : import par glisser-déposer ou SFTP, puis publication	selon le débit	13
4. Affichage	n'importe qui, avec un navigateur	quelques secondes pour la première image	9 à 12

📌 À RETENIR

Le travail lourd (nettoyage, découpage) se fait **une seule fois, hors ligne**. Ensuite, chaque visiteur ne télécharge que des morceaux : c'est ce qui rend l'exploration fluide.

1.5 Chiffres clés

3

types de données : 3D, Live, 2D

64³

voxels par brique (66³ avec bord, format 4)

4

canaux affichés au maximum

4

langues : anglais, français, espagnol, néerlandais

28plug-ins au catalogue
signé**≈ 126 000**lignes de code, hors
tests**512 · 1024 ·
natif**les trois qualités de
rendu**201**fichiers de tests
automatiques

Quelques repères de version, valables à la date de ce document :

- plateforme web **1.59.2** ; pipeline de préparation **0.21.0** ;
- format de données courant : **format 4** (les formats 1 à 3 sont convertis en place, voir chapitre 13) ;
- chaque version est vérifiée par les tests automatiques avant d'être publiée.

1.6 Carte de la documentation

Ce document est organisé comme le trajet de la donnée. Retrouvez votre question ci-dessous.

Carte de la documentation : quelle question, quel chapitre ?

Découvrir

chapitres 1 à 3

- 1 À quoi sert Lumen3D ? Quels types de données ?
- 2 Quelles technologies, et à quoi servent-elles ?
- 3 Où cliquer ? À quoi ressemble chaque page ?

Préparer

le pipeline Python
chapitres 4 à 8

- 4 Que contient un fichier Imaris ?
- 5 Comment le bruit de fond est-il retiré ?
- 6 Comment passer de 14 Go à quelques centaines de Mo ?
- 7 Que sont les briques, les packs, les niveaux de détail ?
- 8 Métadonnées, suivi cellulaire, publication sans risque

Afficher

le navigateur
chapitres 9 à 12

- 9 Comment l'image 3D est-elle calculée ?
- 10 Comment afficher des gigaoctets sans attendre ?
- 11 Couleurs, contrastes, histogrammes : que signifient-ils ?
- 12 Que fait chaque outil de la barre d'outils ?

Administrer

et comprendre
chapitres 13 à 15

- 13 Que peut faire l'administrateur ? (aperçu)
- 14 Mes données sont-elles protégées ? Que se passe-t-il en cas de panne ?
- 15 Que veut dire ce mot ? (glossaire)

▲ Quelle question, quel chapitre ?

👉 ASTUCE

Pressé ? Lisez les encadrés **En 30 secondes** au début de chaque chapitre : ils suffisent pour savoir si la suite vous concerne.

2. Les technologies : qui fait quoi

- Lumen3D mélange cinq langages et quelques formats de fichiers ; chacun a **un seul métier**, et ce chapitre vous dit lequel.
- Tout ce que le navigateur exécute est **fourni par le site lui-même** (aucun service extérieur) : la plateforme fonctionne hors ligne et reste sous contrôle.
- Le serveur ne calcule jamais l'image 3D : il **distribue des fichiers**. Le dessin se fait chez vous, sur votre carte graphique.

2.1 Une page web, c'est un corps humain

Avant les détails, une image pour tout retenir :

HTML : le squelette

Dit ce qu'il y a sur la page : un titre, un bouton, une zone de dessin. Un fichier par page, à la racine du projet.

CSS : les vêtements

Dit à quoi ça ressemble : couleurs, tailles, thème clair ou sombre. 16 feuilles de style.

JavaScript : les muscles

Fait bouger les choses : clic sur un bouton, téléchargement des briques, calcul des mesures.

💡 UNE IMAGE POUR COMPRENDRE

Une page web = un corps. Sans squelette (HTML) rien ne tient ; sans vêtements (CSS) c'est austère ; sans muscles (JavaScript) rien ne bouge. Lumen3D ajoute un quatrième organe : un **œil** très rapide, le shader (section 2.3).

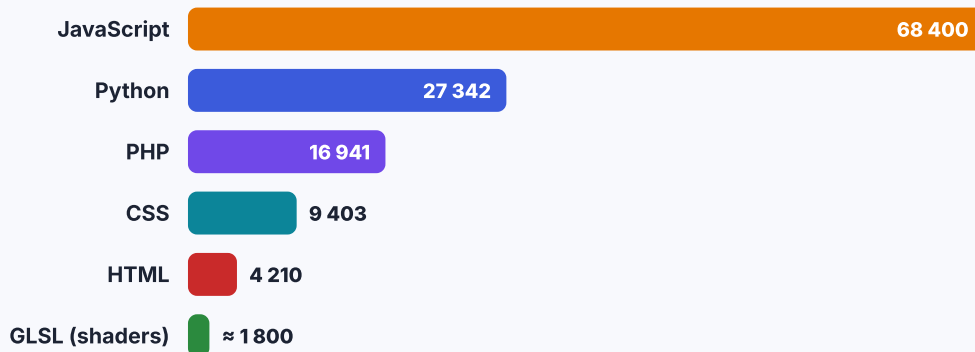
2.2 Le client : ce qui tourne dans votre navigateur

Le « client », c'est votre navigateur. Il reçoit les fichiers HTML, CSS et JavaScript, puis fait tout le travail visible.

- **JavaScript « vanilla »** : du JavaScript pur, organisé en petits modules, sans framework. Environ **68 400 lignes**.
- **Web Workers** : des « employés de l'ombre » du navigateur. Ils décodent les images WebP et font les calculs lourds pour que l'écran ne se fige jamais.
- **Bibliothèques** : Three.js (scène 3D), Lucide (icônes), Plotly (graphiques de suivi). Elles sont copiées dans le dossier `js/vendor/`.

Combien de lignes de code, par langage ?

Fichiers suivis par git, hors bibliothèques tierces, hors tests



≈ 1 800 lignes, écrites à l'intérieur de fichiers JavaScript

Les tests (≈ 26 000 lignes de JavaScript, 12 000 de Python, 3 000 de PHP) ne sont pas comptés.

▲ Les langages, par volume de code (la barre « shaders » est comprise dans le JavaScript).

? POURQUOI CE CHOIX ?

Pourquoi du JavaScript sans framework ? Pas d'étape de fabrication : on modifie un fichier, on recharge la page, c'est tout. Le code reste lisible par quelqu'un qui n'a pas appris un outil de plus, et il n'y a aucune dépendance qui puisse disparaître ou changer d'un mois à l'autre. Seul le panneau d'administration utilise les modules modernes (`import` / `export`).

2.3 GLSL : le programme minuscule exécuté des millions de fois

L'image 3D n'est pas une photo : elle est **calculée à chaque image affichée**. Pour chaque pixel de l'écran, on fait traverser le volume à un rayon imaginaire et on cumule la lumière rencontrée (chapitre 9). Cela représente des millions de petits calculs identiques.

La carte graphique est faite pour cela : elle les exécute en parallèle. Le petit programme qu'elle exécute est écrit en **GLSL**, le langage des **shaders**.

💡 UNE IMAGE POUR COMPRENDRE

Un shader, c'est une recette donnée à un million de cuisiniers. Chaque cuisinier prépare un seul pixel, tous en même temps, avec la même recette. Un processeur ordinaire serait un seul cuisinier très rapide : il perdrait la course.

- Environ **1 800 lignes** de GLSL, écrites dans des fichiers JavaScript (le rendu du volume, la coupe oblique, le recolorage des coupes).
- Le navigateur les confie à **WebGL2**, l'interface standard pour parler à la carte graphique.

🔧 POUR LES CURIEUX

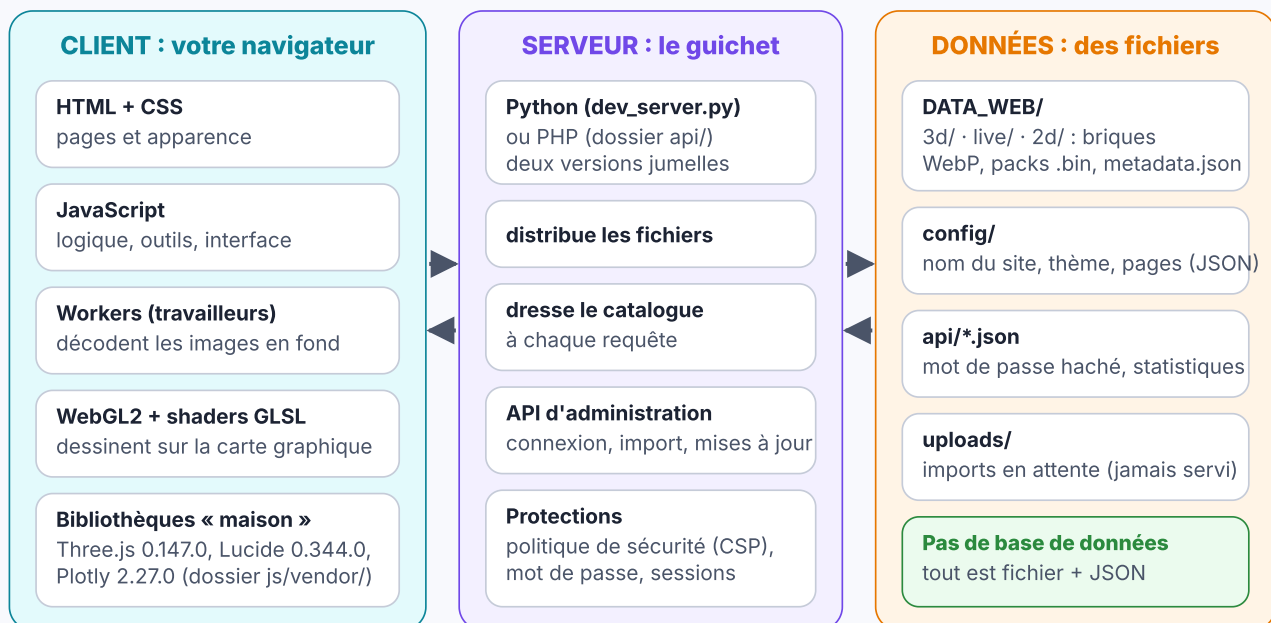
Le volume est stocké dans la carte graphique sous forme de **textures 3D**, découpées en emplacements de 64^3 voxels (66^3 en format 4). Le shader lit ces emplacements ; ceux qui sont vides sont sautés. Les détails sont aux chapitres 9 et 10.

2.4 Le serveur : un guichet, pas un cerveau

Le serveur est un programme qui répond aux demandes du navigateur. Ici, il fait **cinq métiers** :

MÉTIER	EN CLAIR
Distribuer des fichiers	pages, scripts, briques ; avec compression et mise en cache
Dresser le catalogue	la liste des jeux est recalculée à chaque requête en lisant les <code>metadata.json</code>
API d'administration	connexion, édition des fiches, statistiques
Importer	recevoir un dossier par morceaux, le vérifier, le publier
Se mettre à jour	télécharger une version signée, la contrôler, basculer, annuler si besoin

Qui fait quoi : client, serveur, données



Le serveur ne calcule jamais l'image 3D : il tend des fichiers, le navigateur fait le reste.
Flèches : requêtes HTTP du navigateur (→) et fichiers renvoyés (←).

▲ Les trois acteurs : le navigateur dessine, le serveur tend des fichiers, les données sont de simples fichiers.

1 2 3 4 MICRO-EXEMPLE

Le catalogue n'est pas un fichier. Vous copiez un dossier de jeu dans `DATA_WEB/3d/` : il apparaît tout de suite dans l'Explorateur. Rien à régénérer, parce que le serveur relit les fiches à chaque demande (il mémorise le résultat tant qu'aucun fichier n'a changé).

Deux serveurs jumeaux

Le même serveur existe **en deux langages**, avec la même interface vers le navigateur :

Python : `dev_server.py`

- recommandé, port 8080 ;
- bibliothèque standard seulement : rien à installer ;
- c'est lui qui sait se mettre à jour en un bloc, avec retour arrière automatique.

PHP : dossier `api/`

- pour les hébergements classiques d'université ou d'institut, qui offrent PHP mais pas Python ;
- mêmes fichiers, mêmes formats, même mot de passe : un import commencé sous l'un reprend sous l'autre ;
- PHP 8.1 ou plus.

POUR LES CURIEUX

Un petit serveur `fast_server.py` ne fait que distribuer des fichiers (sans administration) : il sert aux mesures de performance. Les deux serveurs complets sont maintenus identiques par des tests automatiques qui comparent leurs résultats.

2.5 Python : l'établi du laboratoire

Le **pipeline de préparation** est écrit en Python, parce que c'est la boîte à outils de la science : lire un fichier HDF5 (`h5py`), calculer sur de gros tableaux (`numpy`), filtrer (`scipy`), écrire des images (`Pillow`).

UNE IMAGE POUR COMPRENDRE

L'établi. Le pipeline est l'atelier où l'on prépare la pièce, hors de la vitrine. Le visiteur ne le voit jamais : il ne voit que la pièce finie, sur l'étagère (`DATA_WEB/`).

- Il tourne sur l'ordinateur du technicien, **une seule fois par jeu de données**.
- Dépendances figées : `h5py` 3.16.0, `numpy` 2.5.1, `scipy` 1.16.0, `Pillow` 11.1.0, `tqdm` 4.67.1.
- Python 3.10 ou plus. Détail des cinq étapes : chapitres 4 à 8.

2.6 Les formats de fichiers : des contenants

Un format est un **contenant** ; ce qui compte, c'est ce qu'on met dedans.

FORMAT	IMAGE MENTALE	RÔLE DANS LUMEN3D
JSON	des fiches cartonnées étiquetées	<code>metadata.json</code> (fiche d'un jeu), <code>manifest.json</code> (l'index des briques), configuration, traductions
WebP sans perte	une boîte hermétique pour une image	chaque brique ; les pixels sont restitués exactement
PNG	idem, format plus ancien	les plans entiers du format 2 (coupes rapides)
<code>.bin</code> (packs)	un classeur à anneaux	des centaines de briques mises bout à bout, pour moins de fichiers

FORMAT	IMAGE MENTALE	RÔLE DANS LUMEN3D
HDF5 (<code>.ims</code>)	un système de fichiers dans un fichier	le fichier Imaris d'origine : dossiers, tableaux et métadonnées dans un seul bloc
glTF / GLB	une maquette 3D en carton	<code>model.glb</code> , la surface exportée avec le suivi cellulaire
TIFF	le négatif de laboratoire	uniquement les téléchargements facultatifs et l'entrée des photographies

À SAVOIR

La photographie d'un jeu 2D est enregistrée en WebP **avec** une légère perte (qualité 90) par défaut. Les briques des volumes, elles, sont toujours sans perte.

2.7 Pourquoi tout est « auto-hébergé »

Three.js, Lucide et Plotly ne sont pas chargés depuis Internet : leurs fichiers sont **dans le dossier du site**. Voici pourquoi.

- 1 Hors ligne** : sur un réseau d'institut fermé, rien ne se casse.
- 2 Sécurité** : la politique de sécurité du navigateur (CSP) n'accepte que du code venant du site lui-même. Un script étranger serait **bloqué**.
- 3 Intégrité** : chaque bibliothèque est vérifiée par une empreinte (SRI) ; si un octet change, elle est refusée.
- 4 Confidentialité** : aucun tiers ne sait qui regarde quel jeu de données.

BIBLIOTHÈQUE	VERSION	RÔLE
Three.js	0.147.0	scène 3D, caméra, cube de départ des rayons
Lucide	0.344.0	icônes des boutons
Plotly	2.27.0	graphiques de suivi (chargée seulement à l'ouverture d'un graphique)

POURQUOI CE CHOIX ?

Pourquoi une version de Three.js aussi ancienne ? La 0.147.0 est figée à dessein : un seul fichier stable, qui ne change pas sous les pieds de la plateforme. Le calcul du volume, lui, est écrit sur mesure (le « lanceur de rayons » du chapitre 9) ; Three.js ne fournit que le cadre.

À SAVOIR

Seule exception : les polices de caractères (Google Fonts), chargées en arrière-plan. Elles ne servent qu'à l'apparence du texte.

2.8 Ce qu'il faut côté navigateur

- un navigateur récent avec **WebGL2** et les **textures 3D** ; sans cela, la plateforme refuse d'ouvrir un volume et affiche un message plutôt que de planter ;
- une carte graphique : plus elle a de mémoire, plus la qualité atteignable est élevée ;
- le thème clair/sombre, la langue et un filtre pour daltoniens sont dans l'en-tête de chaque page.

La plateforme **mesure** la mémoire disponible et s'y adapte :

CLASSE DE CARTE GRAPHIQUE	BUDGET MÉMOIRE RETENU
sans carte (rendu logiciel)	256 Mio
intégrée (portable)	0,5 à 2 Gio
dédiée	3 à 4 Gio
inconnue	1 à 2 Gio

À RETENIR

Si la qualité demandée dépasse le budget, la plateforme choisit le niveau plus petit et vous le dit : elle ne plante pas (chapitre 10).

2.9 Le dossier du projet

À quoi servent les dossiers que vous verrez si vous ouvrez le projet ? Un coup d'œil sur les deux premiers niveaux :

Le dossier du projet, vu de haut (deux niveaux)

Ce que voit le visiteur

*.html	une page = un fichier (accueil, viewer...)
css/	16 feuilles de style (l'habillage)
js/	tout le code du navigateur : core/ pages/ viewers/ components/ workers/ modules/ (outils) vendor/
lang/	traductions : en, fr, es, nl
config/	nom du site, thème, pages
assets/ logos/	images fixes
DATA_WEB/	les jeux de données publiés

Ce qui sert les fichiers

dev_server.py	serveur Python complet
api/	jumeau PHP du serveur
install.php	installateur en un fichier
.htaccess, router.php	règles Apache / PHP
upload_staging.py	moteur d'import
dataset_migrations.py	mise à jour des données
fast_server.py	serveur de test simple

Ce qui prépare les données

preprocess/	le pipeline Python (étapes 1 à 5)
SCRIPTS/	analyses du labo (suivi cellulaire)

Le pipeline tourne sur l'ordinateur du technicien, pas sur le serveur web.

Ce qui garantit la qualité

tests/	201 fichiers de tests automatiques
changelog/	une fiche par version (147)
tools/	fabrication des versions signées
marketplace/	catalogue de plugins signé

Créés sur un serveur en fonctionnement (jamais publics)

uploads/	imports en cours	backups/	sauvegardes avant mise à jour
logs/	journaux du serveur	secrets/	clés de signature (poste local)

Le serveur refuse de les servir : une adresse web ne peut jamais les atteindre.

▲ Le projet vu de haut : visiteur, serveur, préparation, qualité.

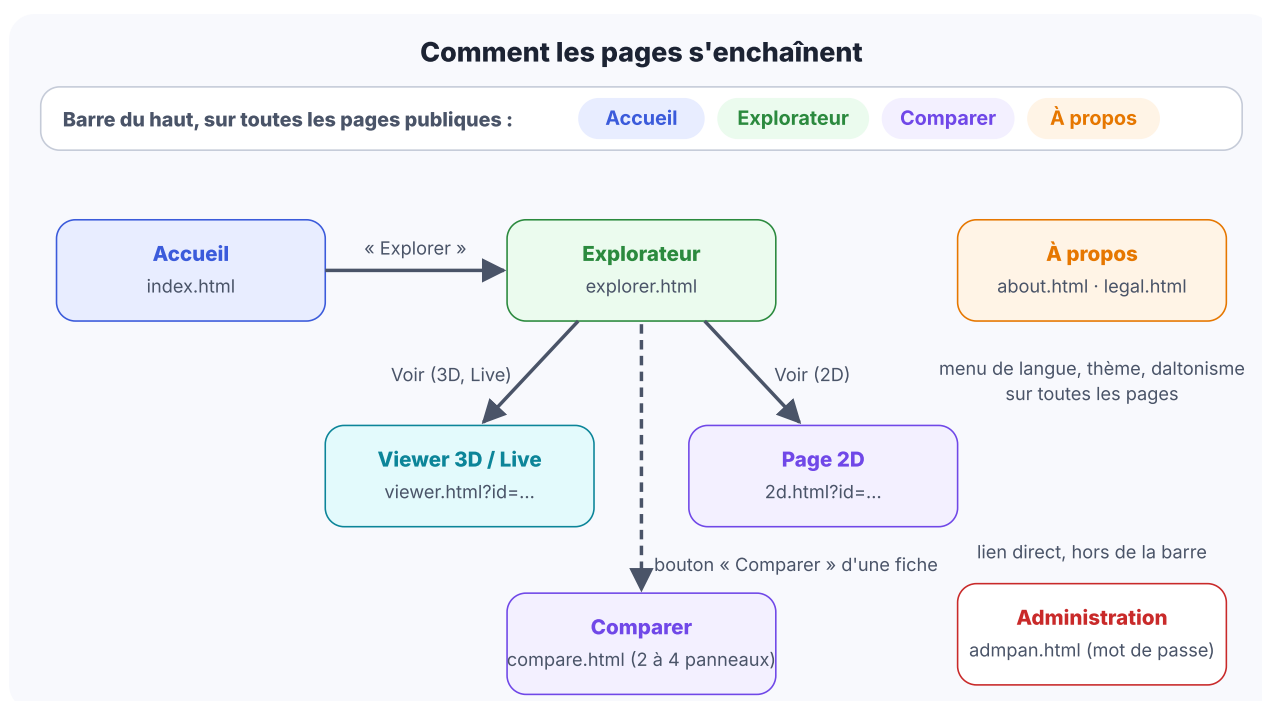
POUR ALLER PLUS LOIN

Les dossiers `DATA_WEB/` (données publiées) et `uploads/` (imports en attente) sont décrits au chapitre 7 et au chapitre 13. Les règles qui empêchent de les atteindre depuis une adresse web sont au chapitre 14.

3. Le tour des pages

- Le site public compte quatre grandes pages : **Accueil**, **Explorateur**, **Viewer** (3D et Live) et **Comparer**, plus la page **2D** et les pages **À propos** / **Mentions légales**.
- On passe de l'une à l'autre par la barre du haut ou par les boutons des fiches ; **l'adresse (URL)** d'une page retient quel jeu de données est ouvert, et même votre vue.
- Chaque section ci-dessous montre une vraie capture d'écran (jeu de démonstration) avec des repères numérotés.

3.1 La carte des pages



▲ Les pages publiques et leurs liens. Les flèches pleines sont des clics ; les pointillés, des liens directs.

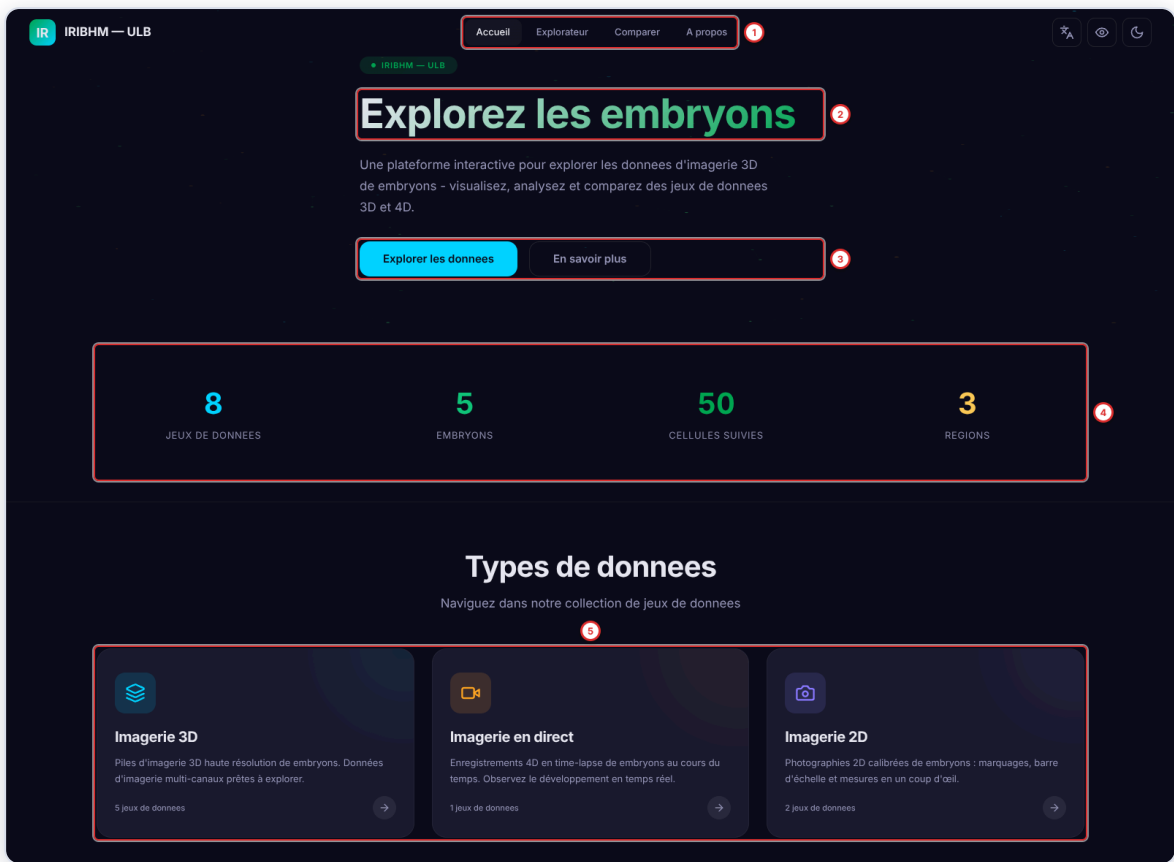
Toutes les pages publiques partagent la même barre du haut : le logo, les quatre liens, puis trois boutons à droite : **langue**, **filtre pour daltoniens** et **thème clair/sombre**.

i À SAVOIR

Les captures de ce document sont en français et en thème sombre. Le nom du site (« IRIBHM — ULB »), les couleurs et même les textes de l'accueil sont ceux de l'instance photographiée : une autre institution peut les avoir changés (chapitre 13).

3.2 L'Accueil

C'est la vitrine : un message d'accueil, quelques chiffres et trois cartes, une par type de données.



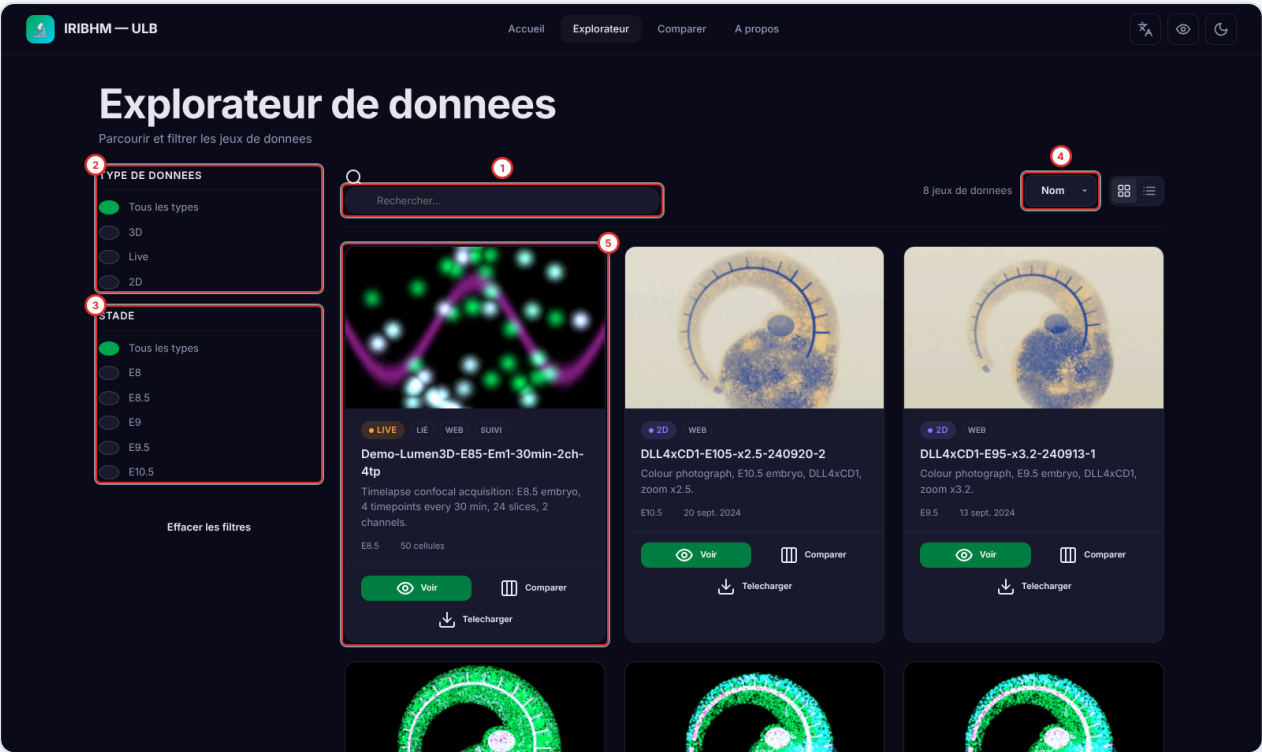
▲ Page d'accueil (index.html).

N	CE QUE C'EST
1	La barre de navigation : Accueil, Explorateur, Comparer, À propos.
2	Le titre : « Explorez les embryons ». Le mot « embryons » vient d'un réglage de l'instance.
3	Explorer les données ouvre l'Explorateur ; En savoir plus ouvre À propos.
4	Les compteurs : jeux de données, embryons, cellules suivies, régions. Ils se calculent sur le catalogue publié.
5	Les cartes de types : un clic ouvre l'Explorateur déjà filtré (<code>explorer.html?type=3d</code>).

En dessous, la section **Jeux de données en vedette** met en avant trois jeux choisis automatiquement (le suivi le plus riche, le volume le plus profond...). L'administrateur peut remplacer toute l'accueil par une page de son cru.

3.3 L'Explorateur

L'Explorateur liste les jeux publiés. Les jeux **masqués** n'y figurent pas.



▲ Explorateur de données (explorer.html).

N	CE QUE C'EST
1	La recherche : elle cherche dans le nom, la description, les noms de canaux, le stade et l'embryon.
2	Le filtre Type de données : 3D, Live ou 2D.
3	Le filtre Stade : une case par stade présent (E8, E8.5, E9...). Effacer les filtres remet tout à zéro.
4	Le tri (Nom, Date, Stade) et le choix grille / liste ; à côté, le nombre de résultats.
5	Une fiche : miniature, type, pastilles, nom, description, stade et date.

Sur chaque fiche, trois actions :

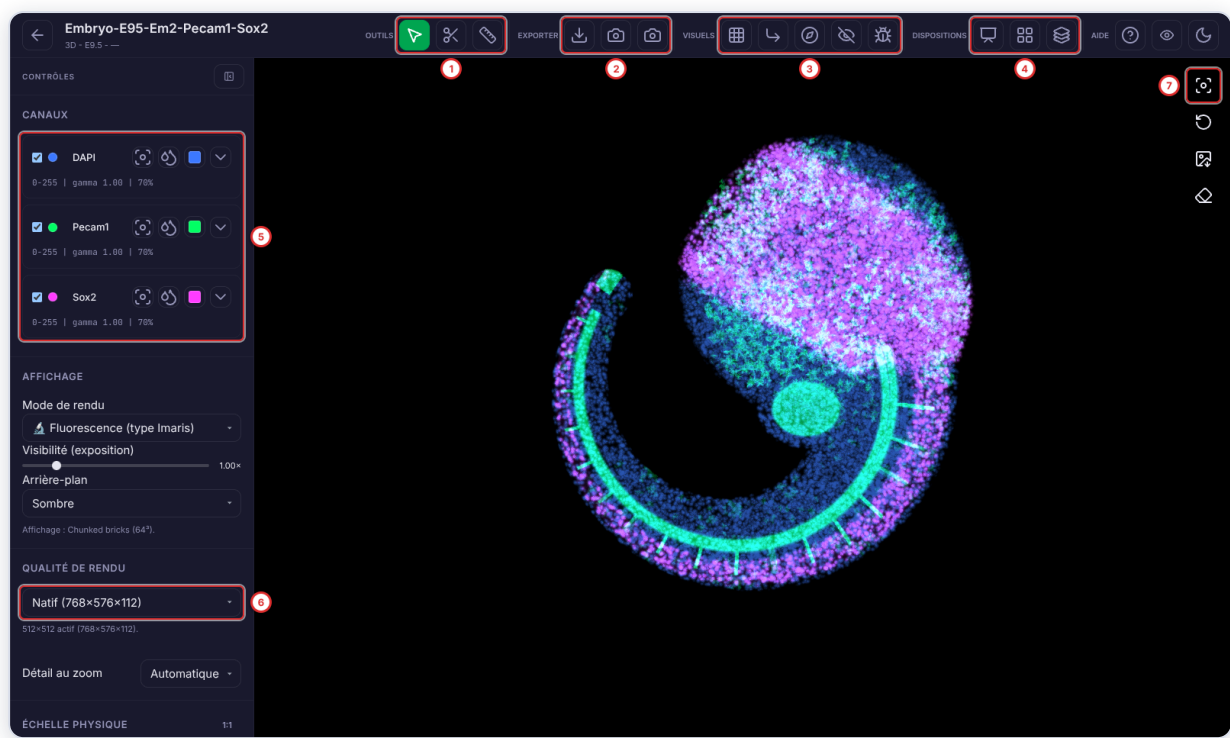
- **Voir** ouvre le viewer (ou la page 2D pour une photographie) ;
- **Comparer** ajoute le jeu à la page Comparer ;
- **Télécharger** ouvre le centre de téléchargement du jeu (originaux et exports, quand l'administrateur les a joints).

👉 ASTUCE

Les pastilles de la fiche en disent long : **Web** (affichable), **Brut** (fichier d'origine à télécharger), **Suivi** (cellules suivies), **Lié** (d'autres jeux sont rattachés, par exemple une photographie du même embryon).

3.4 Le viewer 3D

C'est le cœur de la plateforme. Ouvrez un jeu 3D : le volume apparaît d'abord flou puis s'affine.



▲ Le viewer sur un jeu 3D (viewer.html?id=3d/...).

N	CE QUE C'EST
1	Groupe Outils : Naviguer (par défaut), Couper à travers le volume , Mesurer la distance .
2	Groupe Exporter : Centre de téléchargement , Capture d'écran .
3	Groupe Visuels : grille, axes, axes d'orientation, masquer le volume, débogage des briques.
4	Groupe Dispositions : Mode présentation , Décomposer par canal , Explorateur Z-Stack .
5	Panneau Canaux : une ligne par canal (case, couleur, histogramme réglable).
6	Qualité de Rendu : 512, 1024 ou natif. Le texte dessous donne la qualité réellement active.
7	Quatre boutons sur le volume : centrer l'échantillon, réinitialiser la vue, exporter la vue en PNG, réinitialiser l'espace de travail.

Bouger : glisser = tourner ; **Maj** + glisser (ou bouton droit) = déplacer ; molette = zoomer ; deux doigts = déplacer et pincer.

Sous les canaux, le panneau **Affichage** règle le **Mode de rendu** (par défaut « Fluorescence »), la **Visibilité (exposition)** et l'**Arrière-plan**. Plus bas, **Détail au zoom** charge des briques plus fines quand vous zoomez, et **Échelle Physique** étire l'affichage en Z.

i À SAVOIR

Les boutons dépendent des **plugins installés** sur votre instance. Chapitre 12 : chaque outil ; chapitre 11 : couleurs et histogrammes.

👉 ASTUCE

Partager votre vue. L'adresse du navigateur se termine bientôt par `#state=...` : elle contient caméra, canaux, outil actif et mesures. Copiez-la, la personne qui l'ouvre peut

Ouvrir la vue enregistrée .

3.5 Le viewer Live : la ligne de temps

Un jeu Live s'ouvre dans la même page. Deux choses s'ajoutent : la **ligne de temps** en bas, et, s'il a été suivi, la couche **Points de suivi** dans le panneau des canaux. Sur un vrai time-lapse, la lecture **attend** que l'instant demandé soit chargé : vous voyez des images terminées, à la vitesse où elles arrivent (chapitre 10).



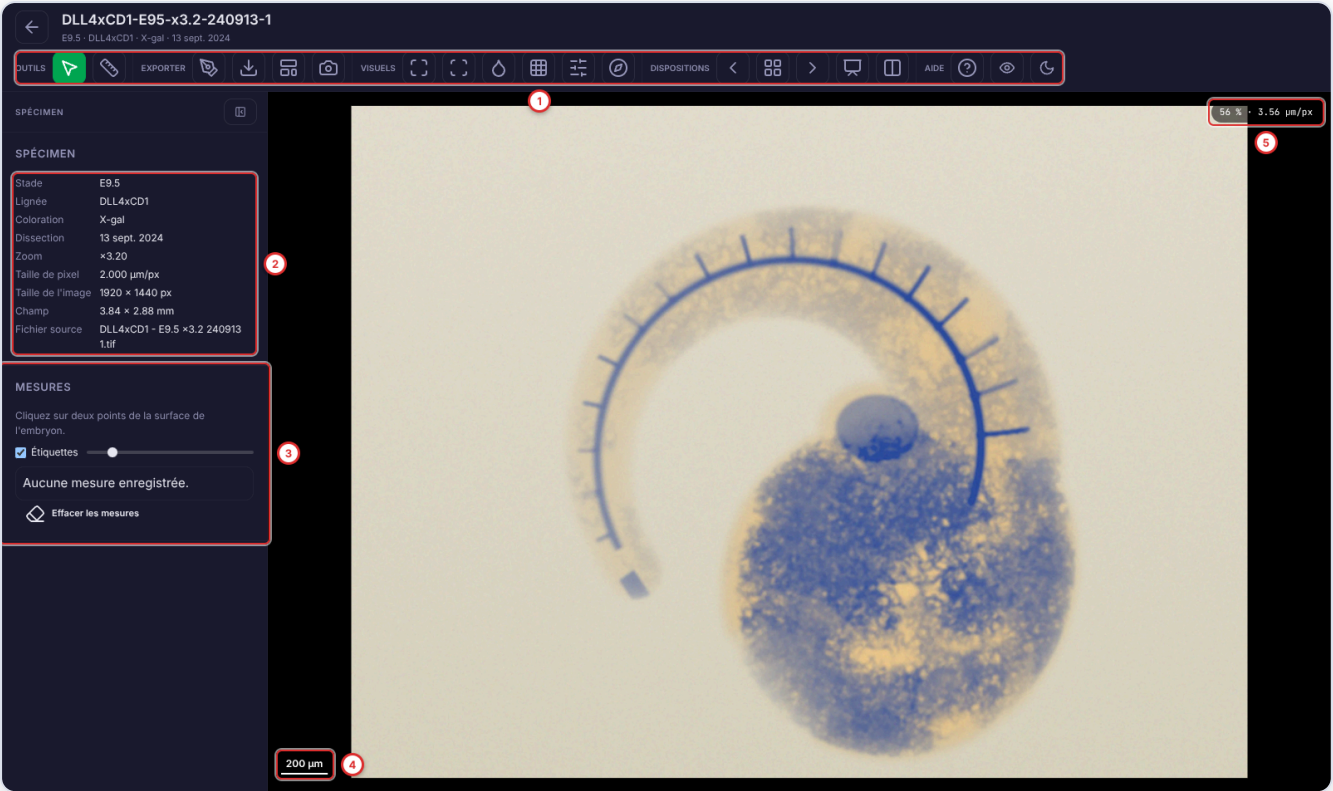
▲ Le viewer sur un jeu Live (jeu de démonstration à 4 instants, 50 cellules suivies). Le menu des outils est ouvert.

N CE QUE C'EST

- 1** Le menu des outils. Quand le titre est long ou la fenêtre étroite, les groupes se replient derrière le bouton . Y figurent les outils de suivi (inspecter une cellule, distance entre cellules, graphiques, trajectoires, surface).
- 2** La couche **Points de suivi** : taille et opacité des sphères, mitoses, fusions, couleur par région.
- 3** Lecture / pause et vitesse (en images par seconde ; chaque clic change de vitesse).
- 4** Le compteur : instant actuel / dernier instant (numérotés à partir de 0).
- 5** Le curseur de temps ; la bande claire indique les instants déjà en mémoire.

3.6 La page 2D

Une photographie de stéréomicroscope, calibrée en micromètres. Pas de canaux ni de 3D : on déplace, on zoome, on mesure.



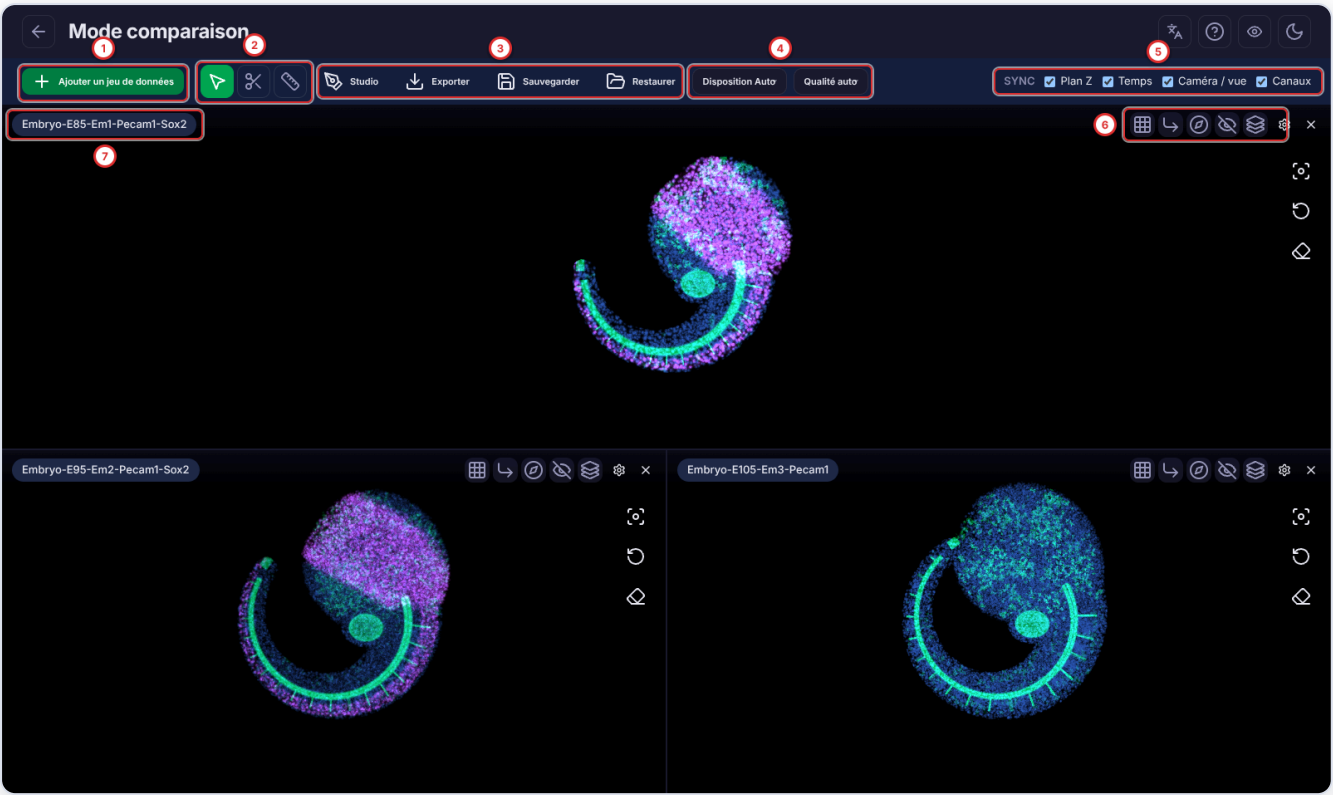
▲ La page 2D (2d.html?id=2d/...).

N	CE QUE C'EST
1	La barre d'outils : Naviguer, Mesurer ; Studio, téléchargement, planche, capture ; ajuster, résolution native 1:1, isoler le marquage ; navigation dans la collection.
2	La fiche du Spécimen : stade, lignée, coloration, dissection, zoom, taille de pixel, champ.
3	La section Mesures : cliquer deux points donne une distance en µm.
4	La barre d'échelle (ici 200 µm), qui suit le zoom.
5	Le zoom affiché, et la taille d'un pixel d'écran en µm.

- Molette = zoom, glisser = déplacer, double-clic = ajuster à l'écran.
- **Parcourir la collection** ouvre une planche-contact de toutes les photographies ; ← et → passent à la suivante.
- La page affiche d'abord un aperçu léger, puis la résolution native dès qu'elle est prête.

3.7 Comparer

Jusqu'à **quatre panneaux** côte à côte. Chaque panneau est un **vrai viewer** (ou une vraie page 2D) intégré : tout ce que vous savez faire dans le viewer, vous le faites ici, et on peut mélanger volumes et photographies.



▲ Comparer trois embryons (E8.5, E9.5, E10.5 : jeu de démonstration).

N	CE QUE C'EST
1	Ajouter un jeu de données : ouvre une fenêtre de choix avec recherche et filtres par type.
2	Les outils communs (Naviguer, Couper, Mesurer...) : un clic s'applique à tous les panneaux.
3	Studio (une figure composée de tous les panneaux), Exporter , Sauvegarder et Restaurer l'espace de travail.
4	Disposition Auto (colonnes, lignes, grille) et Qualité auto .
5	SYNC : ce qui est synchronisé entre panneaux, Plan Z , Temps , Caméra / vue , Canaux . Tout est coché par défaut.
6	Les boutons propres à un panneau (grille, axes, orientation, volume, Z-stack), son réglage et sa fermeture.
7	Le nom du jeu du panneau.

- **Caméra / vue** coché : tourner un embryon fait tourner les autres, même calibrés différemment. Pour des time-lapses de longueurs différentes, on synchronise la **fraction** du temps écoulé.
- **Qualité auto** : les panneaux s'ouvrent à 512, puis montent un par un (jusqu'à 1024 pour deux volumes, 512 au-delà) pour tenir dans une mémoire graphique commune.

- Un seul volume à plusieurs canaux ? Le bouton **Décomposer** le clone en un panneau par canal.
- L'adresse de la page (`#state=...`) retient les panneaux et leur disposition : copiez-la pour partager la comparaison.

3.8 À propos et Mentions légales

À propos présente le projet. Son contenu par défaut est une page modifiable par l'administrateur (éditeur de pages, chapitre 13). **Mentions légales** affiche le texte juridique saisi dans l'onglet **Mentions légales** ; le lien du pied de page n'apparaît que s'il est activé.



▲ La page À propos.

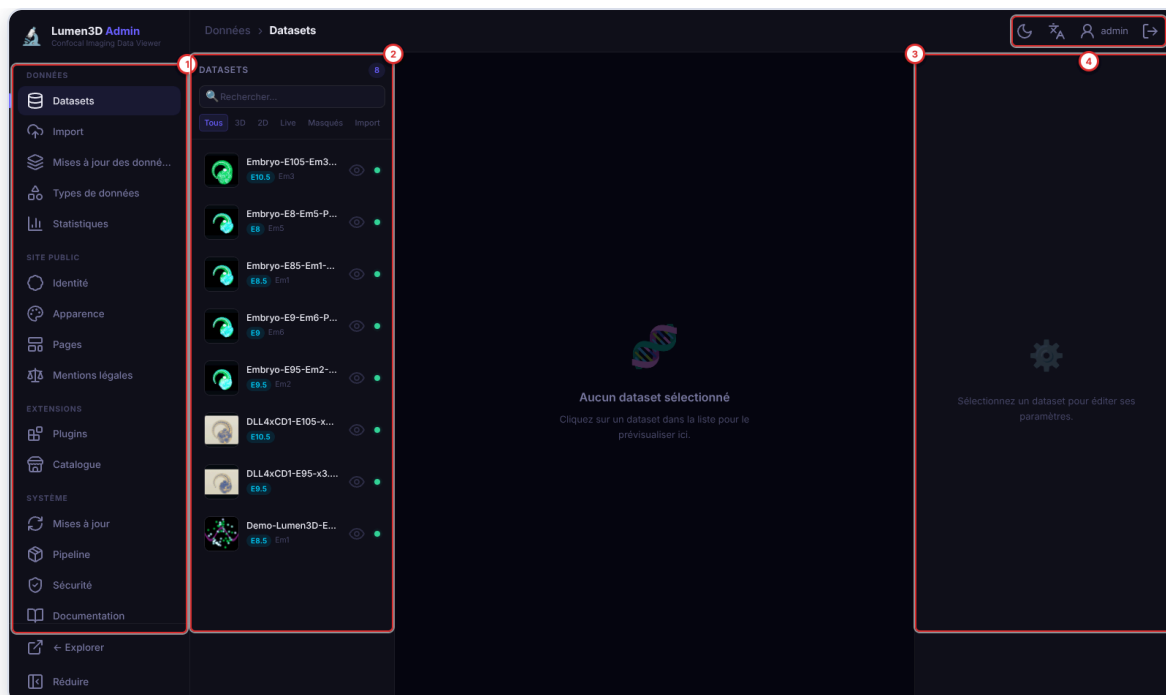
i À SAVOIR

Sur une installation neuve, les mentions légales sont un modèle à compléter (nom de l'organisation, adresse, hébergeur) : ce sont des champs entre crochets que l'administrateur remplit.

D'autres pages peuvent exister : l'administrateur peut créer des **pages personnalisées**, ouvertes par `page.html?slug=<nom>` et ajoutées à la barre du haut.

3.9 L'administration : un aperçu

L'administration vit dans `admpan.html` et demande un mot de passe. Elle n'est pas dans la barre publique. Voici seulement son allure ; le détail est au chapitre 13.



▲ Le panneau d'administration, onglet Datasets.

N	CE QUE C'EST
1	Le menu : quatre familles, Données , Site public , Extensions , Système .
2	La liste des jeux (publiés, masqués, en cours d'import) avec recherche et filtres.
3	La zone d'aperçu du jeu choisi : le viewer y est affiché tel que le verront les visiteurs.
4	Le thème, la langue et la déconnexion.

3.10 Les adresses à connaître

Une adresse (URL) peut tout dire d'une page. Les plus utiles :

ADRESSE	EFFET
<code>viewer.html?id=3d/<dossier></code>	ouvre un volume 3D (même forme pour <code>live/...</code>)
<code>2d.html?id=2d/<dossier></code>	ouvre une photographie
<code>explorer.html?type=3d</code>	Explorateur déjà filtré (<code>3d</code> , <code>live</code> ou <code>2d</code>)
<code>compare.html?add=<id>&add=<id></code>	Comparer avec ces jeux déjà posés (4 au plus)
<code>...#state=...</code>	la vue exacte (caméra, canaux, outils, mesures) : à copier pour partager
<code>page.html?slug=<nom></code>	une page personnalisée

 ATTENTION

Les paramètres `hideHeader` et `panelIndex` servent à la page Comparer pour intégrer des pages sans leur en-tête ; ne les utilisez pas pour un usage normal.

 ASTUCE

Raccourcis clavier (viewer) : `V` ou `Échap` Naviguer ; `C` couper ; `M` mesurer ; `I` inspecter une cellule ; `D` distance entre cellules. Page 2D : `F` ajuster, `B` planche, `←` `→` photographie précédente / suivante.

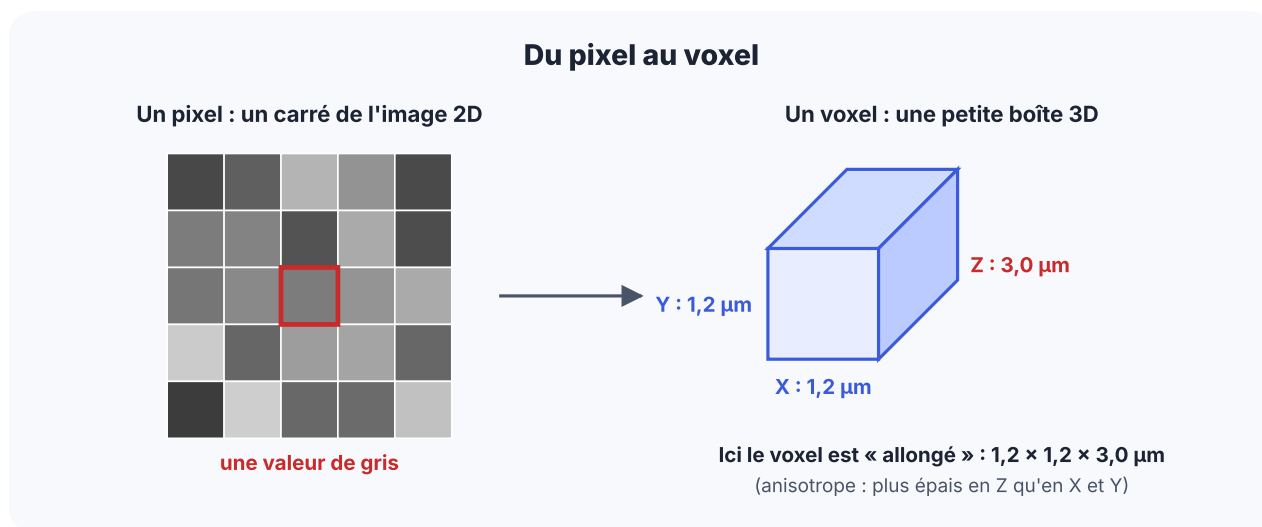
4. Du microscope au fichier Imaris

- Une image 3D est une **pile de coupes** ; chaque case 3D s'appelle un **voxel**, et sa taille réelle en micromètres compte.
- Le fichier **.ims** d'Imaris est un « système de fichiers dans un fichier » (HDF5) : le pipeline y lit seulement la taille de l'image, la calibration, les noms de canaux, l'horloge et les voxels en pleine résolution.
- **L'étape 1** du pipeline écrit un petit fichier **meta.json** : elle ne calcule rien d'autre que la taille du voxel, et n'invente jamais une calibration absente.

4.1 Du pixel au voxel

Une photo numérique est une grille de **pixels** : chaque petit carré porte une valeur de gris (ou de couleur). Un microscope confocal ne fait pas une photo, mais une série de photos prises à des profondeurs différentes.

Un point de cette série s'appelle un **voxel** : un pixel qui a aussi une épaisseur. C'est une petite boîte 3D, et c'est l'unité de base de tout ce document.



▲ Du pixel au voxel. Ici le voxel mesure $1,2 \times 1,2 \mu\text{m}$ dans le plan et $3,0 \mu\text{m}$ en profondeur : c'est la calibration du jeu de démonstration (embryon synthétique).

💡 UNE IMAGE POUR COMPRENDRE

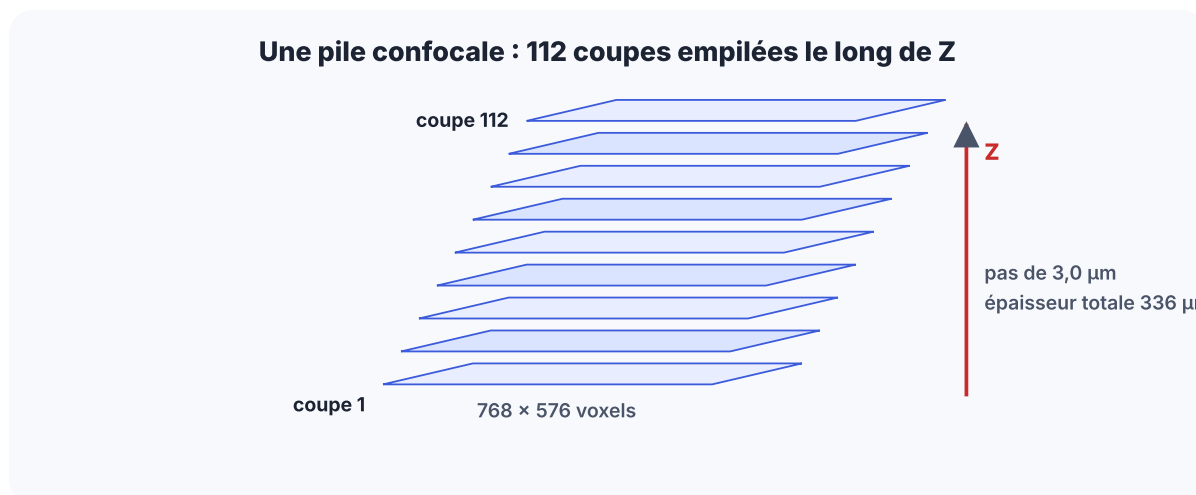
Pensez à une **boîte de Lego** : une plaque plate de plots est une image 2D (pixels). Empilez 112 plaques et chaque plot devient une brique 3D : un voxel. Si vos plaques sont plus épaisses que la largeur d'un plot, vos briques sont « allongées » en hauteur.

i À SAVOIR

Un voxel **anisotrope** (plus épais en Z qu'en X et Y) est la règle en confocal : changer de profondeur coûte du temps de balayage. Le pipeline en tient compte à chaque étape (voir chapitre 6 pour la pyramide, chapitre 9 pour l'affichage).

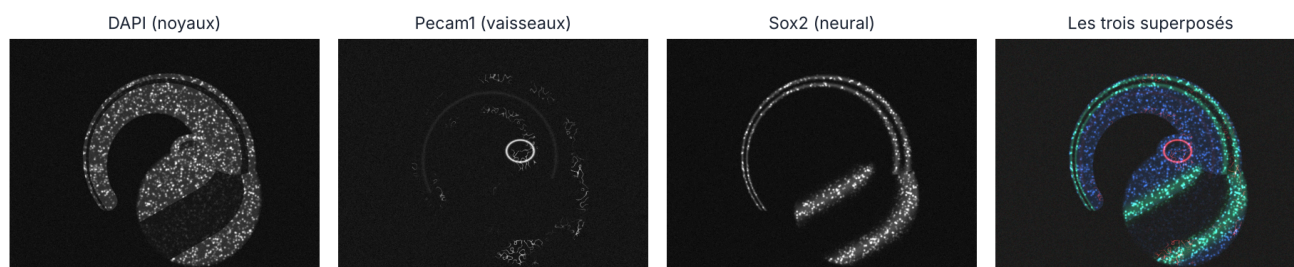
4.2 Une pile de coupes, plusieurs canaux

Le microscope balaye l'échantillon coupe par coupe, le long de l'axe **Z**. Le jeu de démonstration compte **112 coupes** de 768×576 voxels, espacées de $3,0 \mu\text{m}$: l'embryon est donc épais de $112 \times 3,0 = 336 \mu\text{m}$.



▲ Une pile confocale : 112 coupes empilées le long de Z (schéma, chiffres du jeu de démonstration).

Un **canal** est une pile complète enregistrée pour un seul fluorophore, c'est-à-dire une seule couleur d'émission. Le jeu de démonstration a **trois canaux** : DAPI (les noyaux), Pecam1 (les vaisseaux) et Sox2 (le tissu neural).



Coupe z = 56 du jeu de démonstration

▲ La même coupe ($z = 56$) vue dans chacun des trois canaux, puis superposée. Les images viennent du jeu de démonstration (embryon synthétique) ; chaque canal est étiré entre 0 et son 99,9e percentile pour l'affichage.

⌚ EN 30 SECONDES

- 1 canal = 1 pile de coupes = 1 fluorophore.
- Les canaux ne sont **pas mélangés** dans le fichier : chacun a son propre tableau 3D.
- Le mélange de couleurs se fait plus tard, dans le navigateur (chapitre 11).

4.3 Combien de niveaux de gris ? 8 ou 16 bits

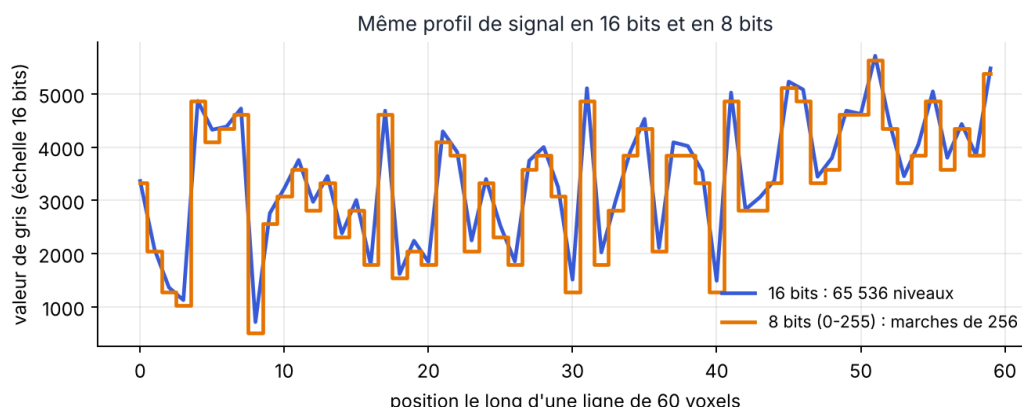
Chaque voxel porte un **entier** : l'intensité de lumière mesurée. Le nombre de bits dit combien de valeurs sont possibles.

PROFONDEUR	VALEURS POSSIBLES	PLAGE
8 bits	$2^8 = 256$	0 à 255
16 bits	$2^{16} = 65\,536$	0 à 65\,535

Les fichiers Imaris du laboratoire sont en 8 ou 16 bits selon l'acquisition ; le pipeline lit les deux. Le jeu de démonstration est en **16 bits** (type `uint16`).

1 2 3 4 MICRO-EXEMPLE

Deux taches de fluorescence valent **1 000** et **1 200** sur une échelle 16 bits : 200 niveaux d'écart, très mesurables. Si l'on divisait simplement par 256 pour passer à 8 bits, elles deviendraient **3** et **4** : un seul cran d'écart. C'est pourquoi le pipeline ne divise pas bêtement : il règle d'abord une « fenêtre » adaptée (chapitre 5).



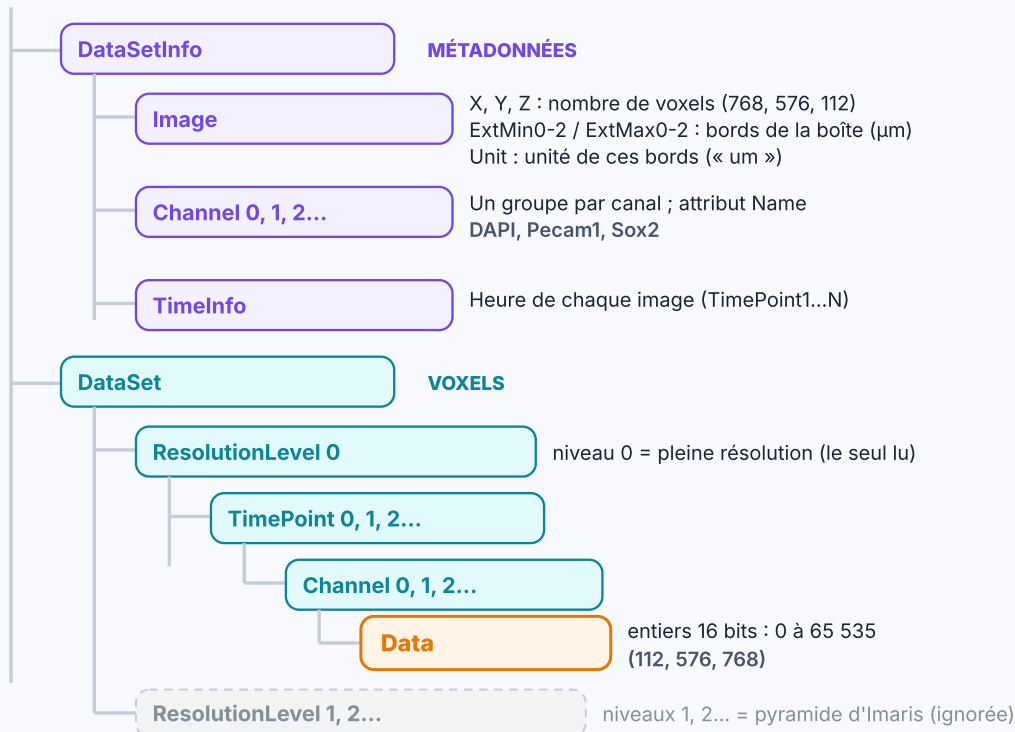
▲ Un vrai profil de 60 voxels (canal Pecam1, jeu de démonstration) : la courbe bleue garde toutes ses nuances en 16 bits ; en orange, les mêmes valeurs rangées dans des marches de 256. Ici le signal est fort et les marches restent discrètes ; sur un signal faible, elles écrasent les détails.

4.4 Un .ims est un « disque dur dans un fichier »

Un fichier **.ims** est un conteneur au format **HDF5**. Imaginez un dossier de dossiers : on y trouve des groupes (les dossiers), des attributs (de petites étiquettes de texte) et des tableaux (les voxels).

L'intérieur d'un fichier .ims (HDF5) : ce que le pipeline lit

fichier .ims



▲ Les chemins réellement lus par le pipeline dans le .ims du jeu de démonstration. En violet : les métadonnées (étape 1). En vert : les voxels (étape 2).

1 2 3 4 MICRO-EXEMPLE

Dans le fichier de démonstration, `DataSetInfo/Image` contient les attributs `X = 768`, `Y = 576`, `Z = 112`, `ExtMin0 = 0.000`, `ExtMax0 = 921.600` et `Unit = um`. Tout est écrit sous forme de **texte** : le pipeline le convertit en nombres.

🔧 POUR LES CURIEUX

Les attributs HDF5 sont parfois stockés comme tableaux d'octets ASCII ; `attr_str` (étape 1) les recolle en texte. Les tableaux de voxels sont compressés (gzip) par blocs de $16 \times 64 \times 64$ voxels : le pipeline lit toujours des sous-boîtes, jamais plus que la taille déclarée.

4.5 La pyramide d'Imaris : ignorée

Imaris range souvent, à côté du niveau 0 (pleine résolution), des versions réduites : `ResolutionLevel 1`, `2` ... C'est sa propre pyramide, pensée pour sa propre fenêtre d'affichage. Le jeu de démonstration n'en contient pas : seul le niveau 0 existe. Un fichier réel du laboratoire en a plusieurs.

Le pipeline **n'utilise que le niveau 0**, et jamais les autres, pour le volume.

? POURQUOI CE CHOIX ?

La pyramide d'Imaris est calculée sur les données **brutes**, avec ses propres tailles de blocs : elle ne correspond ni au découpage en briques de Lumen3D, ni au nettoyage du chapitre 5. Mieux vaut nettoyer le niveau 0 puis fabriquer **sa propre pyramide** à partir du résultat (chapitre 6) : chaque niveau est alors cohérent avec les autres.

4.6 L'étape 1 : lire les métadonnées

L'étape 1 (`1-ims_metadata.py`) ouvre le `.ims`, lit quelques attributs et écrit `meta.json` . Elle ne touche aucun voxel.

CHAMP DE META.JSON	CE QUE C'EST	VALEUR (DÉMO)
<code>width</code> , <code>height</code> , <code>depth</code>	Nombre de voxels en X, Y, Z	768, 576, 112
<code>n_channels</code>	Nombre de canaux	3
<code>n_timepoints</code>	Nombre d'images dans le temps	1
<code>voxel_size</code>	Taille d'un voxel en μm (x, y, z)	1,2 ; 1,2 ; 3,0
<code>extent</code>	Boîte de l'image en μm (min, max)	0 → 921,6 / 691,2 / 336,0
<code>timestamps</code>	Heure de chaque image	2026-03-11 10:00:00
<code>time_interval_minutes</code>	Intervalle médian entre images	vide (une seule image)
<code>channel_names</code>	Noms des canaux	DAPI, Pecam1, Sox2

i À SAVOIR

Le nombre d'images détermine le **type** de dataset : une seule image donne un dataset `3d` ; plusieurs donnent un `live` (séquence temporelle). C'est l'étape 1 qui tranche.

Les noms de canaux sont nettoyés : tout ce qui suit un caractère nul est supprimé, et un nom vide ou du type « ch1 » devient « Channel 1 », « Channel 2 »... Les couleurs ne viennent pas du `.ims` : elles sont attribuées plus tard, avec une palette fixe (chapitre 8).

4.7 La calibration : la taille du voxel

Le fichier ne donne pas la taille du voxel directement. Il donne la **boîte** (ExtMin / ExtMax) et le nombre de voxels. Le pipeline divise.

Calibration : la taille du voxel se déduit de la boîte

$$\text{voxel} = (\text{ExtMax} - \text{ExtMin}) \div N$$

X

La boîte, en μm

921,6 μm

÷ nombre de voxels

768

= 1,2 μm

Y

La boîte, en μm

691,2 μm

÷ nombre de voxels

576

= 1,2 μm

Z

La boîte, en μm

336,0 μm

÷ nombre de voxels

112

= 3,0 μm

▲ Taille du voxel = $(\text{ExtMax} - \text{ExtMin}) \div N$, axe par axe. Valeurs du fichier de démonstration (embryon synthétique).

MICRO-EXEMPLE

Sur X : $(921,6 - 0) \div 768 = 1,2 \mu\text{m}$. Sur Z : $(336,0 - 0) \div 112 = 3,0 \mu\text{m}$. On divise par N (et non par $N - 1$), choix voulu par le code.

Si l'unité du fichier n'est pas des μm , elle est d'abord convertie :

UNITÉ DANS LE FICHIER	FACTEUR VERS MM
um , μm , micron ...	$\times 1$
nm	$\times 0,001$
mm	$\times 1\,000$
m	$\times 1\,000\,000$

ATTENTION

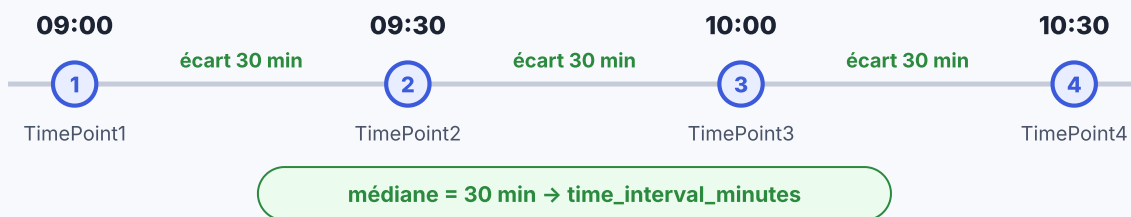
La calibration n'est jamais inventée. Si `ExtMin/ExtMax` manquent ou si l'unité est inconnue, les trois tailles de voxel valent 0 et `extent` est vide ; la plateforme enregistre `calibrationStatus = metadata-missing` : aucune barre d'échelle ne sera inventée.

4.8 L'horloge d'une séquence temporelle

Pour un film (plusieurs images), Imaris note l'heure de chaque image : `TimePoint1` , `TimePoint2` ... (la numérotation commence à 1 ici, alors que les tableaux de voxels commencent à 0).

L'étape 1 convertit ces textes en dates, puis calcule l'**intervalle médian** entre images successives. La **médiane** (valeur du milieu) résiste à un retard ponctuel : une image décalée de quelques minutes ne fausse pas l'intervalle annoncé.

Horloge d'une acquisition : l'intervalle est la médiane des écarts



▲ Exemple sur le jeu de démonstration en séquence (4 images, une toutes les 30 minutes). L'intervalle médian, 30 minutes, devient `time_interval_minutes`.

📌 À RETENIR

- Le voxel = une case 3D ; sa taille vient de $(\text{ExtMax} - \text{ExtMin}) \div N$.
- Seul le niveau 0 du .ims est lu ; la pyramide d'Imaris est ignorée.
- Les noms de canaux, l'horloge et la calibration sont lus ; **rien n'est deviné.**

5. Nettoyer l'image : retirer le bruit de fond

- Le pipeline traite **chaque canal séparément** en cinq temps : mesurer le fond, mesurer le blanc, protéger le signal, lisser le fond, puis convertir de 16 à 8 bits.
- Tout est calculé **à partir de voxels de référence** (les 8 coins, un voxel sur 4) : aucune valeur n'est choisie à la main, et rien n'est « embelli » ni inventé.
- Résultat : un fond **exactement noir (0)**, un signal intact, et des valeurs 8 bits **relatives** à chaque canal. Le .ims original reste la seule source quantitative.

Ce chapitre suit l'**ordre exact du code** (`2-image_processor.py`). Toutes les images viennent du jeu de démonstration (embryon synthétique), canal DAPI, coupe z = 56 ; elles sont calculées avec les fonctions du pipeline lui-même.

Le nettoyage d'un canal, dans l'ordre du code



▲ Les cinq temps du nettoyage d'un canal.

5.1 Pourquoi nettoyer ?

Une caméra de microscope n'est jamais parfaitement noire : même sans échantillon, chaque voxel affiche un petit nombre aléatoire (le **bruit de caméra**). Dans le jeu de démonstration, il oscille autour de 2 600 sur une échelle de 0 à 65 535.

Dans le navigateur, le rendu **cumule** la lumière le long de chaque rayon. Un fond de 2 600 répété sur 112 coupes finit en brouillard gris qui masque l'embryon.

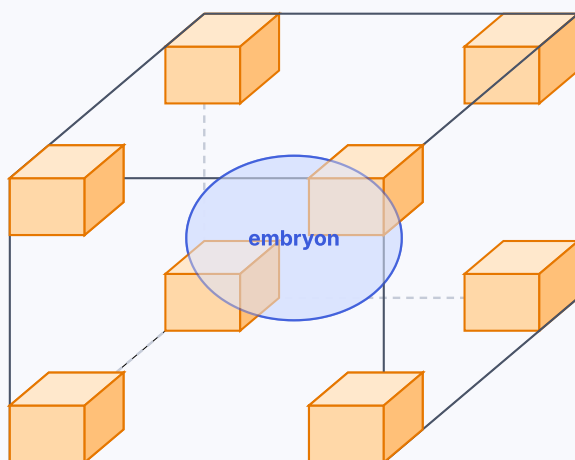
💡 UNE IMAGE POUR COMPRENDRE

C'est une **salle de cinéma avec un écran jamais tout à fait noir** : les lumières de secours éclairent la salle. Le nettoyage règle le « noir » de l'écran sur le niveau de la salle, pour que seul le film reste visible.

5.2 Étape A : mesurer le fond (`bg_floor`)

Où trouver du fond pur, sans embryon ? **Dans les coins** du volume : l'embryon est au centre, rien n'y touche. Le pipeline prélève **8 cubes**, un par coin, de **32 voxels de côté au maximum**.

Les 8 coins : du fond de caméra pur, sans embryon



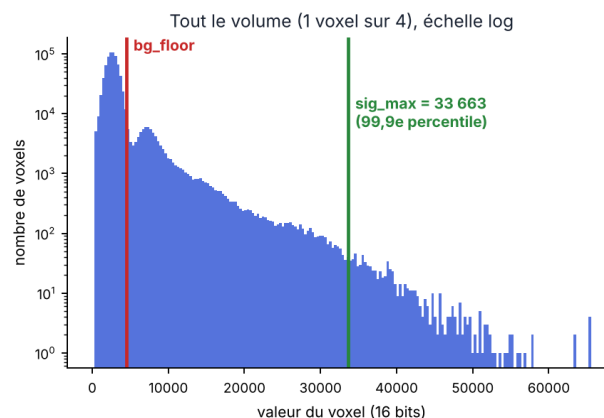
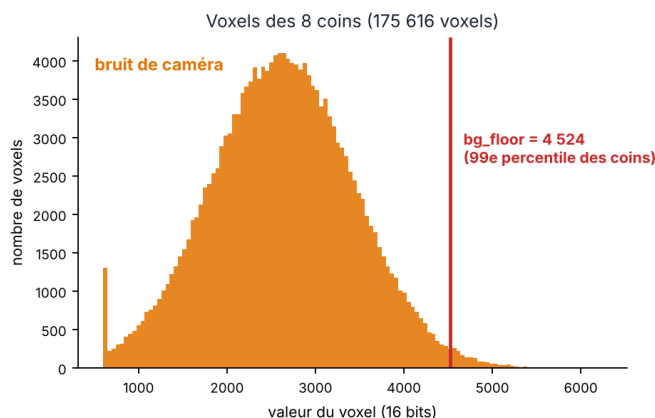
 **8 petits cubes**

jusqu'à $32 \times 32 \times 32$ voxels chacun
(ici 28 : un quart de l'épaisseur $Z = 112$)

▲ Les 8 cubes de coin (schéma). Dans le jeu de démonstration, la pile ne compte que 112 coupes : le côté vaut $\min(32, X \div 4, Y \div 4, Z \div 4) = 28$ voxels, soit $8 \times 28^3 = 175\,616$ voxels de référence.

Puis : **bg_floor = 99e percentile de ces voxels**. Le percentile est la valeur sous laquelle se trouvent 99 % des voxels. Ici :

- la plupart des voxels de coin valent entre 1 500 et 4 000 ;
- 99 % sont **inférieurs à 4 524** ;
- c'est **bg_floor** : le plafond du bruit de caméra.



▲ À gauche : les 175 616 voxels des 8 coins forment une « cloche » de bruit ; la ligne rouge est leur 99e percentile. À droite : tout le volume (1 voxel sur 4, échelle logarithmique) ; le gros pic à gauche est le fond, la longue traîne est le signal (jeu de démonstration, canal DAPI).

MICRO-EXEMPLE

Par construction, 1 % des voxels de coin dépassent **bg_floor** (mesuré : 1,0005 %). Le plus fort voxel de coin vaut 6 238. Un voxel de valeur 3 000 est donc du bruit probable ; un voxel de valeur 20 000 est du signal presque certain.

? POURQUOI CE CHOIX ?

Pourquoi les coins ? Parce que ce sont des voxels sans embryon, sûrs. Le code a testé d'autres règles (5e puis 20e percentile, 10e percentile des coins) avant de retenir le 99e percentile des 8 coins : il couvre presque tout le bruit sans toucher le signal.

⚠ ATTENTION

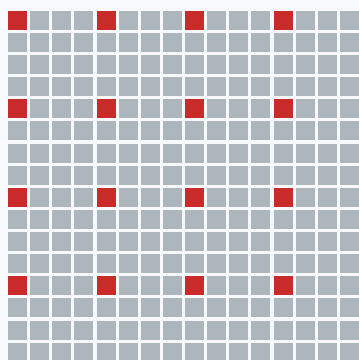
Si un coin touche l'échantillon (coupe serrée, mosaïque de tuiles), `bg_floor` peut être trop haut et couper du signal faible. Le pipeline l'**avertit** dans son journal (« corner noise well above the volume median ») mais ne change rien : il ne corrige pas en silence.

5.3 Étape A (suite) : mesurer le blanc (`sig_max`)

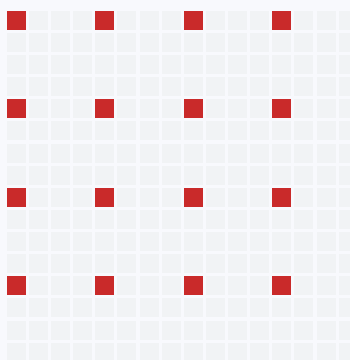
Pour le haut de l'échelle, on regarde **tout le volume**, mais en ne lisant **qu'un voxel sur 4** dans chaque direction (indices 0, 4, 8...). Ça économise 63 voxels sur 64 sans changer la statistique.

Le point blanc : on ne regarde qu'un voxel sur 4 dans chaque direction

Tous les voxels (X, Y, Z)



voxels échantillonnés : indices 0, 4, 8, 12...



$$1/4 \times 1/4 \times 1/4 = 1/64 \text{ des voxels}$$

$192 \times 144 \times 28 = 774\,144$ voxels classés (sur 49 545 216)

▲ Échantillonnage du point blanc : 1/64 des voxels suffisent.

`sig_max` = 99,9e percentile de ces voxels : on **sature** (met à blanc) les 0,1 % les plus brillants. Dans le jeu de démonstration : `sig_max` = 33 663.

🕒 EN 30 SECONDES

- `bg_floor` = 4 524 : tout ce qui est en dessous est du fond.
- `sig_max` = 33 663 : tout ce qui est au-dessus devient blanc.
- La fenêtre utile est donc **[4 524 ; 33 663]**, pas [0 ; 65 535].

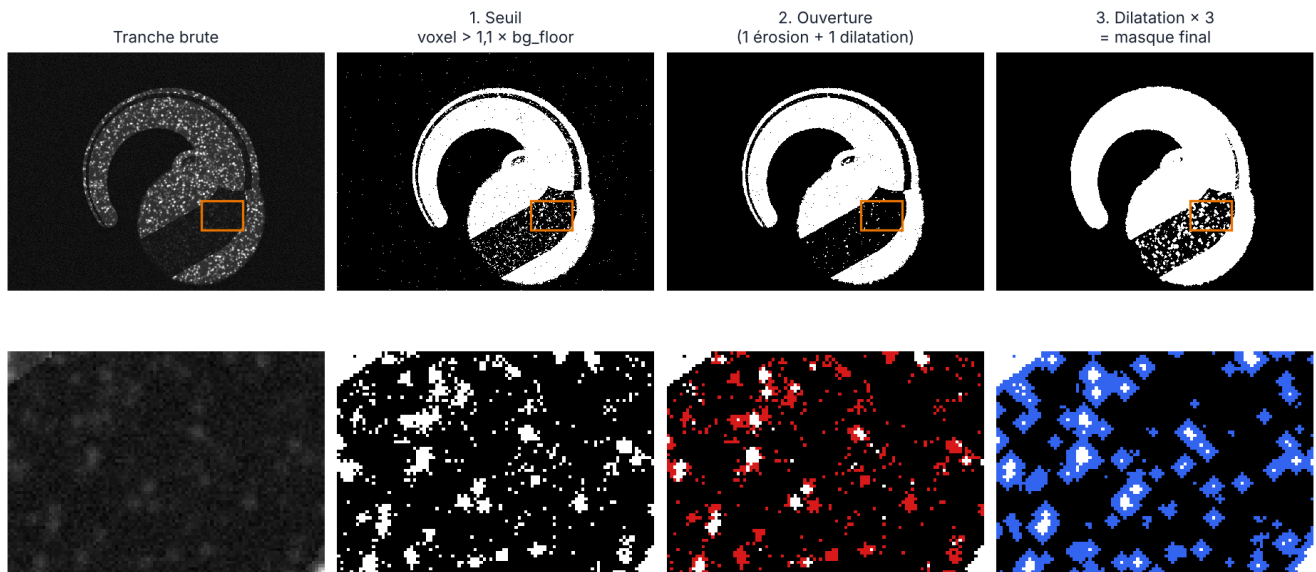
5.4 Le masque : protéger le vrai signal

Avant de lisser le fond, il faut **repérer le signal** pour ne pas le lisser. Trois opérations, appliquées en 3D sur tous les voxels.

1. Seuil. Un voxel est « signal » s'il est **plus grand que $1,1 \times \text{bg_floor}$** (10 % au-dessus de la valeur du plancher). Ici : $1,1 \times 4\,523,85 = 4\,976$.

2. Ouverture (1 itération) : une érosion puis une dilatation. Elle supprime les petits objets isolés (pixels chauds, grains de bruit).

3. Dilatation $\times 3$: le masque est élargi de 3 voxels pour garder la décroissance naturelle de la fluorescence autour des cellules.



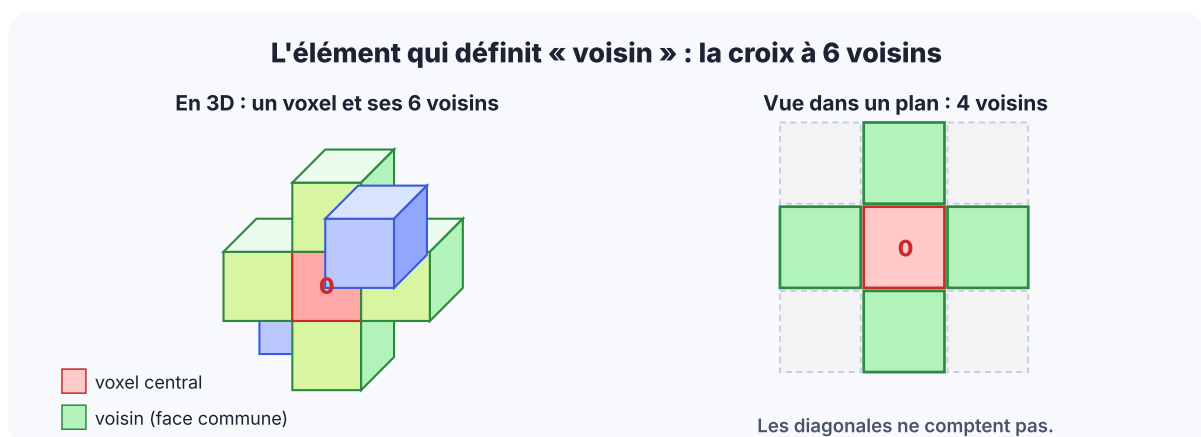
Canal DAPI, coupe $z = 56$ (jeu de démonstration). Ligne du bas : zoom sur la zone encadrée. rouge : voxels retirés - bleu : voxels ajoutés

▲ Les trois étapes du masque sur la coupe $z = 56$. Ligne du bas : zoom. En rouge, les voxels retirés par l'ouverture (4 748 dans cette coupe) ; en bleu, ceux ajoutés par la dilatation.

11,77 % voxels > seuil **11,32 %** après ouverture **13,92 %** masque final

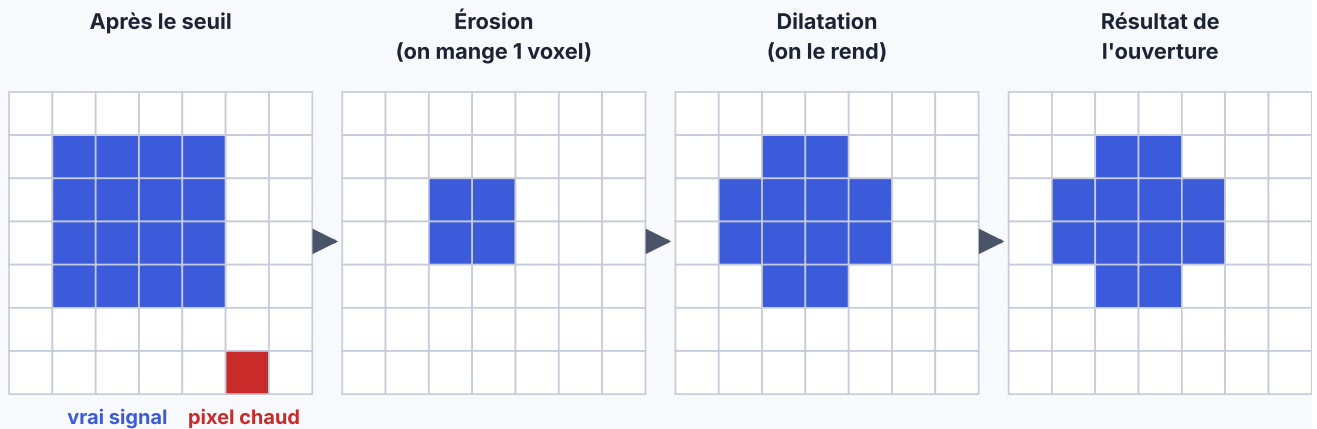
Qu'est-ce qu'un « voisin » ?

L'érosion et la dilatation regardent les voisins d'un voxel. Le pipeline utilise la **croix à 6 voisins** : les deux voisins en X, en Y et en Z, qui partagent une face. Les diagonales ne comptent pas.



▲ L'élément structurant : la croix 3D à 6 voisins (réglage par défaut de scipy, non modifié par le pipeline).

Ouverture sur une grille 7 × 7 : le pixel isolé disparaît, le vrai signal reste



2D, croix à 4 voisins pour le dessin ; le pipeline fait la même chose en 3D avec 6 voisins.

▲ Mini-grille 7 × 7. Le bloc de 4 × 4 pixels survit (ses coins sont un peu arrondis), le pixel chaud isolé disparaît. Calculé avec scipy.

MICRO-EXEMPLE

Sur la grille : 17 pixels allumés au départ (16 de signal + 1 chaud) ; après l'ouverture il en reste **12** : le pixel chaud et les 4 coins du bloc sont partis. Le vrai signal est protégé par la dilatation qui suit (×3).

? POURQUOI CE CHOIX ?

L'ouverture est faite **avant** la dilatation. Dans l'ordre inverse, un pixel chaud serait « protégé » et même **agrandi** par la dilatation. Ici, il reste hors du masque, donc il sera écrasé par la médiane.

5.5 La médiane, hors du masque seulement

Pour chaque voxel, le pipeline calcule la **médiane du cube 3 × 3 × 3** autour de lui (27 valeurs). Puis :

- **dans le masque** → on garde la valeur **originale**, intacte ;
- **hors du masque** → on la remplace par la **médiane**.

Résultat : le signal reste net, le fond est lissé.

Médiane ou moyenne ? Un pixel chaud au milieu de 8 voisins

Les 9 valeurs

2 810	3 120	2 950
3 040	52 000	2 880
2 760	3 090	2 990

Rangées dans l'ordre :

2,8k	2,8k	2,9k	3,0k	3,0k	3,0k	3,1k	3,1k	52k
------	------	------	------	------	------	------	------	-----

médiane = 5e valeur = 2 990

le pixel chaud est écrasé

moyenne = 8404

le pixel chaud contamine tout

Exemple 2D à 9 valeurs ; le pipeline prend 27 voxels ($3 \times 3 \times 3$).

▲ Médiane contre moyenne sur 9 valeurs (exemple 2D) : un pixel chaud à 52 000 est écrasé par la médiane (2 990), mais fait grimper la moyenne à 8 404.

UNE IMAGE POUR COMPRENDRE

La **médiane**, c'est le salaire « du milieu » d'une rue : un milliardaire qui emménage ne la change pas. La **moyenne**, elle, explose. C'est pourquoi un filtre médian écrase un pixel chaud sans brouiller les contours.

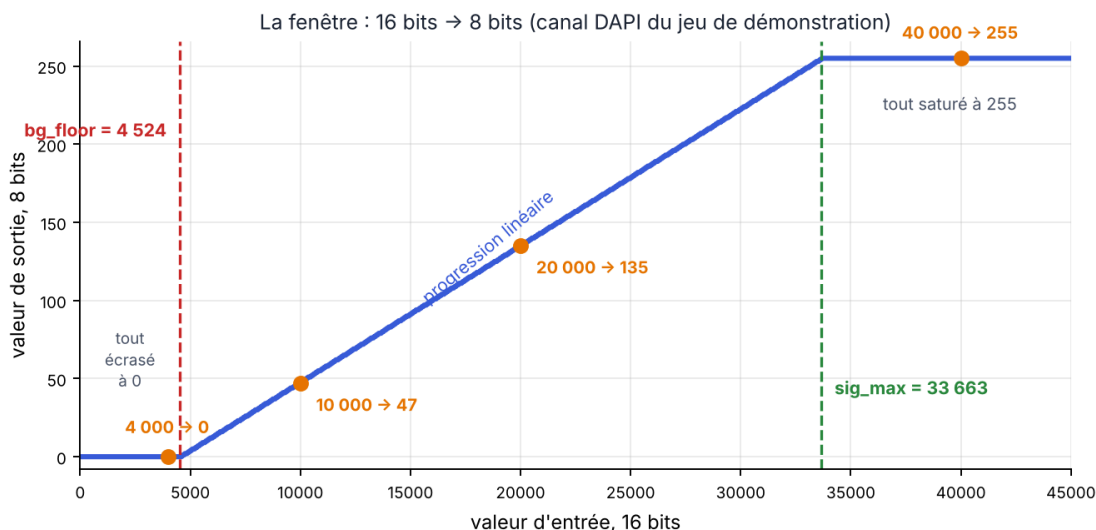
5.6 La fenêtre : de 16 à 8 bits

Dernière opération, voxel par voxel. La formule exacte du code :

MICRO-EXEMPLE

$u8 = \text{tronqué}(255 \times (\text{clip}(v, bg, sig) - bg) \div (sig - bg))$

v est la valeur du voxel (originale dans le masque, médiane en dehors) ; $bg = bg_floor$; $sig = sig_max$; $clip$ ramène v entre bg et sig . « Tronqué » veut dire que l'on jette les décimales (pas d'arrondi).



▲ La fenêtre : plat à 0 sous `bg_floor`, plat à 255 au-dessus de `sig_max`, linéaire entre les deux. Les points orange sont les micro-exemples ci-dessous.

1 2 3 4 MICRO-EXEMPLE

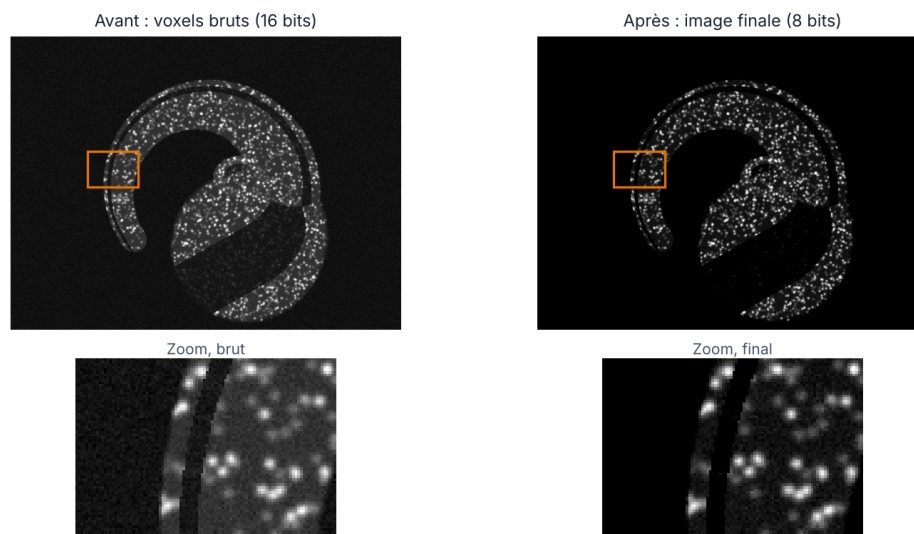
Avec `bg` = 4 523,85 et `sig` = 33 662,57 (donc `sig` – `bg` = 29 138,72) :

- $v = 4\,000 \rightarrow$ sous le plancher $\rightarrow$ **0** ;
- $v = 10\,000 \rightarrow 255 \times 5\,476,15 \div 29\,138,72 = 47,9 \rightarrow$ **47** ;
- $v = 20\,000 \rightarrow 255 \times 15\,476,15 \div 29\,138,72 = 135,4 \rightarrow$ **135** ;
- $v = 40\,000 \rightarrow$ au-dessus du blanc $\rightarrow$ **255**.

i À SAVOIR

La conversion est **linéaire**, sans courbe gamma. Tout voxel $\leq$ `bg_floor` devient **exactement 0** : le fond est un noir parfait, ce qui permet de ne stocker aucune brique vide (chapitre 6). Dans le jeu de démonstration, **88,2** % des voxels du canal DAPI valent 0 après nettoyage.

5.7 Avant et après



Même coupe z = 56, canal DAPI (jeu de démonstration). Le brut est affiché de 0 à sig_max pour une comparaison loyale.

▲ Même coupe avant (voxels bruts affichés de 0 à `sig_max`) et après nettoyage. En bas : zoom sur le bord de l'embryon. Le grain gris du fond a disparu ; les noyaux sont intacts (jeu de démonstration).

Regardez le zoom : à gauche du bord, le fond gris granuleux est devenu noir pur ; à droite, les noyaux brillants ont la même forme et le même contraste.

5.8 Ce qui a été essayé, puis abandonné

Le pipeline a connu des méthodes plus sophistiquées, retirées à la version 0.12.0 :

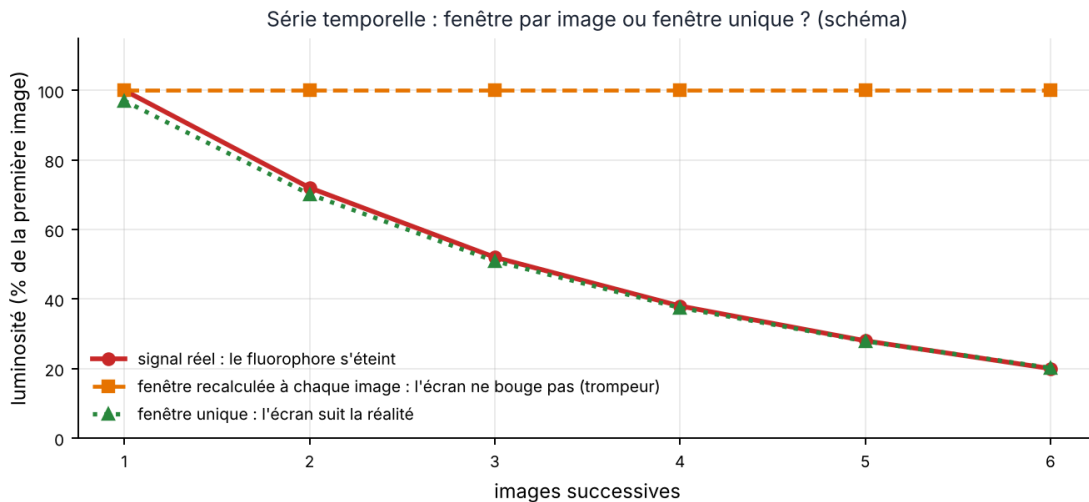
MÉTHODE	POURQUOI ABANDONNÉE
Seuil d'Otsu	Soustraction du masque : créait de gros blobs colorés artificiels
Noise2Void (réseau de neurones)	Même résultat artefactuel ; l'embryon perdait sa texture naturelle

? POURQUOI CE CHOIX ?

Choix actuel : le pipeline **ne retire que le fond global de la caméra** et garde les intensités natives du microscope. Pas de réseau de neurones, pas de correction de champ plat. Un test sur un jeu de référence a retrouvé les 97 empreintes SHA-256 des paquets de production, octet pour octet.

5.9 Séries temporelles : une seule fenêtre

Pour un film (plusieurs images), faut-il une fenêtre par image ? **Non : une seule pour toute la série**, pour chaque canal. Sinon l'image « clignote ».



▲ Schéma (chiffres illustratifs). Un fluorophore s'éteint (photoblanchiment). Si chaque image est étirée sur sa propre fenêtre, l'écran reste constant alors que le signal s'effondre : c'est faux. Avec une fenêtre unique, l'écran baisse comme le signal réel.

Comment la fenêtre unique est-elle mesurée ?

- jusqu'à **8 images** échantillonnées (régulièrement espacées, toujours la première et la dernière) ;
- `bg_floor` = 99e percentile des coins de toutes ces images réunies ;
- `sig_max` = 99,9e percentile **des seuls voxels au-dessus de `bg_floor`**, s'il y en a au moins **1 000**.

? POURQUOI CE CHOIX ?

Dans une série éparse, le signal ne couvre que ~0,4 % des voxels : le 99,9e percentile du volume entier tomberait **dans le fond** et ferait saturer 15 % du vrai signal. En ne classant que les voxels au-dessus du fond, la part saturée tombe à environ 0,06 % (mesures du journal de version 0.15.0).

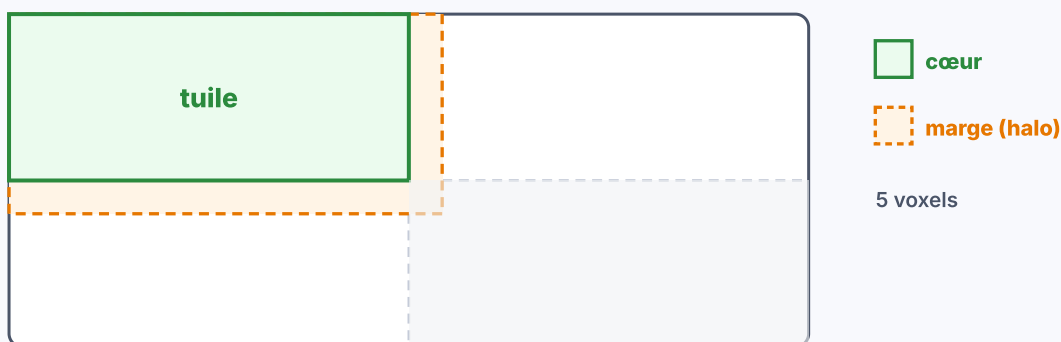
Le photoblanchiment est **mesuré, jamais effacé** : le pipeline enregistre un niveau de signal par image et par canal (99,9e percentile du brut) dans `metadata.json`. Le viewer peut proposer une compensation facultative et réversible.

5.10 Gros volumes : par tuiles, sans changer le résultat

Un canal de $3789 \times 3789 \times 178$ voxels représente 2,56 milliards de voxels, soit 10,2 Go en flottants 32 bits : il ne tient pas en mémoire. Le pipeline le lit donc **par tuiles** d'au plus **24 millions de voxels** (marge comprise).

Gros volume : découpé en tuiles avec une marge de 5 voxels

le canal (jamais chargé en entier)



cœur de la tuile : ce qui est écrit

marge (halo) de 5 voxels : lue pour calculer, jetée ensuite

1 (ouverture : érosion) + 1 (dilatation) + 3 (dilatations) = 5 voxels d'influence

≤ 24 millions de voxels par tuile, marge comprise

Résultat identique octet pour octet, quel que soit le découpage

▲ Découpage en tuiles. La marge de 5 voxels fournit aux calculs de bord (ouverture, dilatations) les voisins qui leur manquent ; seul le cœur est écrit.

Pourquoi 5 voxels ? L'érosion (1) + la dilatation de l'ouverture (1) + les 3 dilatations (3) propagent l'influence d'un voxel sur 5 voxels de distance. La médiane en demande 1 (couvert par la même marge).

MICRO-EXEMPLE

Vérifié sur le jeu de démonstration (3 canaux) : découpé en 4 tuiles (réglage par défaut), puis en **256 tuiles** (0,5 M de voxels), le volume 8 bits est **identique octet pour octet** au calcul d'un seul bloc, et à celui du dataset publié par le pipeline (coupes $z = 0, 30, 56$ et 100).

5.11 Ce que cela change pour vous

ATTENTION

Les valeurs 8 bits sont **relatives**, pas absolues. Chaque canal a sa propre fenêtre [`bg_floor` ; `sig_max`] ; 128 sur DAPI et 128 sur Pecam1 ne représentent pas la même quantité de lumière. Ne comparez pas d'intensités entre canaux ni entre datasets.

- Dans une **série temporelle**, les valeurs sont comparables entre images **d'un même canal** (fenêtre unique).
- Le fichier **.ims original** n'est jamais modifié ; c'est la source pour toute mesure d'intensité. Il peut être téléchargé depuis le dossier `download/` si le dataset a été publié avec ses fichiers d'origine.
- Un signal **très faible** (sous $1,1 \times \text{bg_floor}$) ou réduit à un voxel isolé peut être effacé : c'est le prix d'un fond parfaitement noir. Dans le masque, en revanche, les valeurs d'origine sont gardées.

À RETENIR

1. `bg_floor` = 99e percentile de 8 cubes de coin ; `sig_max` = 99,9e percentile d'un voxel sur 4.
2. Masque = seuil $1,1 \times \text{bg_floor}$ → ouverture → dilatation $\times 3$.
3. Médiane $3 \times 3 \times 3$ **hors masque** ; le signal reste intact.
4. 8 bits = fenêtre linéaire $[\text{bg_floor} ; \text{sig_max}] \rightarrow [0 ; 255]$, tronquée ; fond = 0 exact.
5. Une seule fenêtre pour tout un film ; les tuiles ne changent pas le résultat.

6. Réduire la taille sans rien perdre d'important

- Un embryon de plusieurs gigaoctets ne passe pas sur Internet tel quel : le pipeline le **réduit** en quatre mouvements — 16 → 8 bits, pyramide de résolutions, suppression des briques vides, compression **sans perte**.
- Aucun de ces mouvements ne modifie une intensité du niveau le plus fin : ce que vous voyez à pleine résolution est **exactement** le résultat du nettoyage du chapitre 5.
- Sur le jeu de démonstration : **297 Mo de voxels bruts** → **9,3 Mo de briques**, soit environ 32 fois moins.

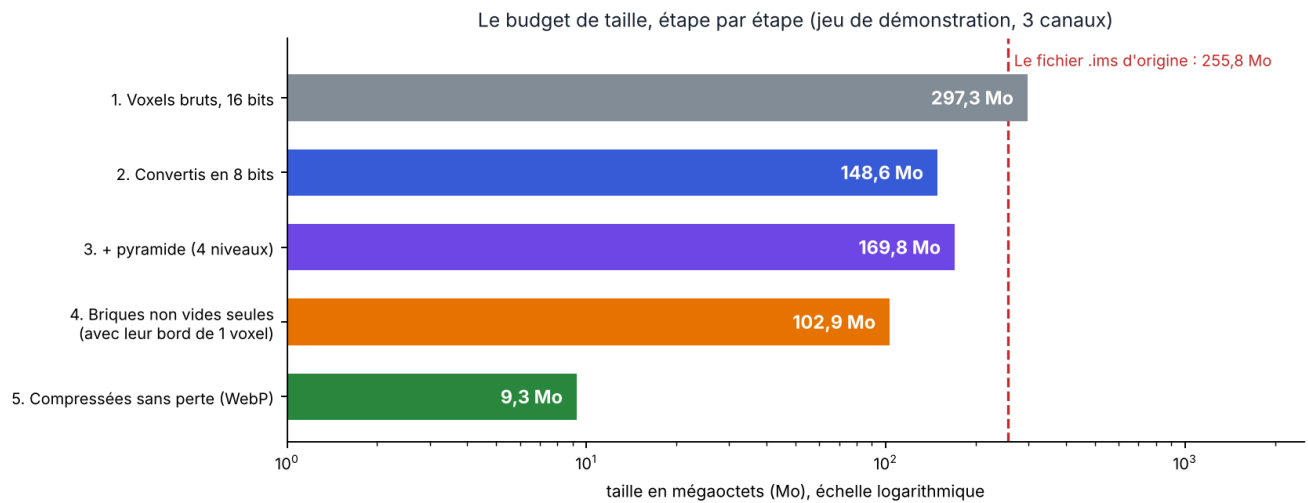
Tous les chiffres « démo » de ce chapitre sont **mesurés** sur l'embryon synthétique [Embryo-E95-Em2-Pecam1-Sox2](#) (768 × 576 × 112 voxels, 3 canaux), publié par le vrai pipeline (format 4).

6.1 Le budget de taille en chiffres

Prenons d'abord un **gros** embryon du laboratoire : 3789 × 3789 × 178 voxels, 4 canaux. Cela fait 10,2 milliards de voxels.

ÉTAPE	CALCUL	TAILLE
Voxels bruts, 16 bits	10,22 G voxels × 2 octets	20,4 Go
Convertis en 8 bits	10,22 G voxels × 1 octet	10,2 Go
+ pyramide de résolutions	× 1,286 pour ce découpage	13,1 Go
Fichier .ims réel de ce type	compressé par Imaris	14,4 Go

Les deux dernières étapes (briques vides retirées, compression) dépendent du contenu de l'image. Mesurons-les sur le jeu de démonstration, étape par étape :



▲ Budget de taille du jeu de démonstration. L'axe est logarithmique : chaque graduation vaut dix fois la précédente.

297 Mo 16 bits bruts

149 Mo en 8 bits

103 Mo briques non
vides

9,3 Mo sur le disque

i À SAVOIR

La pyramide **ajoute** des octets (+14 % ici) : elle est le prix de l'affichage rapide. Le gain vient des deux étapes suivantes : ne pas stocker le vide, et compresser.

6.2 De 16 bits à 8 bits : diviser par deux

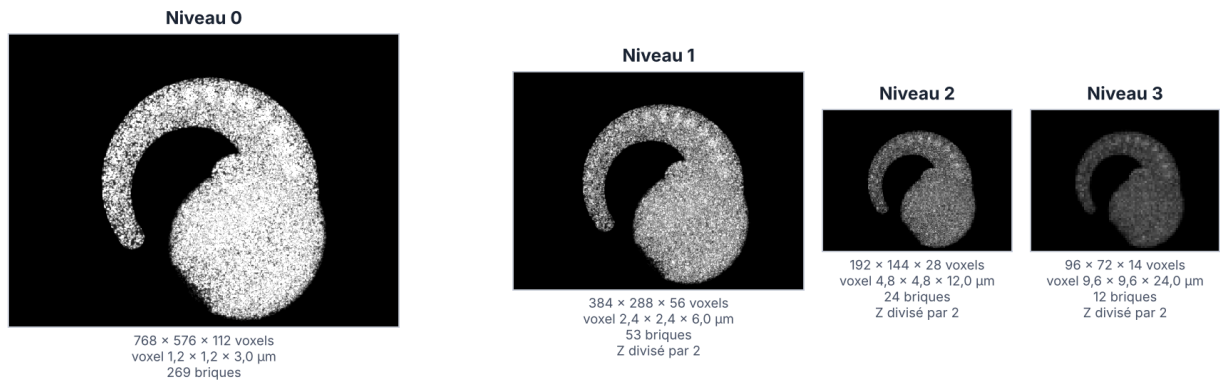
C'est la fenêtre du chapitre 5 : un voxel de 2 octets devient un voxel d'1 octet. On perd la finesse des 65 536 niveaux, mais le navigateur n'en affiche que 256 : l'écran ne sait pas montrer plus.

⚠ ATTENTION

C'est **la seule étape qui perd de l'information** (les nuances au-dessus de 256 niveaux), et elle a lieu au chapitre 5, pas ici. Tout ce qui suit dans ce chapitre est **sans perte**.

6.3 La pyramide de résolutions

Quand vous regardez l'embryon entier, vous n'avez pas besoin de tous les voxels : l'écran ne compte que quelques millions de pixels. Le pipeline prépare donc **plusieurs versions** de l'image, de plus en plus petites : les **niveaux**.



▲ Les quatre niveaux du jeu de démonstration, dessinés à l'échelle (projection de l'intensité maximale du canal DAPI). Même forme, de moins en moins de voxels.

💡 UNE IMAGE POUR COMPRENDRE

C'est une **carte routière** : la carte du pays pour voyager, la carte de la ville pour se repérer, le plan de la rue pour trouver la porte. On ne charge que la carte utile.

La règle du format 4

- **Niveau 0** : l'image nettoyée, **telle quelle** (jamais rééchantillonnée).
- **Niveau suivant** : X et Y sont **divisés par 2** (arrondi vers le haut) et leur voxel double.
- **Z est divisé par 2 seulement si** la profondeur du voxel reste inférieure ou égale à **1,5 fois** le nouveau voxel en XY. Sinon Z est gardé.
- On s'arrête quand le plus grand côté en XY est **inférieur ou égal à 128** voxels.

? POURQUOI CE CHOIX ?

Un voxel trop « plat » ou trop « allongé » déforme l'image. Diviser Z seulement quand il est encore comparable à XY ramène peu à peu le voxel vers un **cube** (c'est la convention des formats d'images biomédicales modernes, comme OME-Zarr).

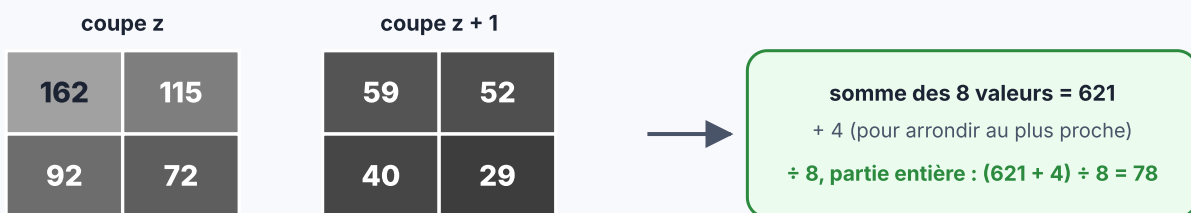
Le calcul d'un voxel du niveau suivant

Chaque voxel du niveau suivant est la **moyenne** d'un bloc de $2 \times 2 \times 2$ voxels du niveau précédent. Le calcul se fait en **nombre entiers, arrondi à la valeur la plus proche, la moitié vers le haut** : $(\text{somme} + n \div 2) \div n$, partie entière, n étant le nombre de voxels du bloc.

1 2 3 4 MICRO-EXEMPLE

Bloc réel du jeu de démonstration : 162, 115, 92, 72, 59, 52, 40, 29. Somme = **621**. On ajoute 4 (la moitié de 8), puis on divise par 8 : $(621 + 4) \div 8 = 78,1 \rightarrow$ **78**. Sans l'ajout de 4, on aurait obtenu 77 : le biais aurait assombri chaque niveau.

Réduire : la moyenne d'un bloc $2 \times 2 \times 2$ (valeurs réelles du jeu de démonstration)



→ 1 voxel du niveau suivant : 78

▲ Le calcul d'un voxel du niveau suivant, sur un bloc réel du jeu de démonstration.

Au bord du volume, un bloc peut n'avoir que 1 voxel de large : on ne moyenne alors **que les voxels qui existent**.

L'échelle du jeu de démonstration

Voxel de départ : $1,2 \times 1,2 \times 3,0 \mu\text{m}$. Pour chaque niveau, la règle de Z est testée avec le **nouveau** voxel XY.

NIVEAU	VOXELS ($X \times Y \times Z$)	VOXEL (MM)	Z DIVISÉ ?
0	$768 \times 576 \times 112$	$1,2 \times 1,2 \times 3,0$	(base)
1	$384 \times 288 \times 56$	$2,4 \times 2,4 \times 6,0$	oui : $3,0 \leq 1,5 \times 2,4 = 3,6$
2	$192 \times 144 \times 28$	$4,8 \times 4,8 \times 12,0$	oui : $6,0 \leq 1,5 \times 4,8 = 7,2$
3	$96 \times 72 \times 14$	$9,6 \times 9,6 \times 24,0$	oui : $12,0 \leq 1,5 \times 9,6 = 14,4$

On s'arrête au niveau 3 : $96 \leq 128$.

POUR LES CURIEUX

Quand Z n'est pas divisé. Pour une pile $3789 \times 3789 \times 178$ avec un voxel $(0,33 ; 0,33 ; 1,876 \mu\text{m})$ (tailles de voxel illustratives), le niveau 1 garde Z : $1,876 > 1,5 \times 0,66 = 0,99$. Le niveau 2 le divise : $1,876 \leq 1,5 \times 1,32 = 1,98$. Le résultat est $3789 \times 3789 \times 178 \rightarrow 1895 \times 1895 \times 178 \rightarrow 948 \times 948 \times 89 \rightarrow 474 \times 474 \times 45 \rightarrow 237 \times 237 \times 23 \rightarrow 119 \times 119 \times 12$ (6 niveaux).

À SAVOIR

J'ai reconstruit cette pyramide avec les fonctions du pipeline à partir du résultat du chapitre 5, puis comparé aux briques publiées : **358 briques sur 358 identiques**, voxel pour voxel.

6.4 Compresser sans rien perdre

Pourquoi « sans perte » ?

Une compression **avec perte** (JPEG) change légèrement les valeurs pour gagner de la place : acceptable pour une photo de vacances, pas pour une **mesure**. Une compression **sans perte** permet de retrouver chaque voxel à l'identique.

Comment ça marche, en clair

L'idée : un voxel ressemble à son voisin. Plutôt que de ranger chaque valeur, on range **la différence avec le voisin**, qui est petite et se compresse bien.

Compression sans perte : on range la différence avec le voisin

Valeurs d'une ligne de pixels

74	73	69	72	66	66	62	62	61	67	64	67	68	72
----	----	----	----	----	----	----	----	----	----	----	----	----	----

Différence avec le voisin de gauche

74	-1	-4	+3	-6	+0	-4	+0	-1	+6	-3	+3	+1	+4
----	----	----	----	----	----	----	----	----	----	----	----	----	----

Les valeurs vont jusqu'à 74 : il faut 8 bits chacune.

Les différences restent entre -6 et +6 : bien moins de bits.

On retrouve chaque valeur en additionnant : rien n'est perdu.

▲ Une vraie ligne de 14 voxels du canal Sox2 (jeu de démonstration). Les différences sont petites ; en les additionnant dans l'ordre, on retrouve les valeurs d'origine.

POUR LES CURIEUX

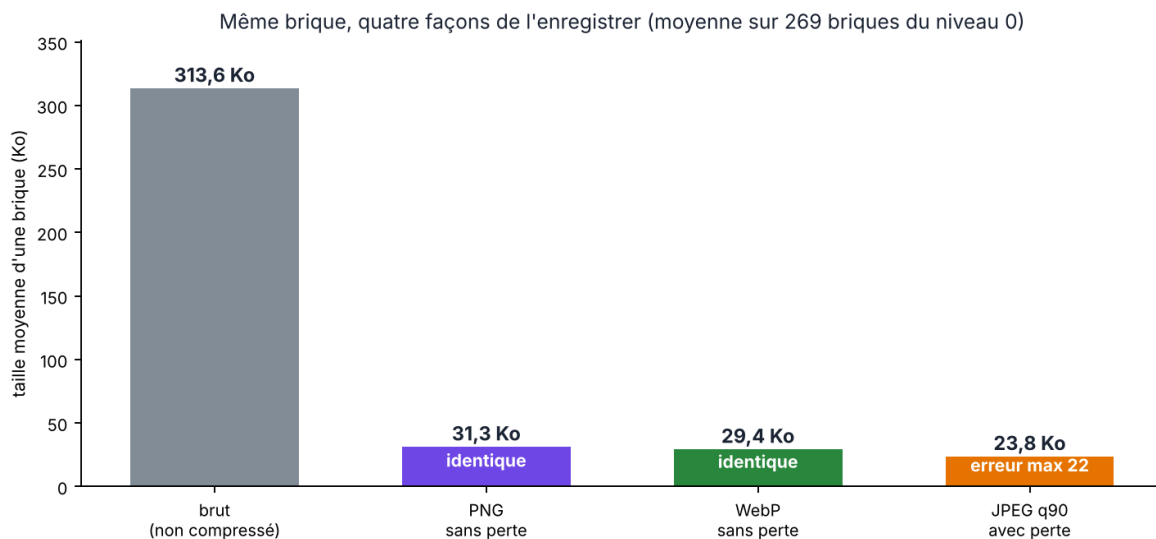
Les compresseurs réels sont plus fins : ils prédisent un voxel à partir de plusieurs voisins (gauche, haut...), puis codent économiquement les écarts, les plus fréquents en premier. Le principe reste celui-ci, et il est exactement réversible.

Les briques sont des images WebP sans perte

Une brique 3D n'est pas une image 2D. Le pipeline range donc ses 66 coupes côte à côte dans une **mosaïque 594 × 528 pixels** (9 colonnes × 8 lignes ; chapitre 7), et enregistre la mosaïque en **WebP sans perte**.

La mesure : quatre façons d'enregistrer la même brique

J'ai pris les **269 briques non vides du niveau 0** (3 canaux) du jeu de démonstration et enregistré chaque mosaïque de 594 × 528 pixels de quatre façons.

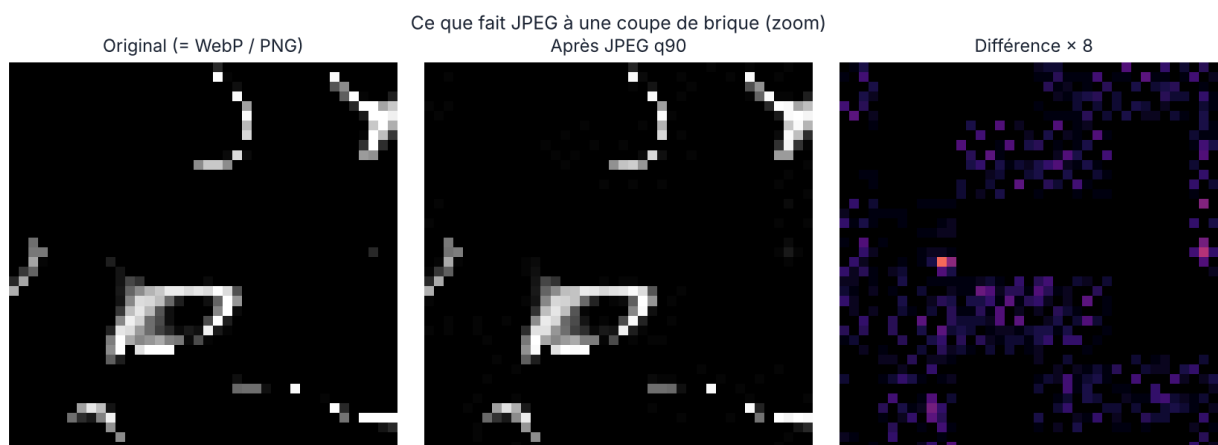


▲ Taille moyenne d'une brique. Brut : 313,6 Ko. PNG (meilleure compression) : 31,3 Ko. WebP sans perte (réglage du pipeline) : 29,4 Ko. JPEG qualité 90 : 23,8 Ko, mais avec une erreur allant jusqu'à 22 niveaux de gris.

FORMAT	TAILLE MOYENNE	EXACT ?
Brut (non compressé)	313,6 Ko	oui
PNG, niveau 9 optimisé	31,3 Ko	oui
WebP sans perte (choix du pipeline)	29,4 Ko	oui
JPEG, qualité 90	23,8 Ko	non

⚠ ATTENTION

JPEG est le plus petit, mais il est faux. Sur ces briques : erreur maximale de 22 niveaux (sur 255), **11,7 %** des voxels modifiés, et **1 644 202 voxels qui valaient exactement 0 sont devenus non nuls** (2,2 % du fond). Or le « 0 exact » est ce qui permet de reconnaître une brique vide (6.5) : JPEG le détruirait.



▲ Zoom sur une coupe de brique (canal Pecam1) : après JPEG, de faibles halos gris apparaissent autour des structures fines, là où l'original est parfaitement noir. La différence est amplifiée 8 fois à droite.

? POURQUOI CE CHOIX ?

Pourquoi WebP plutôt que PNG ? Sur ces briques, WebP sans perte est seulement **6 % plus petit** que le meilleur PNG (29,4 contre 31,3 Ko). Le code donne une autre raison principale : le navigateur **décode WebP nativement** et vite (`createImageBitmap`), alors qu'il n'existe aucun format d'image 3D. Les tranches 2D de `planes/`, elles, sont en PNG : le viewer les lit avec un petit décodeur PNG maison, jamais via le canevas du navigateur, dont la lecture n'est pas garantie exacte.

i À SAVOIR

Les octets du pipeline sont reproductibles : en recodant les mosaïques avec les mêmes réglages (Pillow, WebP sans perte, méthode 4), j'ai retrouvé **exactement les mêmes octets** que dans les fichiers publiés. Seule la version de la bibliothèque WebP compte : c'est pourquoi Pillow est épinglé.

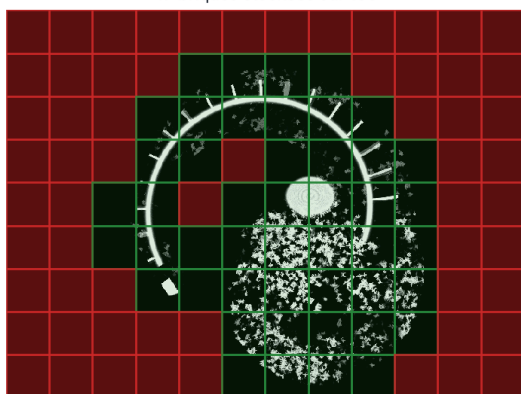
6.5 Ne pas stocker le vide

Un embryon de souris ne remplit pas son cadre. Beaucoup de briques ne contiennent que du noir : à quoi bon les enregistrer ?

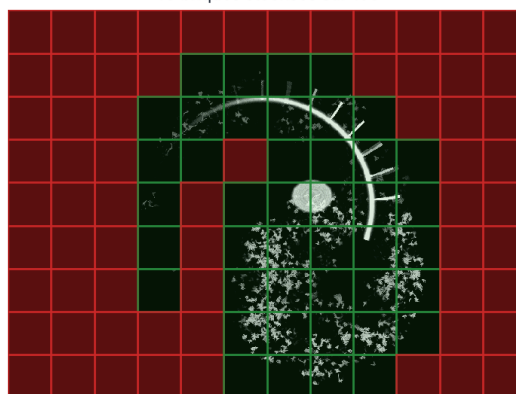
La règle (format 4) : une brique est stockée **si et seulement si** son intérieur de $64 \times 64 \times 64$ voxels contient **au moins un voxel ≥ 1** . Pas de tolérance : un seul voxel faible suffit à la garder.

Briques stockées (vert) et briques vides non stockées (rouge) : niveau 0, canal Pecam1

Couche de briques $z = 0$ (coupes 0–63)
47 briques stockées sur 108



Couche de briques $z = 1$ (coupes 64–111)
43 briques stockées sur 108



▲ Niveau 0, canal Pecam1, deux couches de briques. Vert : brique stockée. Rouge : brique vide, non stockée. Les briques sont tracées sur une projection de l'intensité maximale de la couche.

Le comptage exact, lu dans `index.bin` du jeu de démonstration (3 canaux) :

NIVEAU	BRIQUES POSSIBLES	BRIQUES STOCKÉES
0	648 (216 par canal)	269 (DAPI 100, Pecam1 90, Sox2 79)
1	90	53
2	27	24
3	12	12

NIVEAU	BRIQUES POSSIBLES	BRIQUES STOCKÉES
Total	777	358 (46 %)

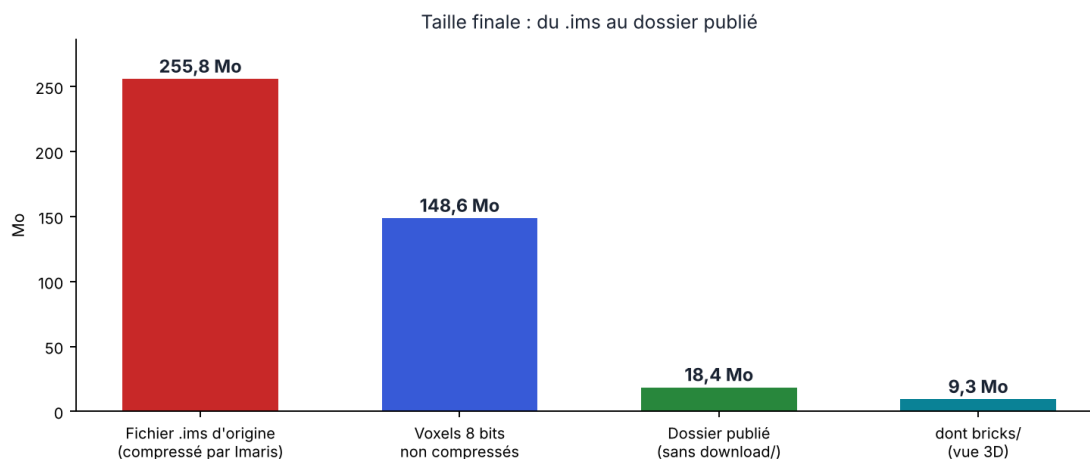
MICRO-EXEMPLE

Le canal Pecam1 n'a que **0,75 %** de voxels non nuls, mais **90 briques sur 216** du niveau 0 sont stockées : quelques vaisseaux fins traversent beaucoup de briques. Une brique stockée n'est pas pleine ; elle contient au moins un voxel.

À SAVOIR

Une brique **absente** vaut zéro partout, y compris sur son bord de 1 voxel. Le viewer ne la télécharge pas : l'espace vide ne coûte ni octets, ni temps de transfert (chapitre 10).

6.6 Taille finale : du .ims au dossier publié



▲ Du fichier d'origine au dossier publié, pour le jeu de démonstration. Mesures en octets sur le disque (somme des fichiers).

CONTENU	TAILLE
Fichier <code>.ims</code> d'origine (compressé par Imaris)	255,8 Mo
<code>bricks/</code> (affichage 3D)	9,28 Mo
<code>planes/</code> (coupes natives, PNG)	8,64 Mo
<code>mips/</code> (projections par couche)	0,47 Mo
<code>thumbnail.webp</code> + <code>metadata.json</code>	0,04 Mo
Dossier publié (hors <code>download/</code>)	18,4 Mo

14× plus petit que le
.ims

28× pour la seule vue
3D

46 % briques stockées

⚠ ATTENTION

Le dossier [download/](#) (fichier d'origine, TIFF, projections...) n'est pas compté : il ne sert pas à l'affichage. Il pèse 506 Mo ici, car il contient le .ims.

📌 À RETENIR

- 16 → 8 bits est la **seule** perte, décidée au chapitre 5 ; tout le reste est exact.
- La pyramide : $X, Y \div 2$ à chaque niveau, $Z \div 2$ seulement tant que son voxel $\leq 1,5 \times$ le voxel XY ; moyenne entière arrondie vers le haut ; arrêt à 128 voxels.
- Compression **WebP sans perte** : JPEG est plus petit mais abîme les valeurs et le « 0 exact ».
- Une brique est stockée seulement si son intérieur contient un voxel ≥ 1 : 358 briques stockées sur 777 possibles.

7. Le découpage en briques

- Un volume de plusieurs gigaoctets ne peut pas être chargé d'un bloc : on le découpe en **briques** (des petits cubes de $64 \times 64 \times 64$ voxels) que le navigateur va chercher une par une.
- Chaque brique devient **une seule image WebP** (sans perte) ; les briques sont rangées dans des **paquets**, retrouvées grâce à un **index** de quelques kilo-octets.
- Deux structures annexes, [planes/](#) et [mips/](#), rendent les coupes et les projections rapides ; le tout forme le **format de données 4**.

Dans ce chapitre, les images de données (mosaïque, couture, paquets...) viennent toutes du **jeu de démonstration** : l'embryon synthétique [Embryo-E95-Em2-Pecam1-Sox2](#) ($768 \times 576 \times 112$ voxels, 3 canaux), publié par le vrai pipeline. Ce n'est pas une donnée du laboratoire.

7.1 Pourquoi découper un volume ?

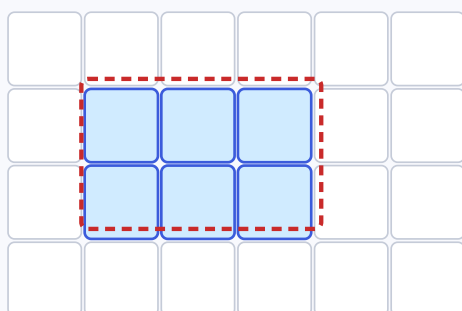
Un embryon typique du laboratoire fait $3789 \times 3789 \times 178$ voxels sur 4 canaux : environ **10,2 Go** une fois réduit à 8 bits. Aucun navigateur n'avale cela d'un bloc, et vous n'en regardez à chaque instant qu'une petite partie.

💡 UNE IMAGE POUR COMPRENDRE

Une carte en ligne. Quand vous zoomez sur une ville, le site ne télécharge pas la planète : il charge seulement les **tuiles** qui sont dans votre fenêtre, à la bonne finesse. Les briques sont les tuiles de votre embryon, avec une dimension de plus.

Ne charger que les morceaux qu'on regarde

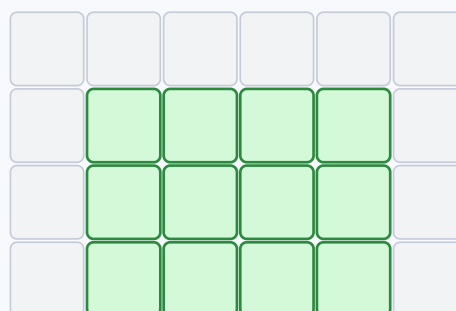
Une carte en ligne



6 × 4 tuiles, 6 chargées

votre écran = le cadre rouge

Un volume en briques



seules les briques visibles ET non vides

sont téléchargées (gris = vide, jamais stocké)

Embryon réel typique : $3789 \times 3789 \times 178$ voxels $\times$ 4 canaux $\approx$ 10,2 Go : trop gros pour un seul morceau.

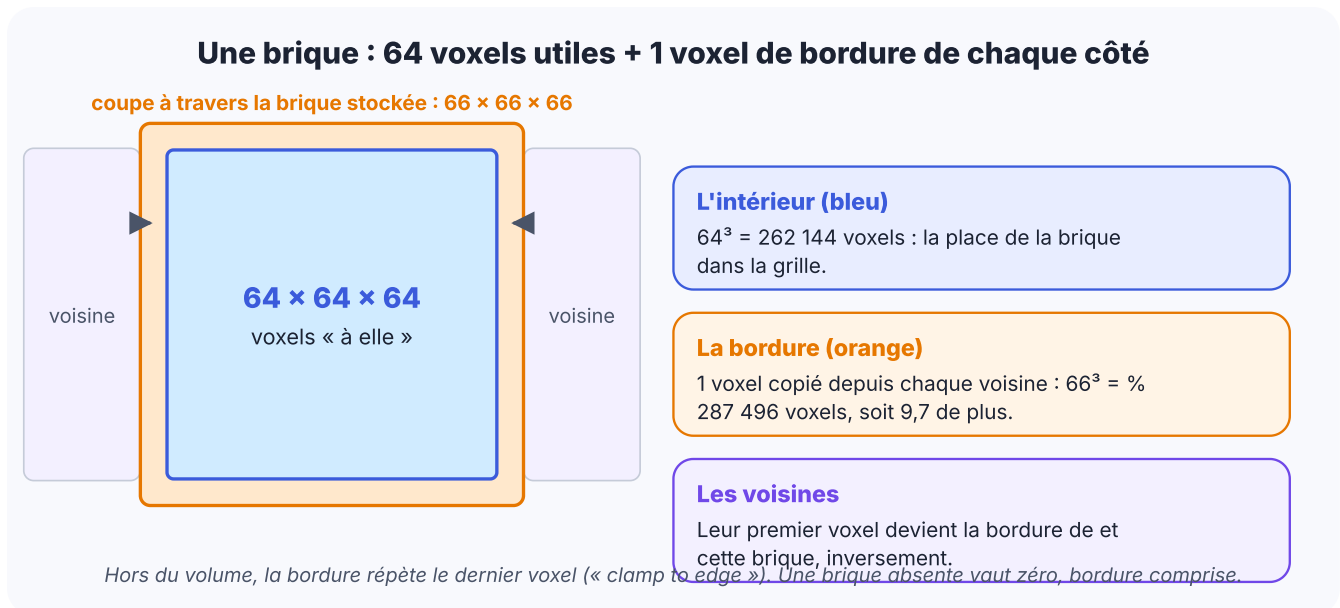
▲ Comme les tuiles d'une carte, seules les briques visibles et non vides sont téléchargées.

Trois avantages, tous visibles à l'écran :

- **le début est rapide** : les premières briques arrivent en une fraction de seconde ;
- **la mémoire est bornée** : la carte graphique ne reçoit que ce qu'elle peut loger (chapitre 10) ;
- **le vide ne coûte rien** : une brique entièrement noire n'est jamais écrite sur le disque.

7.2 Une brique : 64 voxels utiles et une bordure

Une brique couvre **64 voxels** dans chaque direction. Mais elle est stockée avec **un voxel de plus de chaque côté** : $64 + 1 + 1 = 66$. Ce voxel de bordure est une copie du voxel voisin, pris dans la brique d'à côté.

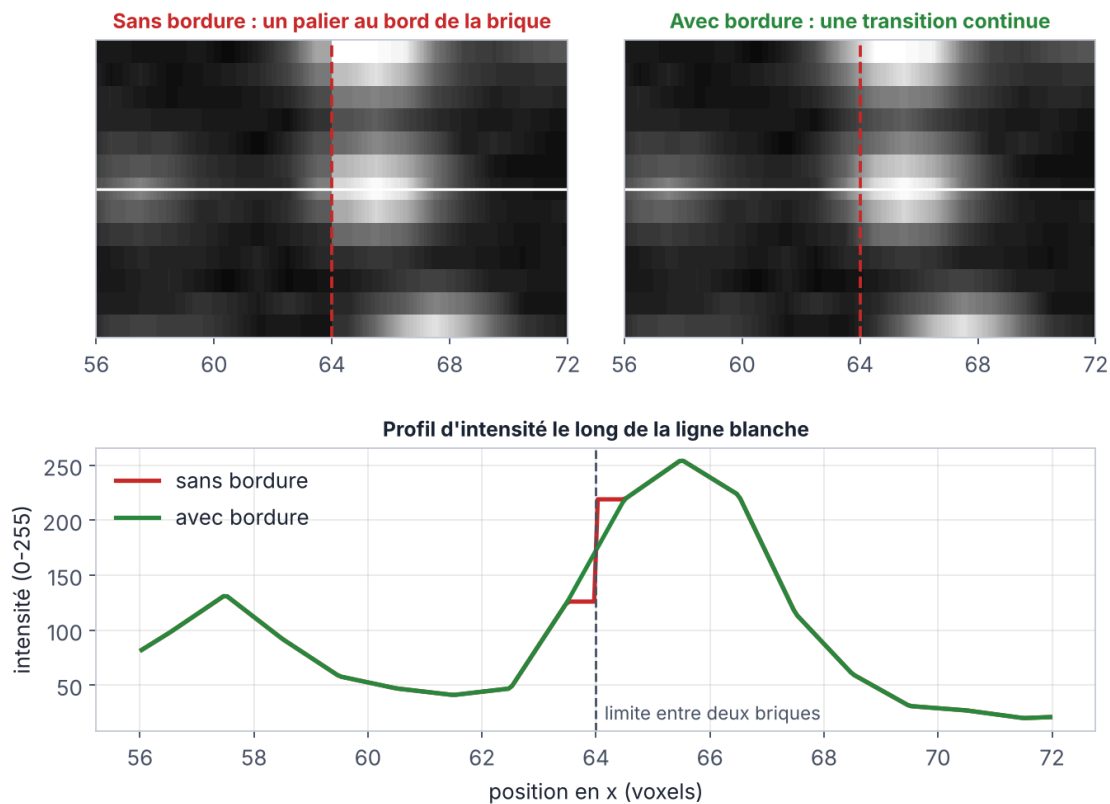


▲ Le cœur de la brique (bleu) est entouré d'une bordure d'un voxel (orange) copiée sur les briques voisines (violet).

? POURQUOI CE CHOIX ?

À quoi sert la bordure ? Pour dessiner une image lisse, la carte graphique *interpole* : la valeur à un point est un mélange pondéré des voxels qui l'entourent. Au bord d'une brique, il manque le voisin de l'autre côté. Sans bordure, la carte graphique devrait répéter le dernier voxel : on verrait un quadrillage régulier à la limite des briques. Avec la bordure, le voisin est là, et l'image est continue.

La figure suivante le montre sur de vraies données : le même profil d'intensité, à cheval sur la limite entre deux briques (en voxels 56 à 72).



▲ Sans bordure (rouge) la valeur fait un palier puis saute ; avec bordure (vert) elle varie sans coupure. Jeu de démonstration, canal DAPI.

1.2.4 MICRO-EXEMPLE

Le surcoût de la bordure. $66^3 = 287\,496$ voxels contre $64^3 = 262\,144$: **9,7 % de plus** stockés, en échange d'images sans couture à tous les niveaux de qualité.

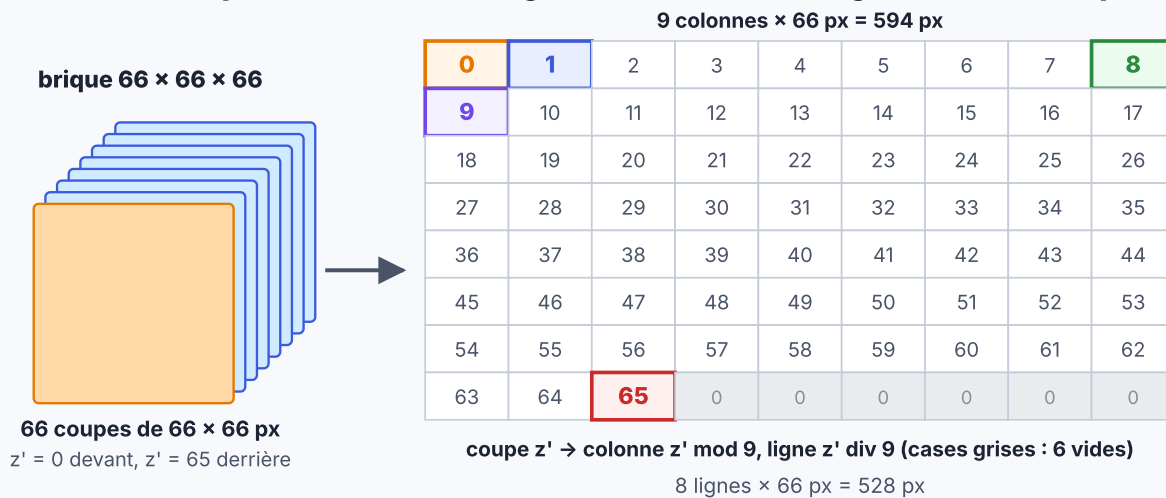
Deux règles pour les cas limites :

- **hors du volume**, la bordure répète le dernier voxel (« clamp to edge ») ;
- une **brique absente** (vide) vaut zéro partout, bordure comprise : on ne reconstruit jamais une bordure à partir des voisines.

7.3 D'un cube à une image : la mosaïque

Le navigateur sait décoder une image 2D très vite, mais n'a pas de format d'image 3D. L'astuce : la brique de 66 coupes est **posée à plat** en une seule image, comme un planche-contact de 66 photos.

66 coupes → une seule image de 9 colonnes × 8 lignes (594 × 528 px)



▲ Les 66 coupes d'une brique rangées en 9 colonnes × 8 lignes ; les 6 dernières cases restent vides.

Chaque coupe fait 66 × 66 pixels ; la grille fait donc $9 \times 66 = 594$ pixels de large et $8 \times 66 = 528$ de haut, en niveaux de gris 8 bits. La règle est simple :

1 2 3 4 MICRO-EXEMPLE

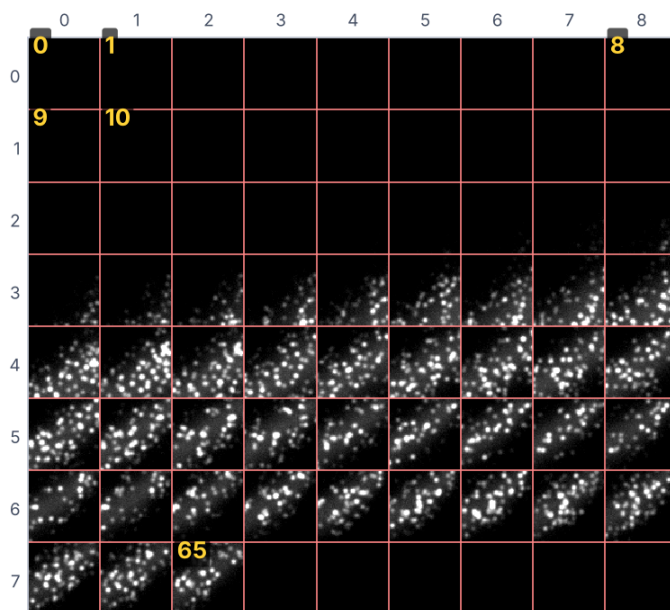
Où est la coupe z' ? Colonne = z' modulo 9, ligne = z' divisé par 9 (division entière).

COUPE Z'	COLONNE	LIGNE	COIN HAUT-GAUCHE (PX)
0	0	0	(0 ; 0)
1	1	0	(66 ; 0)
8	8	0	(528 ; 0)
9	0	1	(0 ; 66)
65	2	7	(132 ; 462)

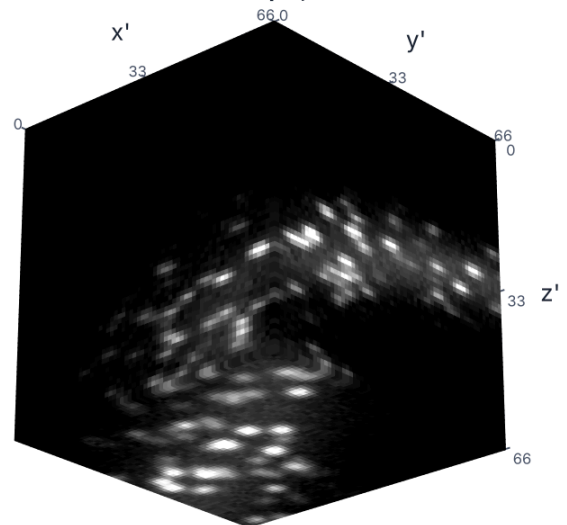
Pour un voxel (x', y', z') de la brique de 66³, son pixel dans l'image est : **colonne = (z' mod 9) × 66 + x'**, **ligne = (z' div 9) × 66 + y'**. Le voxel intérieur numéro i est à l'indice i + 1, à cause de la bordure.

Voici une vraie brique du jeu de démonstration, **telle qu'elle est stockée**, à côté de son cube en 3D.

La brique (7, 5, 0) du canal Sox2, niveau 0 : l'image WebP stockée



La même brique, en 3D



▲ La brique (7, 5, 0) du canal Sox2 : l'image WebP décodée, avec les numéros de quelques coupes, et le cube reconstitué.

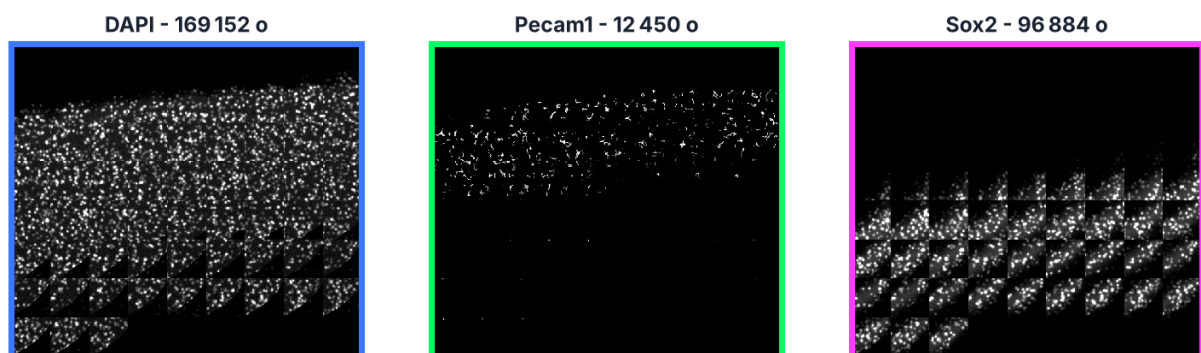
À SAVOIR

Les premières coupes sont noires parce que l'embryon de démonstration n'occupe que le bas de cette brique. Les cases 66 à 71 (les 6 dernières, en bas à droite) sont toujours vides : 72 places pour 66 coupes.

Une image par canal

Chaque canal a ses **propres** briques : la brique (7, 5, 0) existe en trois exemplaires indépendants (DAPI, Pecam1, Sox2). Le navigateur les assemble en couleur au dernier moment. Un canal peu dense, comme Pecam1 ici, donne une image beaucoup plus petite.

Une image par canal : la même brique (7, 5, 0)



▲ Les trois canaux de la même brique : même géométrie, contenus et poids différents.

POUR LES CURIEUX

Encodage. Chaque mosaïque est enregistrée en WebP *sans perte* (bibliothèque Pillow, `lossless=True`, qualité 75, méthode 4) : le voxel relu est identique au voxel écrit. Une brique n'est conservée que si son intérieur 64^3 contient **au moins un voxel ≥ 1** ; sinon elle est « absente » (longueur 0 dans l'index). Cette règle est calculée canal par canal.

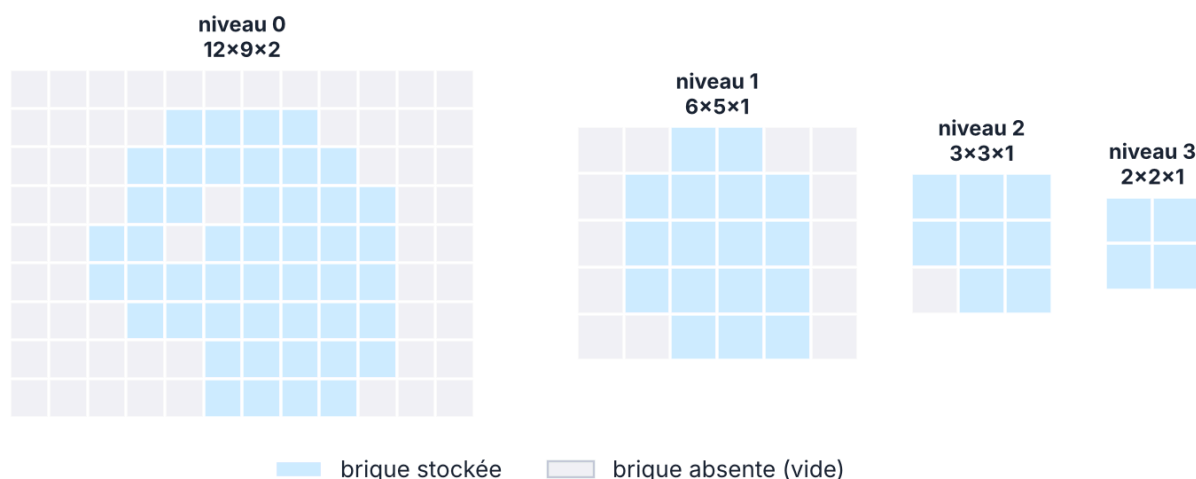
7.4 La grille de briques de chaque niveau

Le chapitre 6 a construit la pyramide de résolutions. À chaque niveau, le volume est découpé en une **grille** de briques : on arrondit au supérieur le nombre de voxels divisé par 64.

NIVEAU	VOXELS (X, Y, Z)	GRILLE DE BRIQUES	BRIQUES STOCKÉES (3 CANAUX)
0	$768 \times 576 \times 112$	$12 \times 9 \times 2$	269
1	$384 \times 288 \times 56$	$6 \times 5 \times 1$	53
2	$192 \times 144 \times 28$	$3 \times 3 \times 1$	24
3	$96 \times 72 \times 14$	$2 \times 2 \times 1$	12

La grille du niveau 0 compte $12 \times 9 \times 2 = 216$ cases par canal, mais seules 100 (DAPI), 90 (Pecam1) et 79 (Sox2) contiennent du signal : les autres coins sont vides.

La grille de briques de chaque niveau (couche $z = 0$, canal Pecam1)



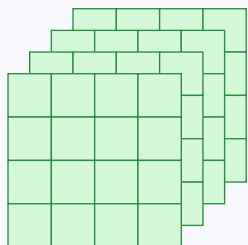
▲ La grille de briques de chaque niveau pour une couche et un canal : en gris, les briques jamais écrites.

7.5 Les paquets : des boîtes de 64 briques

Des milliers de petits fichiers seraient lents à servir. Les briques d'un même niveau et d'un même canal sont donc regroupées dans des **paquets** : `bricks/l0/c1/p000000.bin` est le premier paquet du niveau 0, canal 1.

Les paquets : des « boîtes de 64 briques » rangées par super-blocs $4 \times 4 \times 4$

un super-bloc
 $4 \times 4 \times 4 = 64$ briques voisines



Pourquoi des super-blocs ?

Une coupe en XZ ou YZ traverse une rangée de briques : avec des super-blocs, ces briques sont dans peu de paquets, donc peu de requêtes.

Mesuré (simulation, embryon $5735^2 \times 172$) : coupe YZ = 66 paquets au lieu de 668.

Un paquet contient toujours des super-blocs entiers ; un super-bloc trop gros est coupé dans l'ordre des briques.

▲ Un paquet contient des super-blocs entiers de $4 \times 4 \times 4$ briques : une coupe croise peu de paquets.

Les règles de remplissage :

- les briques sont classées par **super-blocs de $4 \times 4 \times 4$ briques voisines** ;
- un paquet contient des **super-blocs entiers**, au plus **64 briques** ou **16 Mio** ;
- si un seul super-bloc dépasse la limite, il est coupé dans l'ordre des briques.

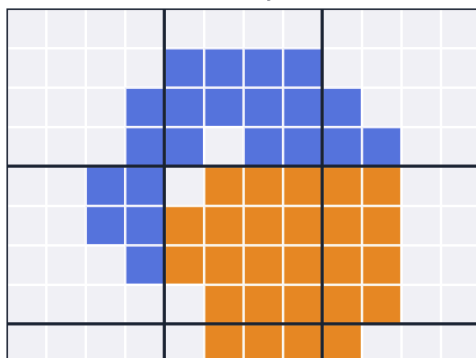
💡 UNE IMAGE POUR COMPRENDRE

Un déménagement. On range les cartons pièce par pièce (les super-blocs), pas au hasard dans les camions. Quand vous cherchez « tout ce qui est dans la cuisine », il suffit d'ouvrir un camion.

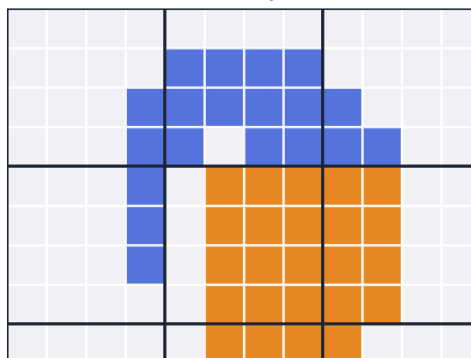
Dans le jeu de démonstration, le niveau 0 du canal Pecam1 tient dans **deux paquets** (en bleu et orange ci-dessous) : les traits noirs sont les limites des super-blocs.

Quelles briques dans quel paquet ? (niveau 0, canal Pecam1)

couche de briques $z = 0$



couche de briques $z = 1$



■ paquet p00000 ■ paquet p00001 ■ vide

▲ Quelles briques dans quel paquet, pour les deux couches du niveau 0 (canal Pecam1).

7.6 L'index et le manifeste

Pour récupérer une brique, le navigateur doit savoir **dans quel paquet** elle est, **où elle commence** et **combien elle pèse**. Ces trois nombres sont dans `bricks/index.bin`, un petit fichier binaire : **10 octets par case de brique**.

Anatomie de index.bin (le jeu de démonstration : 7 846 octets)



Toutes les valeurs sont en « petit-boutiste » (little-endian) : l'octet de poids faible vient en premier.

▲ Un fichier `index.bin` : un en-tête, une ligne par niveau, puis une entrée de 10 octets par case de brique.

Voici les octets réels du jeu de démonstration, décodés à la main : l'en-tête, puis la brique (7, 5, 0) de chaque canal.

CE QUE L'ON LIT	OCTETS (HEXADÉCIMAL)	VALEUR DÉCODÉE
début du fichier	4c 42 49 58	« LBIX » (la signature)
niveaux · canaux	04 00 · 03 00	4 niveaux · 3 canaux
niveau 0	0c 00 00 00 09 00 00 00 02 00 00 00 06 00 00 00	grille 12 × 9 × 2, 6 paquets
brique (7, 5, 0), DAPI	01 00 ce d3 07 00 c0 94 02 00	paquet 1, début 512 974, longueur 169 152
brique (7, 5, 0), Pecam1	01 00 88 1c 01 00 a2 30 00 00	paquet 1, début 72 840, longueur 12 450
brique (7, 5, 0), Sox2	00 00 a0 e2 03 00 74 7a 01 00	paquet 0, début 254 624, longueur 96 884

POUR LES CURIEUX

Lecture d'un nombre. Les octets sont en « petit-boutiste » : `c0 94 02 00` se lit à l'envers, `0x000294c0` = 169 152. Une longueur de **0** signifie « brique absente » (le paquet et le début valent alors aussi 0). Ce fichier fait $12 + 4 \times 16 + 7\,770 = 7\,846$ octets ; sur un grand embryon, l'ancien manifeste JSON pesait 7,6 Mo contre 233 ko pour l'index binaire.

À côté de l'index, `bricks/manifest.json` décrit le jeu de briques en clair. Extrait réel (abrégé) :

```
{
  "schema": "iribhm-bricks-v3",
  "formatVersion": 4,
  "channels": 3,
  "brickSize": 64,
  "apron": 1,
  "brickPacking": {
    "mode": "grid",
    "cols": 9,
    "rows": 8,
    "slice": 66
  },
  "encoding": "webp-lossless",
  "levels": [
    {
      "level": 0,
      "dimensions": {
        "x": 768,
        "y": 576,
        "z": 112
      },
      "voxelSize": {
        "x": 1.2,
        "y": 1.2,
        "z": 3.0
      },
      "gridSize": {
        "x": 12,
        "y": 9,
        "z": 2
      },
      "brickCount": 269,
      ...
    },
    ...
  ],
  "index": {
    "url": "index.bin",
    "bytes": 7846,
    "sha256": "aea09808...",
    ...
  },
  "histograms": [...]
}
```

La signature `sha256` de l'index est vérifiée avant usage : un index altéré est refusé. Les `histograms` servent aux curseurs de contraste (chapitres 8 et 11).

7.7 L'arbre d'un jeu de données publié

Voici le contenu réel du dossier du jeu de démonstration (abrégé ; les tailles sont celles du disque).

```
3d/Embryo-E95-Em2-Pecam1-Sox2/
├─ metadata.json          1,8 ko    les faits du jeu de données
├─ thumbnail.webp        33,6 ko    la vignette
├─ bricks/                9,3 Mo    les briques (format 4)
│   └─ manifest.json      3,7 ko
│   └─ index.bin          7,8 ko
│   └─ l0/ c0/ p00000.bin  1,4 Mo    niveau 0, canal 0
│       └─ p00001.bin      3,9 Mo
│       └─ c1/ ... c2/ ...
│       └─ l1/ l2/ l3/ ...    niveaux plus grossiers
├─ planes/                8,6 Mo    un fichier par plan Z
│   └─ manifest.json
│   └─ z00000.bin ... z00111.bin    (112 fichiers)
├─ mips/                  473 ko    un fichier par couche de 64 plans
│   └─ manifest.json
│   └─ l00000.bin l00001.bin
└─ download/              (voir chapitre 8)
```

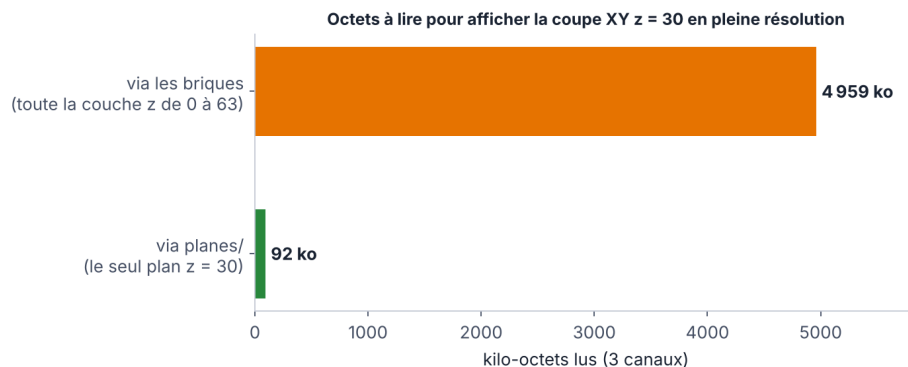
i À SAVOIR

Les niveaux se répartissent ainsi : niveau 0 = 7,9 Mo, niveau 1 = 1,1 Mo, niveau 2 = 0,18 Mo, niveau 3 = 0,03 Mo. Les niveaux grossiers sont minuscules : c'est pourquoi une première image apparaît presque instantanément.

7.8 `planes/` : une coupe XY sans lire 64 plans

Les briques sont des cubes. Pour afficher **une** coupe XY à pleine résolution (par exemple pour une figure du Studio), il faudrait lire toutes les briques de la couche, soit **64 plans de données** pour n'en utiliser qu'un.

Le dossier `planes/` contient donc le niveau 0 **redécoupé plan par plan** : un fichier `zNNNNN.bin` par plan Z (`z00000.bin` , `z00001.bin` ...). Chaque plan est coupé en tuiles de **512 × 512** pixels, enregistrées en PNG sans perte ; une tuile entièrement noire n'occupe aucun octet.



▲ Octets lus pour la coupe z = 30 des trois canaux : 5 078 ko par les briques contre 94 ko par planes/ (54 fois moins).

MICRO-EXEMPLE

Le chiffre du jeu de démonstration. La coupe XY z = 30 des trois canaux demande 5 078 326 octets via les briques de la couche 0, contre **94 053 octets** via `planes/z00030.bin`. Sur un grand embryon du laboratoire, une coupe de ce genre représentait environ 190 Mo, soit 40 secondes à 5 Mo/s.

Le résultat est **pixel pour pixel identique** à celui des briques, qui restent la solution de repli. `planes/` ne sert qu'aux coupes XY ; la vue 3D, les coupes XZ, YZ et obliques utilisent toujours les briques. Prix : environ 1,3 fois la taille du niveau natif sur le disque.

POUR LES CURIEUX

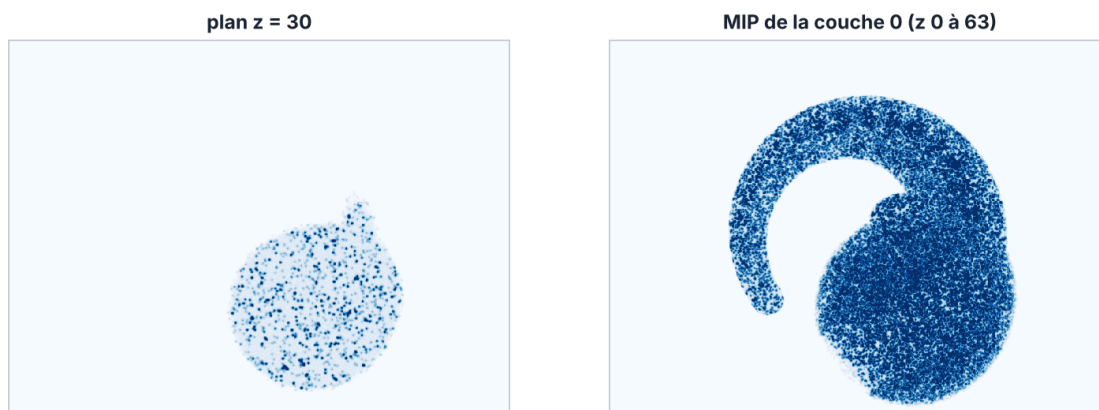
Un fichier plan commence par un en-tête `LPLN` (16 octets), suivi d'une table (décalage 8 octets + longueur 4 octets par tuile et par canal), puis des PNG en niveaux de gris, filtre « None », compression zlib niveau 6. Le navigateur les décode avec son propre décodeur PNG (jamais par lecture d'un canevas, qui n'est pas garantie exacte).

7.9 `mips/` : le maximum d'une couche de 64 plans

Une **projection d'intensité maximale** (MIP) garde, pour chaque pixel, la valeur la plus brillante rencontrée le long de Z. Pour un empilement épais, la calculer à la demande lirait tous les plans.

Le dossier `mips/` en garde donc une copie toute faite **pour chaque couche de 64 plans** (`l00000.bin` = plans 0 à 63, `l00001.bin` = plans 64 à 111...), à la pleine résolution XY.

Une coupe, et le maximum d'une couche de 64 plans (canal DAPI)



▲ Une coupe (à gauche) et le MIP de la couche de 64 plans (à droite). Jeu de démonstration, canal DAPI.

Pour une épaisseur quelconque, le Studio combine : les **MIP des couches entièrement contenues** dans l'épaisseur, et les **plans individuels** des deux extrémités.

Un MIP sur une épaisseur : les MIP de couches + les plans des extrémités



▲ Épaisseur de 220 plans : 3 MIP de couches + 28 plans d'extrémité, soit 31 images au lieu de 220, avec un résultat identique.

7.10 Les formats de données 1 à 4

Au fil des versions, l'organisation des fichiers a évolué. Chaque jeu de données porte un numéro, `formatVersion`, dans son `metadata.json`. Le format 4 est le format actuel.

FORMAT	CE QU'IL AJOUTE	MIGRATION
1	briques seules (v2 : 64 ³ sans bordure, mosaïque 8 × 8, un manifeste JSON)	—
2	<code>planes/</code> : coupes XY rapides	<code>m002-planes</code>
3	<code>mips/</code> : MIP par couche de 64 plans	<code>m003-layer-mips</code>

FORMAT	CE QU'IL AJOUTE	MIGRATION
4	briques v3 : bordure 66^3 , mosaïque 9 × 8, paquets en super-blocs, <code>index.bin</code>	<code>m004-bricks-v3</code>

Le pipeline actuel (version 0.21.0) écrit **directement le format 4**. Un jeu de données publié plus tôt n'est pas perdu : la plateforme sait le mettre à niveau.

« Mises à jour des données »

Comme une mise à jour de logiciel, l'onglet **Mises à jour des données** du panneau d'administration (chapitre 13) détecte les jeux de données restés en format 1, 2 ou 3 et les convertit **sur place**, un par un. Le niveau natif est conservé voxel pour voxel : rien n'est recalculé à partir de l'original.

Dans ce navigateur

Votre navigateur fait le travail (décodage et encodage WebP) et envoie les fichiers au serveur.

Sur le serveur

Le serveur convertit lui-même, si son hébergement sait encoder du WebP sans perte.

Pause et reprise

Le travail est découpé en petites unités notées dans un journal : on peut mettre en pause, fermer, reprendre, même en changeant d'exécutant.

📌 À RETENIR

- Une brique = 64^3 voxels utiles + 1 voxel de bordure = 66^3 , stockée en **une image WebP sans perte** de 594×528 pixels (9 × 8 coupes).
- Les briques sont dans des **paquets** (super-blocs de $4 \times 4 \times 4$) ; `index.bin` donne paquet, début et longueur de chacune (10 octets par brique).
- `planes/` accélère les coupes XY, `mips/` les projections ; format actuel = 4.

8. La fin du pipeline : métadonnées, suivi cellulaire, publication

- Après les briques, le pipeline fabrique la **vignette**, les **histogrammes** et le fichier `metadata.json` (stade, embryon, calibration, canaux), et, pour une série temporelle, le **suivi cellulaire**.
- Le résultat est publié « **tout ou rien** » : le jeu de données déjà en ligne n'est jamais laissé à moitié remplacé, et ce que vous avez saisi à la main est conservé.
- Vous lancez tout par `RUN.bat`, puis vous copiez le dossier produit sur le serveur (FTP) ou vous le glissez dans l'onglet **Import**.

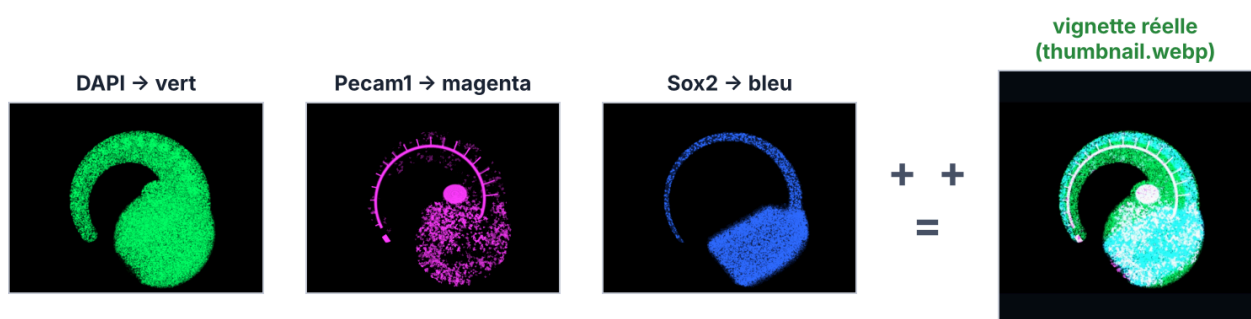
Les images de données de ce chapitre viennent du **jeu de démonstration** (embryons synthétiques générés pour cette documentation, traités par le vrai pipeline) : ce ne sont pas des données du laboratoire.

8.1 La vignette

La vignette est la petite image de la carte du jeu de données dans l'explorateur. Elle est faite à partir de la **première image** de l'acquisition, en prenant le niveau de résolution le plus fin dont le grand côté ne dépasse pas 1024 pixels.

Pour chaque canal, le pipeline calcule un **MIP** (le voxel le plus brillant sur toute l'épaisseur), le colorie, puis **additionne** les canaux : où deux canaux se chevauchent, les couleurs s'ajoutent.

La vignette : le MIP de chaque canal, colorié puis additionné



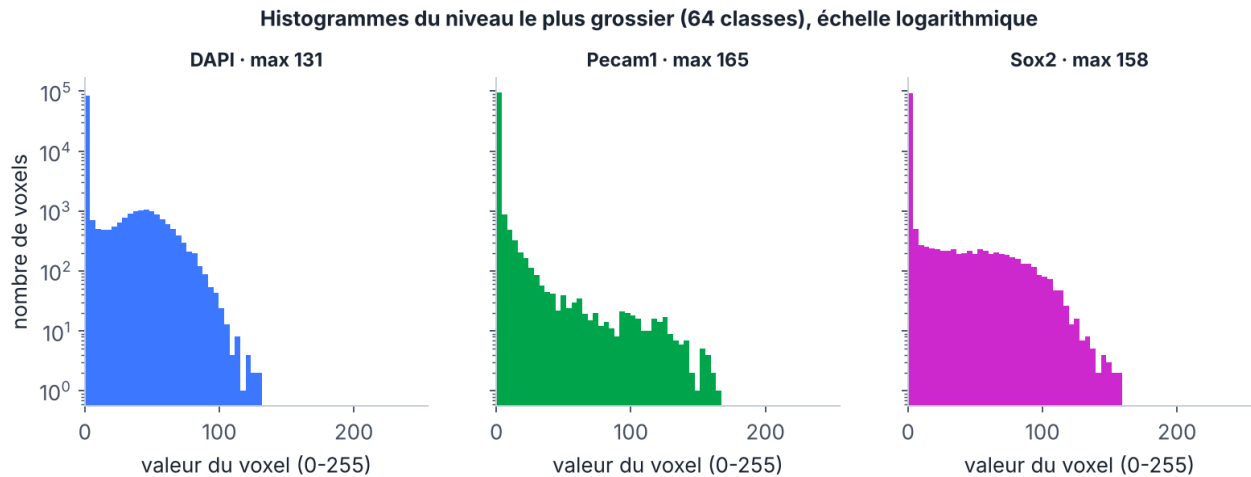
▲ Trois MIP coloriés, additionnés, donnent la vignette réelle du jeu de démonstration.

POUR LES CURIEUX

Couleurs de la vignette, dans l'ordre des canaux : vert, magenta, bleu, rouge, jaune, violet, cyan (rappelées en boucle). Elles n'ont pas de lien avec les couleurs choisies dans le viewer. L'image est réduite à 512 pixels pour son grand côté (interpolation de Lanczos), centrée sur un carré de fond `#080a12`, puis enregistrée en **WebP avec perte**, qualité 88 (33,6 ko ici).

8.2 Les histogrammes

Pour chaque canal, le pipeline compte combien de voxels ont chaque valeur de gris, sur le **niveau le plus grossier** (le plus rapide à lire). Les valeurs 0 à 255 sont réparties en **64 classes** de largeur 3,98.



▲ Les trois histogrammes stockés dans le manifeste du jeu de démonstration (échelle logarithmique : le fond à 0 écrase tout le reste en échelle linéaire).

Ils sont rangés dans `bricks/manifest.json` (`counts` , `edges` , `total` , `max` , `mean` , `std`) ; le viewer les affiche dans le panneau des canaux (chapitre 11). Pour une série temporelle, il y en a **un jeu par image**, car le photoblanchiment changerait l'échelle d'un jeu commun.

8.3 `metadata.json` : la carte d'identité du jeu de données

C'est le seul endroit où sont écrits les faits sur un jeu de données ; le catalogue du site est calculé à partir de lui à chaque requête. Extrait **réel** (abrégé) du jeu de démonstration :

```
{
  "id": "3d/Embryo-E95-Em2-Pecam1-Sox2",
  "formatVersion": 4,
  "type": "3d",
  "stage": "E9.5", "stageNumeric": 9.5, "embryo": "Em2",
  "dimensions": { "x": 768, "y": 576, "z": 112, "c": 3, "t": 1 },
  "voxel_size": { "x": 1.2, "y": 1.2, "z": 3.0 },
  "calibrationStatus": "exact",
  "channels": [ { "name": "DAPI", "color": "#3D7BFF",
    "min": 0.0, "max": 1.0, "gamma": 1.0 }, ... ],
  "description": "Confocal imaging stack: E9.5 fixed embryo, ...",
  "volumeSources": [ { "kind": "bricks", "multiscale": true, ... } ],
  "hidden": false
}
```

CHAMP	CE QUE C'EST
<code>id</code> , <code>type</code>	le type (<code>3d</code> ou <code>live</code>) et le dossier : <code>3d/<nom></code>
<code>formatVersion</code>	le format de données (chapitre 7) ; écrit par le pipeline, jamais édité à la main
<code>stage</code> , <code>embryo</code>	lus dans le nom du fichier (section suivante)

CHAMP	CE QUE C'EST
<code>dimensions</code> , <code>voxel_size</code>	taille en voxels et en micromètres, lue dans le fichier Imaris
<code>calibrationStatus</code>	<code>exact</code> si les trois tailles de voxel sont connues, sinon <code>metadata-missing</code> : aucune échelle n'est inventée
<code>channels</code>	nom, couleur, fenêtre (<code>min</code> , <code>max</code>) et gamma de départ de chaque canal
<code>volumeSources</code>	où le viewer trouve les briques

À SAVOIR

Pour une série temporelle (`live`), le fichier contient aussi `timeline` (nombre d'images, intervalle, horodatages) et `intensityNormalization` (la fenêtre de normalisation commune à toute la série, et un niveau de signal par image pour mesurer le photoblanchiment).

Stade et embryon, lus dans le nom du fichier

Le nom du fichier `.ims` est **interprété**. Le stade est un jeton `E...` encadré de tirets, de soulignés ou d'espaces ; l'embryon est un jeton `Em<n>` , ou, à défaut, le nombre qui suit directement un stade entier.

NOM (EXTRAIT)	STADE	EMBRYON	À REMARQUER
<code>...-E8.5-...</code> ou <code>E8,5</code>	E8.5	—	point ou virgule
<code>...-E8-5-...</code>	E8.5	—	tiret : confirmé par le laboratoire
<code>...-E85-...</code>	E8.5	—	forme compacte
<code>...-E105-...</code>	E10.5	—	deux chiffres de jour si ça commence par 1
<code>...-E95-Em2-...</code>	E9.5	Em2	embryon explicite
<code>...-E8-1-DAPI-...</code>	E8	Em1	<code>-1</code> n'est pas une fraction : après un tiret, seules .25, .5 et .75 en sont
<code>...-E95-1-...</code>	E9.5	Em1	le nombre après le stade
<code>Embryo-Pecam1</code>	inconnu	—	aucun jeton <code>E...</code>

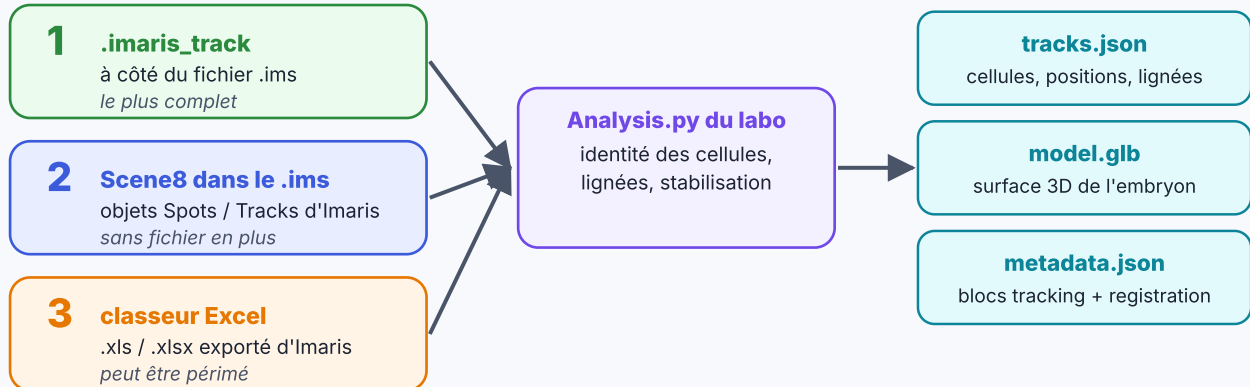
ATTENTION

Le jeton doit être encadré de tirets, de soulignés ou d'espaces : `-E85-Em1-` est lu correctement, `E85Em1` non. Si le stade est faux, corrigez-le dans l'onglet **Datasets** : le pipeline le conservera aux passages suivants (section 8.7).

8.4 Le suivi cellulaire (séries temporelles)

Pour une série temporelle seulement, le pipeline cherche une **analyse de suivi** (cellules, trajectoires, divisions) et l'attache au jeu de données. Il la cherche à trois endroits, du meilleur au moins bon.

Où le pipeline va chercher le suivi cellulaire (par ordre de priorité)



Le suivi n'est cherché que pour une série temporelle ; s'il échoue, le volume est publié quand même.

▲ Les trois sources possibles, du fichier le plus complet au classeur le plus fragile.

? POURQUOI CE CHOIX ?

Pourquoi Scene8 passe avant Excel ? Un classeur exporté peut être périmé : sur un jeu de données réel, il contenait 155 cellules alors que le volume en comptait 172. Les objets enregistrés dans le `.ims` sont toujours à jour. Le suivi est cherché automatiquement (option `--tracking auto`) ; si quelque chose échoue, le volume est publié quand même.

Le suivi apporte trois fichiers ou blocs :

- **tracks.json** (et sa copie compressée **tracks.json.gz**) : pour chaque cellule, son identifiant, sa région, ses positions à chaque image en μm (stabilisées et brutes), ses marqueurs (rouge = division, noir = fusion), sa cellule-mère et ses filles ;
- **model.glb** : la **surface 3D** de l'embryon à chaque image, reconstruite à partir des cellules (au moins 4 par image) ;
- dans **metadata.json**, les blocs **tracking** (compte de cellules, régions, origine des données) et **registration** (la transformation de stabilisation, ci-dessous).

1 2 3 4 MICRO-EXEMPLE

Le jeu de démonstration en temps. 50 cellules, 4 images, 3 régions (Posterior 18, Anterior 16, Lateral 16), 1 division et 0 fusion ; **model.glb** pèse 412 ko et **tracks.json** 32,7 ko. Les positions sont numérotées à partir de 1 (comme Imaris) et recalées sur les images du volume, numérotées à partir de 0 (décalage -1).

Stabiliser : annuler les mouvements de l'embryon

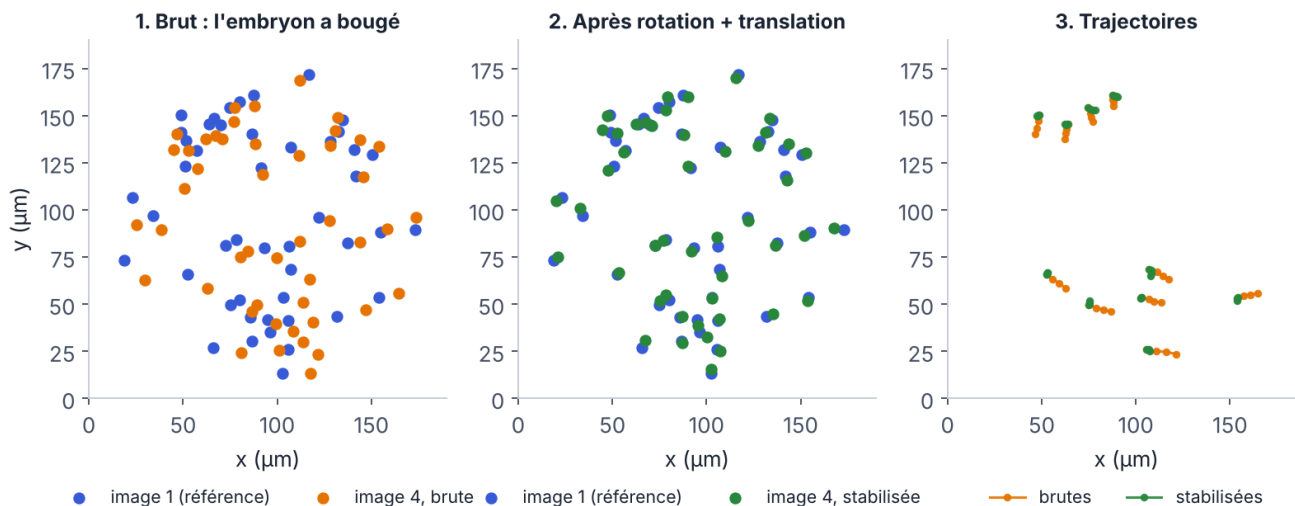
Pendant des heures, l'embryon dérive et tourne un peu dans le champ. Le pipeline **le remet dans la même position** à chaque image, afin que le mouvement visible soit celui des cellules et non celui de

l'échantillon.

UNE IMAGE POUR COMPRENDRE

Un calque transparent. Dessinez les cellules de l'image 1 sur un calque, puis celles de l'image 4 sur un autre. Pour les superposer, vous faites glisser le calque et vous le faites pivoter, mais vous ne l'étirez jamais : c'est exactement un mouvement « rigide ».

Sur le jeu de démonstration, voici la même idée avec les vraies cellules (47 cellules présentes aux images 1 et 4, projection sur le plan x-y).



▲ À gauche, les cellules de l'image 4 ont dérivé ; au centre, après rotation de 7,03 ° et translation, elles se superposent à l'image 1 ; à droite, les trajectoires brutes dérivent, les stabilisées restent en place.

L'algorithme utilisé s'appelle **Kabsch** (décomposition en valeurs singulières). Il choisit des **cellules de référence** (celles qui ne se divisent pas, présentes à au moins deux images) et cherche, image après image, la rotation et la translation qui les superposent au mieux ; il en faut au moins 3 en commun.

POUR LES CURIEUX

Les maths. Soient P les positions stabilisées de l'image n-1 et Q les positions brutes de l'image n (cellules de référence), centrées sur leurs centres de gravité c_P et c_Q . On forme $H = Q_C^T P_C$, on la décompose en $H = U S V^T$, puis $R = V \cdot \text{diag}(1, 1, \text{signe}(\det(V U^T))) \cdot U^T$ (une vraie rotation, jamais un miroir) et $t = c_P - R c_Q$. La transformation (R, t) est appliquée à **toutes** les cellules de l'image n. La première image est la référence.

Le bloc **registration** : appliquer la même chose aux images

Le suivi est stabilisé, mais **pas les images**. Comme le mouvement est rigide, la même transformation permet au viewer de superposer les trajectoires aux images en déplaçant simplement les coordonnées de lecture, **sans rééchantillonner** un seul voxel. Pour chaque image, le fichier garde une matrice 4 × 4, l'angle et la translation :

IMAGE	ROTATION	TRANSLATION (MM)	RÉSIDU (MM)
1	0°	(0 ; 0 ; 0)	4×10^{-14}
2	2,33°	(-5,65 ; 5,01 ; -0,37)	6×10^{-14}

IMAGE	ROTATION	TRANSLATION (MM)	RÉSIDU (MM)
3	4,66°	(-11,09 ; 10,60 ; -1,01)	4×10^{-14}
4	7,03°	(-16,14 ; 16,39 ; -1,79)	8×10^{-14}

📌 À RETENIR

Test de rigidité. Le pipeline vérifie que la transformation est bien rigide : le plus grand résidu doit rester $\leq 0,05 \mu\text{m}$. Ici il vaut $7,6 \times 10^{-14} \mu\text{m}$ (zéro, à l'erreur d'arrondi près), donc la stabilisation est appliquée aux images. Si le test échouait, un avertissement l'indiquerait et elle ne serait pas appliquée.

8.5 Publier « tout ou rien »

Un traitement dure parfois des heures. Pendant ce temps, **le jeu de données déjà en ligne continue de fonctionner** : tout est construit dans un dossier de travail privé, au même endroit du disque que `DATA_WEB`, et n'est installé qu'à la fin.

💡 UNE IMAGE POUR COMPRENDRE

La vitrine d'un magasin. On prépare la nouvelle vitrine dans l'arrière-boutique ; on n'enlève l'ancienne qu'une fois la nouvelle entièrement montée. Si un problème survient, les clients voient toujours l'ancienne.

Publication « tout ou rien » : on ne change la vitrine que quand la nouvelle est prête



Échec AVANT l'étape 4 : tout est remis comme avant ; le jeu de données publié n'a jamais cessé de fonctionner.

Plantage en plein milieu : au lancement suivant, le marqueur permet de finir ou d'annuler proprement, selon que `metadata.json` (dont l'empreinte sha256 est dans le marqueur) est déjà en place ou non.

▲ Les cinq temps de la publication ; `metadata.json` est écrit en dernier, c'est le point de non-retour.

Un **nouveau** jeu de données arrive en un seul renommage de dossier. Pour un jeu **existant**, seuls les éléments du pipeline sont remplacés (`bricks/` , `planes/` , `mips/` , la vignette, et `tracks.json` , `tracks.json.gz` , `model.glb` si un nouveau suivi existe). Tout le reste du dossier, comme `download/` ou `gallery/` , n'est jamais touché.

8.6 Ce que le pipeline conserve quand on retraite un jeu de données

Vous avez corrigé un nom, orienté l'embryon, ajouté des légendes dans l'administration ? Un nouveau passage du pipeline **ne l'écrase pas** : il fusionne, juste avant l'installation, les champs saisis à la main dans le nouveau `metadata.json`.

- **Identité** : `name`, `description`, `stage`, `stageNumeric`, `embryo`, `line`, `staining`, `reporter`, `tags`, `notes`, `created` ;
- **Affichage** : `hidden`, `orientation`, `orientationAxes`, `upsideDown`, `defaultView`, `exposure` ;
- **Liens et images** : `gallery`, `linkedTrackingId`, `relatedIds`, et tout champ que le nouveau passage ne produit pas (par exemple `tracking`).

Les couleurs et réglages des canaux sont aussi conservés, tant que le **nombre de canaux** n'a pas changé. À l'inverse, `formatVersion` n'est **jamais** repris : il reflète toujours la structure réellement écrite sur le disque.

⚠ ATTENTION

Si un plantage survient pendant l'installation (coupure de courant), le marqueur `.swap-in-progress` permet au passage suivant de **finir ou d'annuler** proprement le changement. Ne supprimez pas ce fichier à la main.

8.7 Le dossier `download/`

Avec l'option `--with-downloads`, le pipeline prépare, **après** la publication (un échec ici ne défait donc jamais le jeu de données), un dossier de fichiers à télécharger. Contenu réel du jeu de démonstration :

```
download/
├─ Embryo-E95-Em2-Pecam1-Sox2.ims          255,8 Mo  l'original
├─ Embryo-E95-Em2-Pecam1-Sox2.tif          254,6 Mo  composite ImageJ
├─ Embryo-E95-Em2-Pecam1-Sox2_C1_DAPI_MIP.png    368 ko   un MIP par canal
├─ ..._C2_Pecam1_MIP.png ..._C3_Sox2_MIP.png
├─ Embryo-E95-Em2-Pecam1-Sox2_web.zip          18,4 Mo  le jeu de données web
└─ README.txt                                provenance
```

- L'**original .ims** est un **lien physique** (« hard link ») : il apparaît dans `download/` sans occuper de place en plus ; il n'est jamais modifié.
- Le **.tif** est un composite pour ImageJ/Fiji au **format natif** (8 ou 16 bits, valeurs brutes), calibré en μm , avec les couleurs et la plage d'affichage de chaque canal. Il est tiré du niveau de résolution Imaris le plus proche de 2048 pixels.
- Les **MIP en PNG** donnent un aperçu par canal (normalisation entre les 1^{er} et 99,9^e centiles).
- Le **_web.zip** archive le dossier servi, sans recompression (WebP et PNG le sont déjà).
- Le **README.txt** rappelle dimensions, tailles de voxel, canaux et comment citer le jeu de données.

Pour une série temporelle, le `.tif` et les MIP ne montrent qu'**une image** (la première) ; le `.ims` les contient toutes.

8.8 Les photographies 2D, en une demi-page

Les photographies (stéréomicroscope, TIFF exporté depuis un fichier Leica) ont leur **propre outil**, `2d_importer.py` : un fichier TIFF donne un dossier `2d/<nom>/`.

FICHIER	RÔLE	JEU DE DÉMONSTRATION
<code>image.webp</code>	l'image en taille native	1920 × 1440 px, 191 ko
<code>preview.webp</code>	aperçu de 640 px, affiché d'abord	10,6 ko
<code>thumbnail.webp</code>	vignette 512 px	12,8 ko
<code>metadata.json</code>	stade, ligne, date, taille de pixel (2,0 µm ici)	écrit en dernier

? POURQUOI CE CHOIX ?

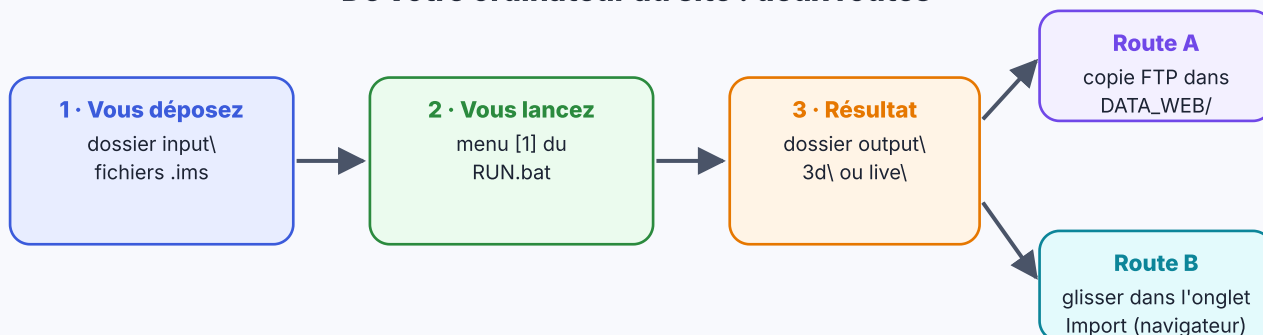
Pourquoi WebP « avec perte » par défaut ? Il divise la taille par environ 40 par rapport au TIFF d'origine (8,3 Mo contre 191 ko ici), avec une qualité de 90. Mais le WebP avec perte **sous-échantillonne la couleur** (la chrominance est stockée avec moitié moins de pixels). Or l'isolement de la coloration X-gal du viewer lit le **rapport bleu/rouge** pixel par pixel : sur les contours nets, ce rapport est légèrement perturbé. L'option `--lossless` conserve chaque pixel exact, au prix de fichiers plus gros.

La taille de pixel n'est lue que si le TIFF déclare une unité en microns ; sinon `pixelSizeUm` reste vide et le viewer n'affiche **ni échelle ni mesure**. Le stade, le zoom et la date de dissection sont lus dans le nom du fichier, avec le même lecteur de stade que les volumes.

8.9 Lancer tout cela : RUN.bat

Le **pack Pipeline** (un dossier à décompresser sur un PC Windows) contient tout. Double-cliquez sur `RUN.bat` : il vérifie l'intégrité du pack (empreintes SHA-256), prépare Python et les bibliothèques, puis affiche un menu.

De votre ordinateur au site : deux routes



Dans les deux cas, le jeu de données apparaît dans le catalogue sans rien régénérer.

▲ De votre ordinateur au site : on dépose les fichiers, on lance, puis on copie ou on glisse le résultat.

CHOIX	CE QUE FAIT LE MENU
[1]	Preprocessing de volumes Imaris (.ims → <code>output\</code> , suivi inclus)
[2]	Analyse de tracking Imaris (Excel → <code>tracking\OUTPUT\</code>)
[3]	Import de photographies 2D (.tif → <code>output\2d\</code>)
[4]	Attacher un tracking à un dataset déjà traité
[5]	Vérifier l'environnement seulement
[0]	Quitter

Le menu pose quelques questions (dossier des `.ims` , dossier de sortie, archives de téléchargement ou non) puis lance `run_preprocess.py` . Vous pouvez aussi l'appeler vous-même :

```
python run_preprocess.py --input input --output DATA_WEB --only "*E95*" --with-downloads
```

OPTION	EFFET
<code>--input</code>	dossier des <code>.ims</code> (non récursif, un fichier à la fois)
<code>--output</code>	la racine <code>DATA_WEB</code> : le résultat va dans <code>3d/<nom>/</code> ou <code>live/<nom>/</code>
<code>--only</code>	filtre sur le nom des fichiers (ex. <code>"*E8*"</code>)
<code>--with-downloads</code>	prépare aussi le dossier <code>download/</code> (plus long)
<code>--tracking</code>	<code>auto</code> (défaut), <code>off</code> , ou le chemin d'un fichier de suivi

Le type est décidé tout seul : plus d'une image dans le fichier, c'est `live` , sinon `3d` . `Ctrl+C` demande confirmation avant d'arrêter, et nettoie les fichiers temporaires.

Comment le résultat arrive sur le serveur

- **Route A, par FTP** : copiez le dossier produit (`3d/<nom>` ou `live/<nom>`) dans le `DATA_WEB` du serveur. Il apparaît aussitôt dans le catalogue, il n'y a rien à régénérer.
- **Route B, sans FTP** : dans le panneau d'administration, ouvrez l'onglet `Import` et glissez le dossier dans la zone `Glissez un dossier ici` . Le transfert reprend là où il s'est arrêté si vous le relancez ; ensuite, cliquez sur `Publier` .

⚠ ATTENTION

Un jeu de données publié par l'onglet `Import` est **masqué par défaut** : allez dans l'onglet `Datasets` et activez sa visibilité pour qu'il apparaisse dans l'explorateur public (chapitre 13).

À RETENIR

- La vignette, les histogrammes et `metadata.json` décrivent le jeu de données ; le suivi cellulaire est une **couche** d'une série temporelle, stabilisée par un mouvement rigide.
- La publication est **tout ou rien**, et ce que vous avez corrigé à la main est conservé aux retraitements.
- `RUN.bat` → choix `[1]` → dossier `output\` → FTP ou onglet `Import` .

9. Le viewer 3D : Three.js et le lancer de rayons

- Chaque pixel de l'écran est calculé par **un rayon** qui traverse le volume : c'est le **lancer de rayons**, exécuté par la carte graphique.
- **Three.js** fournit la scène, la caméra et les textures ; le calcul qui compte pour la science (les trois modes de rendu) est **écrit sur mesure** et reste lisible.
- Pendant que vous tournez l'embryon, la netteté baisse un instant pour garder la fluidité ; au repos, tout revient à pleine qualité.

9.1 Deux cerveaux : le processeur et la carte graphique

Votre ordinateur possède deux types de « cerveaux ». Le **processeur** (CPU) est généraliste. La **carte graphique** (GPU) est spécialisée dans une seule chose : faire le même petit calcul sur des milliers de points à la fois.

Pourquoi le calcul se fait sur la carte graphique

LE PROCESSEUR (CPU) : un chef



Très habile, mais seul
prépare les assiettes une par une

1 920 × 1 080 pixels = 2 millions d'assiettes

→ trop lent pour tourner un embryon en direct

LA CARTE GRAPHIQUE (GPU) : des commis



Chacun fait le même petit geste
... chacun sur son propre pixel, tous en même temps

le petit geste = le « shader » (programme du pixel)

→ une image fluide, 60 fois par seconde visées

▲ Un grand chef contre des milliers de commis : la carte graphique gagne dès qu'il faut répéter le même geste des millions de fois.

💡 UNE IMAGE POUR COMPRENDRE

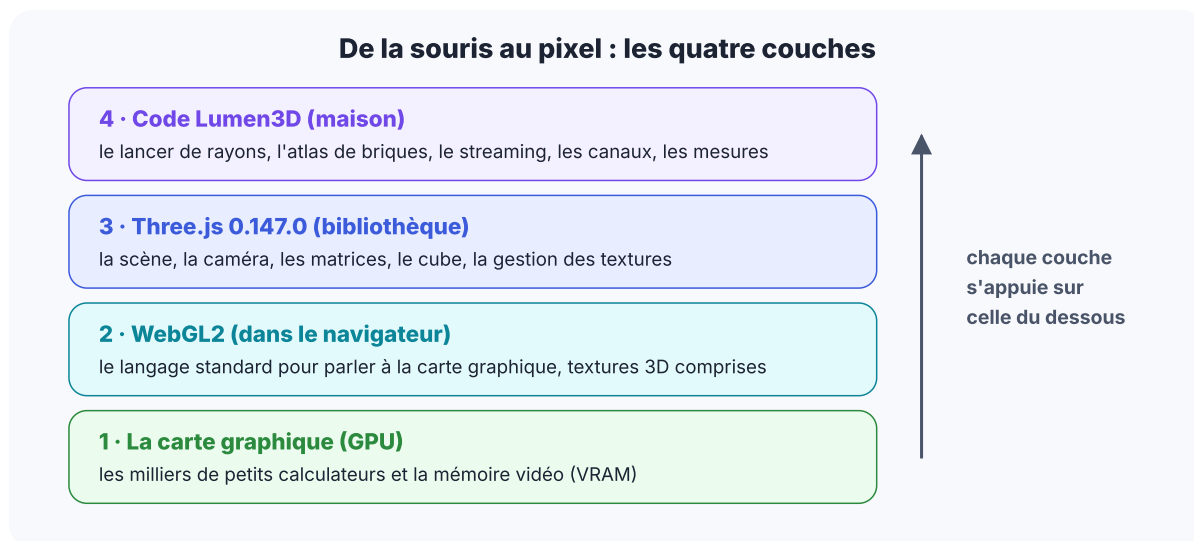
Un grand chef étoilé prépare une assiette parfaite en deux minutes. Pour servir 2 millions de convives, il lui faudrait des années. Mille commis, chacun avec **la même recette simple**, servent la salle en un instant. Un écran de 1 920 × 1 080 pixels, c'est exactement cette salle.

Dans Lumen3D, la « recette » s'appelle un **shader** : un petit programme écrit une fois, exécuté simultanément pour chaque pixel. Tout le calcul du volume se passe là.

9.2 WebGL2 et Three.js : qui fait quoi

WebGL2 est le langage standard que les navigateurs comprennent pour parler à la carte graphique. Il sait notamment manipuler des **textures 3D** : des blocs de voxels stockés directement dans la mémoire vidéo. C'est indispensable pour un volume.

Three.js (version **0.147.0**) est une bibliothèque qui se place au-dessus de WebGL2 et s'occupe de la « plomberie ». Elle est **hébergée sur le serveur de la plateforme** (dossier `js/vendor/`), pas téléchargée depuis Internet.



▲ Les quatre couches entre votre souris et le pixel affiché.

FOURNIS PAR THREE.JS	ÉCRIT SUR MESURE POUR LUMEN3D
Le moteur de dessin (le « renderer »), la scène	Les shaders du lancer de rayons (3 modes de rendu)
La caméra en perspective (angle de vue 45°)	L' atlas de briques et sa table des pages
Le cube (une boîte à 6 faces qui sert de point de départ aux rayons)	Le streaming des briques (chapitre 10)
Les matrices, quaternions, vecteurs	La qualité adaptative et le budget de mémoire
L'enveloppe des textures 3D	La lecture de profondeur sous la souris (mesures)
Le chargeur de modèles 3D (surface du suivi cellulaire)	Les canaux, histogrammes, filtre gaussien

? POURQUOI CE CHOIX ?

- **Mûr et léger** : Three.js est une bibliothèque très répandue, qui ne demande aucun outil d'assemblage (pas de « build ») : un fichier, un navigateur.
- **Hors ligne** : copiée sur le serveur, elle fonctionne sans Internet.
- **Auditable** : la partie qui détermine ce que vous voyez (les formules de rendu) est du code du laboratoire, commenté ligne à ligne. Un moteur de jeu complet n'était pas nécessaire : il aurait apporté des milliers de fonctions inutiles à la microscopie.

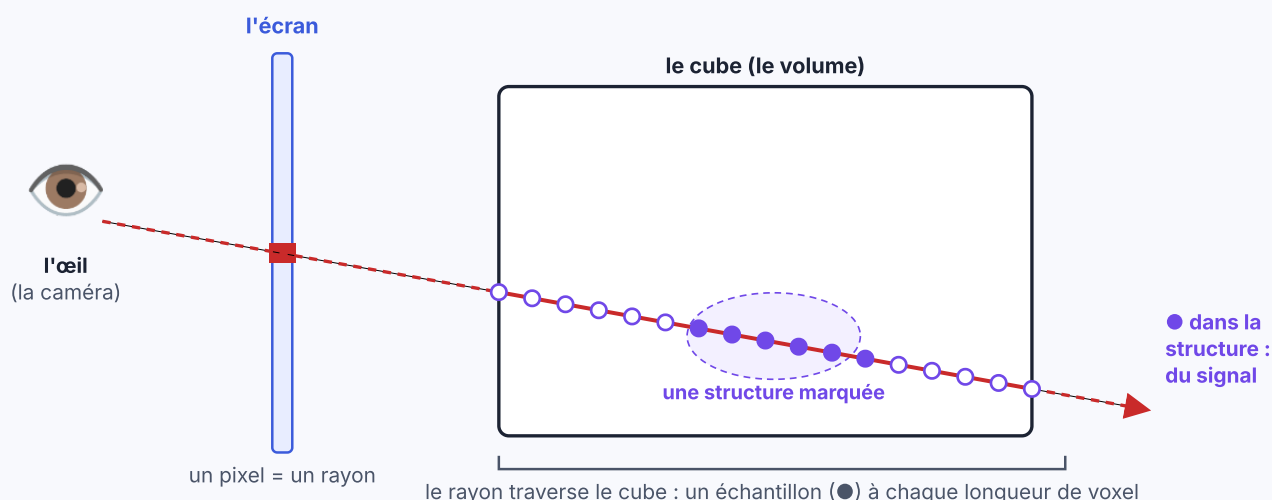
i À SAVOIR

Le viewer exige **WebGL2** (il n'existe pas de version dégradée en WebGL1). Si la carte graphique ne sait pas créer de textures 3D, le chargement est refusé avec un message clair, jamais par un plantage de l'onglet.

9.3 Le lancer de rayons, pas à pas

Le volume est un énorme tableau de valeurs. Pour en faire une image, on procède **pixel par pixel**. Chaque pixel de l'écran lance un **rayon** depuis l'œil, qui traverse le cube. On lit le volume le long de ce rayon, puis on résume ce qu'on a rencontré en une seule couleur.

Le lancer de rayons : un rayon par pixel



1 · DÉPART

On lance un rayon de l'œil, à travers ce pixel.

2 · ENTRÉE

On calcule où le rayon entre et sort du cube.

3 · PAS

Tous les 1 voxel : on lit la valeur de chaque canal.

4 · COMBINER

Les échantillons se combinent en UNE couleur : le pixel.

▲ Un rayon par pixel : entrée dans le cube, un échantillon par voxel, puis combinaison.

- 1 Départ.** Le pixel désigne une direction : celle qui part de l'œil et passe par ce pixel.
- 2 Entrée et sortie.** Le shader calcule où le rayon entre dans le cube et où il en sort (c'est « l'intersection rayon / boîte »). Hors du cube, le pixel reste transparent.
- 3 Pas régulier.** On avance d'un **voxel** à chaque fois et on lit la valeur de chaque canal.
- 4 Combinaison.** Les lectures sont résumées en une couleur, selon le mode de rendu (section 9.6).

Un échantillon par voxel

Au repos, le viewer prend **exactement un échantillon par longueur de voxel**, quelle que soit la direction du rayon. C'est ce qui garantit qu'une **structure d'un seul voxel ne passe jamais à travers**

les mailles du filet.

MICRO-EXEMPLE

Le jeu de démonstration E9.5 fait $768 \times 576 \times 112$ voxels. Un rayon qui traverse tout le cube dans la longueur (axe X) fait donc **768 échantillons**. Un rayon en diagonale, plus long ($\sqrt{768^2 + 576^2 + 112^2} \approx 966$ voxels), en fait **environ 966**.

POUR LES CURIEUX

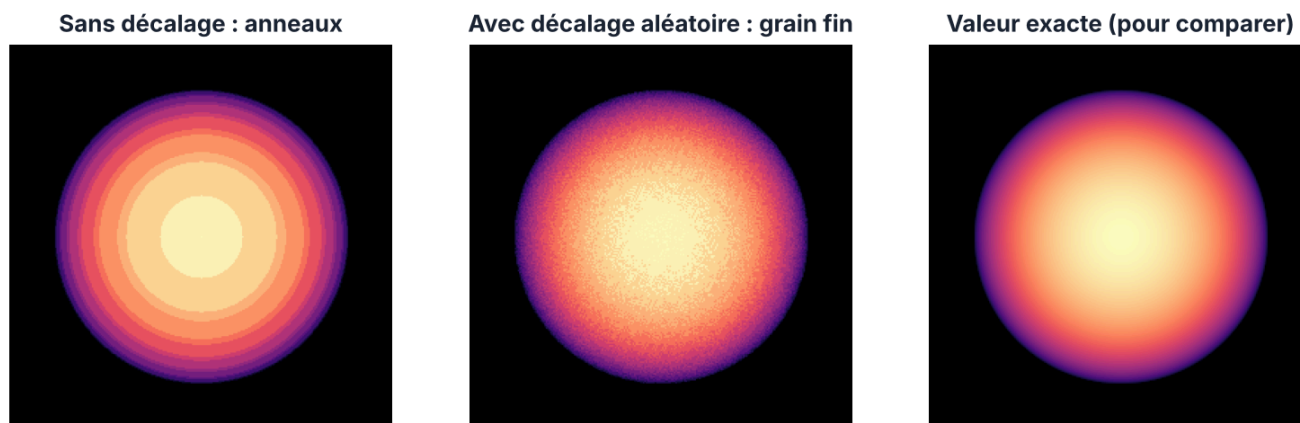
Le pas (en unités du cube) est $\text{delta} = 1 / (\text{taux} \times \text{nu})$, où $\text{nu} = \|\text{direction} \times (\text{nombre de voxels du niveau})\|$ mesure combien de voxels le rayon traverse par unité de longueur et $\text{taux} = 1,0$ au repos. Le pas est donc uniforme en voxels, pas en micromètres. Si le rayon demandait plus de 4 096 pas, ils sont répartis plus largement (voir 9.9).

9.4 Le grain aléatoire contre les bandes

Avec un pas fixe, tous les rayons voisins échantillonnent à la **même profondeur**. Résultat : des **anneaux** (« banding »), comme les courbes de niveau d’une carte. Ils n’existent pas dans l’échantillon : ils sont créés par le calcul.

La parade est simple : le **premier échantillon de chaque rayon est décalé** d’une fraction aléatoire du pas (un nombre pseudo-aléatoire calculé à partir de la position du pixel). Les anneaux se transforment en un **grain fin**, que l’œil moyenne et ignore.

Une sphère uniforme, un échantillon tous les 14 pixels (simulation)

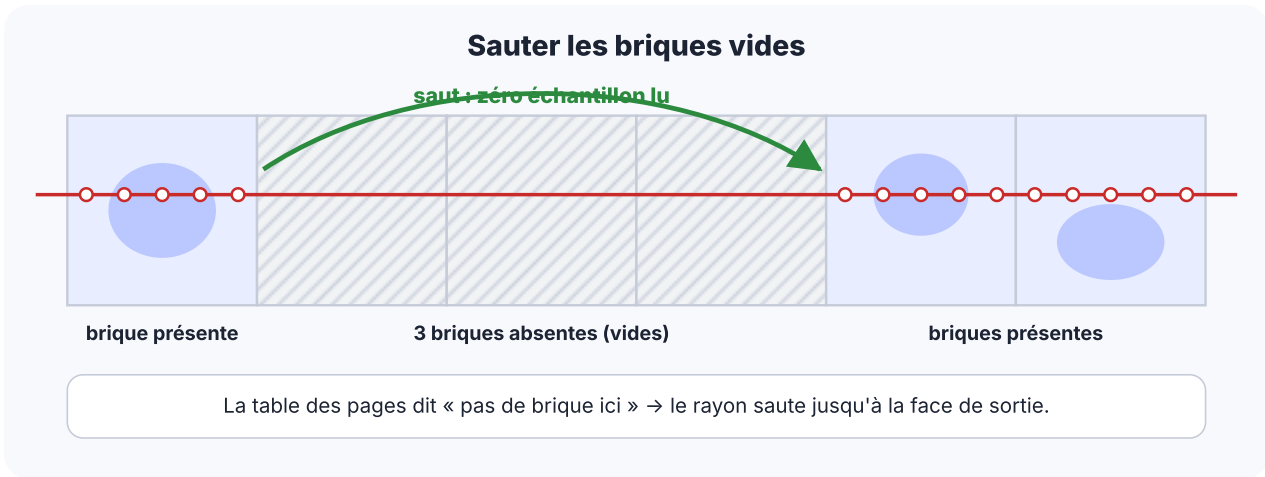


▲ Simulation : sphère uniforme, un échantillon tous les 14 pixels. À gauche, les anneaux ; au milieu, le grain après décalage aléatoire.

Le même décalage est utilisé à l’export d’une image : il se poursuit d’une tuile à l’autre, sans couture.

9.5 Sauter le vide

Un embryon n’occupe qu’une partie de sa boîte. Les briques sans signal ne sont même pas stockées (chapitre 7). Quand un rayon arrive dans une brique **absente**, il n’y lit rien : il **saute directement** à la face de sortie de cette brique.



▲ Le rayon franchit trois briques vides sans lire un seul voxel.

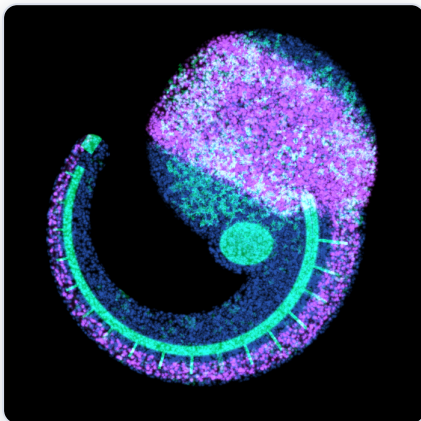
Le saut respecte le même quadrillage aléatoire que le reste du rayon : l'image ne change pas, elle est seulement calculée plus vite.

i À SAVOIR

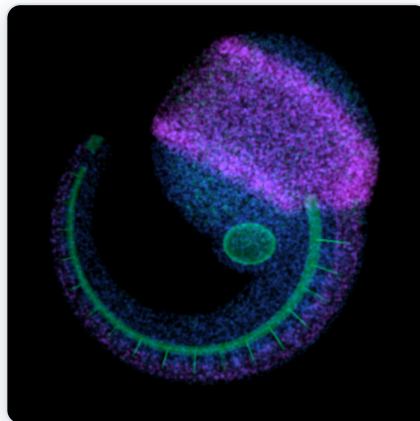
Une brique absente se lit comme des **zéros** : du noir qui ne coûte presque rien, pas un trou dans l'espace.

9.6 Les trois modes de rendu

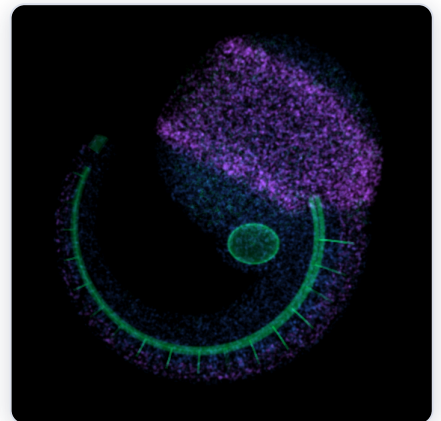
Le menu **Mode de rendu** (panneau de gauche) propose trois façons de résumer les valeurs d'un rayon. Voici **le même point de vue** du jeu de démonstration E9.5 (DAPI, Pecam1, Sox2), rendu trois fois.



▲ **Fluorescence (type Imaris)**



▲ **Fluorescence naturelle (profondeur)**



▲ **Structure (DVR)**

i À SAVOIR

Les captures du milieu et de droite sont éclairées avec un curseur **Visibilité (exposition)** plus haut (voir la légende de la figure suivante) : ces deux modes produisent naturellement une image plus sombre à exposition égale.

MODE	CE QU'IL FAIT LE LONG DU RAYON	PROFONDEUR ?
Fluorescence (mode par défaut)	garde la valeur la plus forte de chaque canal	non, tout se superpose
Fluorescence naturelle	chaque point brille ; ce qui est devant assombrit ce qui est derrière	oui
Structure (DVR)	empile des couches semi-transparentes de l'avant vers l'arrière	oui

Mode 1 : Fluorescence (le maximum)

C'est le rendu « classique » des logiciels de microscopie. Pour **chaque canal séparément**, le viewer retient la **valeur maximale** rencontrée sur le rayon. Les couleurs des canaux sont ensuite **additionnées**.

MICRO-EXEMPLE

Un rayon croise du vert à 0,85 puis du magenta à 0,75. Le pixel reçoit 0,85 de vert **et** 0,75 de magenta : le magenta n'est pas caché par le vert qui est devant. Vert + magenta additionnés donnent du blanc là où les deux sont forts.

POUR LES CURIEUX

$mip = \max(mip, v)$ pour chaque canal ; couleur finale = $\sum mip_i \times couleur_i \times exposition$. Le rayon s'arrête tôt quand **chaque** canal actif a atteint son plafond (opacité $\times 0,999$). Un pixel dont le maximum est inférieur à 0,004 reste transparent.

Mode 2 : Fluorescence naturelle (émission et absorption)

Image mentale : chaque fluorophore est une **petite lampe** qui brille dans sa couleur, et la matière dense devant elle **filtre** la lumière. On obtient du relief et de la profondeur, tout en gardant des couleurs fidèles.

POUR LES CURIEUX

Pour chaque échantillon de densité d = valeur du canal le plus fort (si $d > 0,0025$) :

- opacité du segment (loi de Beer-Lambert) : $a = 1 - \exp(-absorption \times delta \times d)$ avec $absorption = 1,8$;
- lumière émise : $T \times emissionGain \times exposition \times delta \times \sum v_i \times couleur_i$ avec $emissionGain = 2,2$, où T est la **transmittance** (la part de lumière qui n'a pas encore été absorbée, de 1 à 0) ;
- puis $T \leftarrow T \times (1 - a)$; arrêt quand $T < 0,004$.

À la fin, la **luminance** (formule Rec. 709) est compressée par une courbe de Reinhard étendue (point blanc 2,0) et les trois composantes sont multipliées par **le même rapport** (« verrouillage de la chromaticité ») : un fluorophore très brillant reste vert, il ne vire jamais au blanc. Seul le recouvrement de 3 ou 4 canaux est éclairci (au plus 50 %). Une saturation de 1,18 est appliquée à luminance constante.

⚠ ATTENTION

L'absorption n'est **pas une mesure** : c'est une constante d'affichage (1,8), indépendante de l'échantillon. Elle sert à donner du relief, pas à quantifier l'opacité réelle du tissu.

Mode 3 : Structure (DVR)

Le DVR (« Direct Volume Rendering ») empile des couches **semi-transparentes** comme des vitres teintées, en partant de la plus proche. Une fois la pile presque opaque (97 %), le rayon s'arrête : ce qui est derrière n'est plus visible.

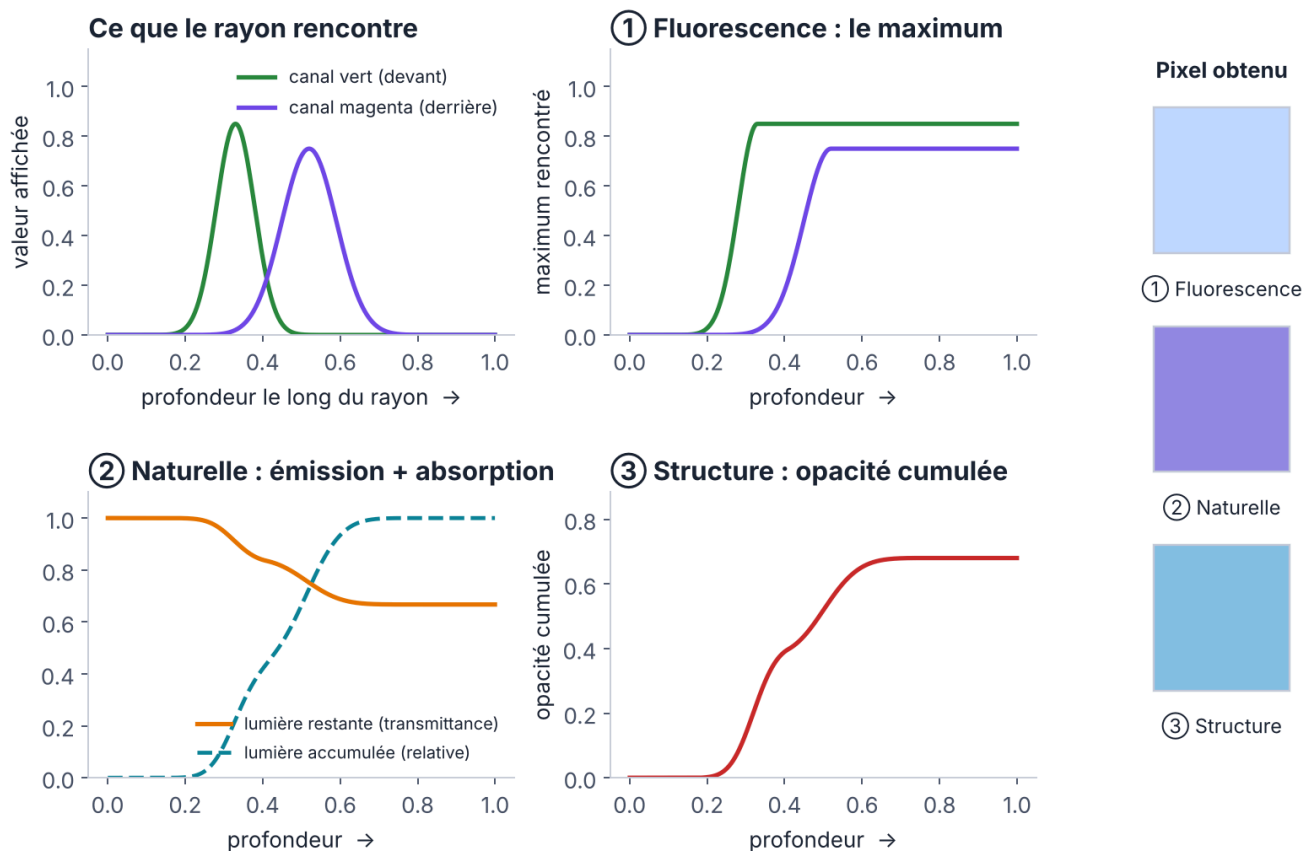
🔧 POUR LES CURIEUX

Pour chaque échantillon d'opacité locale $a = \max \text{ des canaux}$ (si $a > 0,01$) : $\text{pas} = 1 - (1 - 0,05 \times a)^{(\text{delta} / 0,01)}$ puis $\text{couleur} += (1 - \alpha) \times \text{pas} \times \text{couleur_locale}$ et $\alpha += (1 - \alpha) \times \text{pas}$.

L'exposant fait qu'une colonne de matière opacifie **selon sa longueur**, pas selon le nombre d'échantillons : l'image ne change pas quand le viewer allège le pas pendant la rotation.

Un rayon, trois réponses

Imaginons un rayon qui rencontre d'abord un nuage vert, puis, un peu derrière, un nuage magenta qui le recouvre en partie.



▲ Le même rayon, résumé par les trois modes (calcul avec les formules du viewer ; curseur d'exposition 1 pour le mode 1, 4 pour le mode 2 et 3 pour le mode 3, comme sur les captures).

- **Fluorescence** : on garde le plus fort de chaque canal, **sans tenir compte de l'ordre**.

- **Naturelle** : la lumière du nuage magenta est un peu **atténuée** par le nuage vert qui est devant (la transmittance descend à environ 0,67).
- **Structure** : le nuage vert **cache** partiellement le magenta ; l'opacité cumulée finit autour de 0,68.

9.7 Lisser entre les voxels

Entre deux voxels, le viewer ne montre pas des marches d'escalier : il **interpole**. Sur le **format 4** (celui de tous les jeux de démonstration), la carte graphique mélange les huit voxels voisins : c'est l'**interpolation trilinéaire**.

Pour que ce mélange ne déborde jamais d'une brique sur sa voisine, chaque brique est stockée avec **1 voxel de bordure** (66^3 au lieu de 64^3). Aux frontières entre briques, l'image est donc continue, sans couture.

📌 À RETENIR

- **Format 4** et texture 3D complète : interpolation trilinéaire (lisse).
- **Ancien format 2**, quand l'atlas est découpé en briques : le voxel le plus proche est affiché tel quel (blocs visibles en zoom).

9.8 Fluide quand vous bougez, net quand vous vous arrêtez

Faire tourner un volume à 60 images par seconde demande de calculer chaque image en moins de **16,7 ms**. Le viewer mesure le temps de chaque image pendant que vous glissez et **ajuste deux réglages** pour tenir ce budget.

Résolution adaptative : fluide quand vous bougez, nette quand vous vous arrêtez

AU REPOS : qualité maximale



**1 échantillon
par voxel**

pixels : 1 par pixel
de l'écran (jusqu'à $\times 2$
sur écran Retina)

PENDANT LE GLISSEMENT : allégé



**0,1 à 0,75 échantillon
par voxel**

pixels : de $\times 0,25$ à $\times 1$
de la définition de repos
(départ : $\times 0,75$ et 0,35)

Le régulateur : viser 16,7 ms par image (60 images/s)

1 • on mesure
le temps d'une image

trop lent ($> 1,3 \times 16,7$ ms)
→ moins de pixels, puis moins d'échantillons

assez rapide ($\leq 1,1 \times$)
→ on remonte en douceur

▲ Pendant le glissement, la définition de l'image et le nombre d'échantillons sont réglés en continu ; dès que vous vous arrêtez, tout revient à la qualité maximale.

RÉGLAGE	PENDANT LE GLISSEMENT	AU REPOS
Définition de l'image	de $\times 0,25$ à $\times 1$ (départ $\times 0,75$)	1 pixel d'écran (jusqu'à $\times 2$)

RÉGLAGE	PENDANT LE GLISSEMENT	AU REPOS
Échantillons par voxel	de 0,1 à 0,75 (départ 0,35)	1,0

- L'interaction est considérée comme terminée **250 ms** après le dernier mouvement.
- Si une image dépasse 1,3 × 16,7 ms, on baisse d'abord la définition, puis les échantillons ; si elle reste sous 1,1 ×, on remonte doucement.
- Le temps est mesuré par le **chronomètre de la carte graphique** quand le navigateur le propose, sinon par l'intervalle entre deux images.

À SAVOIR

Rien n'est dessiné en continu : le viewer ne calcule une image que si quelque chose change (caméra, réglage, arrivée de briques). Pendant le chargement, les redessins dus aux nouvelles briques sont groupés **toutes les 250 ms** au plus.

9.9 Le garde-fou contre les plantages de la carte graphique

Sous Windows, si un seul calcul de la carte graphique dure plus de **2 secondes environ**, le système la réinitialise : c'est un « TDR » et l'écran clignote. Le viewer s'en protège.

POUR LES CURIEUX

Pour une image **au repos**, le nombre maximal d'échantillons par rayon est `clamp([6 × 109 / nombre de pixels], 256, 4096)`. Exemple : un écran de 1 920 × 1 080 donne `[6 × 109 / 2 073 600] = 2 893` échantillons au plus par rayon ; le même écran en pixels doublés (Retina, 8,3 millions de pixels) en donne 723.

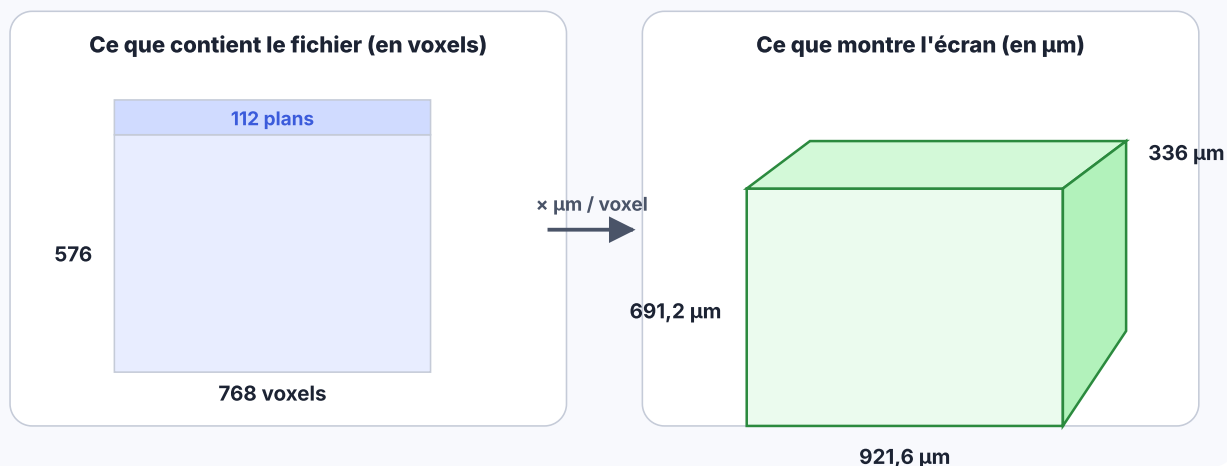
Ce plafond est ajusté en cours d'usage : multiplié par `max(0,5 ; 150 / durée)` si une image au repos dépasse 200 ms, relevé de 10 % si elle reste sous 60 ms, et divisé par deux après une perte de contexte graphique (chapitre 10). Pendant un glissement, il vaut un dixième de cette valeur.

Pour vous, cela signifie qu'un rayon exceptionnellement long peut, sur un très grand écran, être échantillonné un peu plus largement qu'un voxel. Dans l'usage normal (volumes de quelques centaines de voxels), le plafond n'est jamais atteint.

9.10 Les vraies proportions : calibration et étirement en Z

Le cube n'est pas dessiné d'après le **nombre de voxels** mais d'après les **dimensions réelles en micromètres**. Les voxels sont souvent plus épais en Z qu'en X et Y ; sans correction, l'embryon paraîtrait écrasé.

Le cube a les vraies proportions de l'échantillon



▲ Le fichier compte des voxels ; l'écran montre des micromètres. Jeu de démonstration E9.5 : voxels de $1,2 \times 1,2 \times 3,0 \mu\text{m}$.

MICRO-EXEMPLE

Jeu E9.5 : $768 \times 576 \times 112$ voxels de $1,2 \times 1,2 \times 3,0 \mu\text{m}$.

- X : $768 \times 1,2 = 921,6 \mu\text{m}$; Y : $576 \times 1,2 = 691,2 \mu\text{m}$.
- Z : $(112 - 1) \times 3,0 + \text{épaisseur de coupe optique } 3,0 = 336 \mu\text{m}$.
- Le plus grand côté (X) vaut 1 ; le cube mesure donc $1 \times 0,75 \times 0,365$ ($336 / 921,6$).

Le viewer classe la calibration en trois états : **exacte** (tailles de voxel et épaisseur de coupe connues), **estimée**, ou **absente**. Sans calibration, la **barre d'échelle** 3D est cachée plutôt que fausse.

Dans le panneau **Échelle Physique**, le curseur **Surcharge Z** (de $\times 0,25$ à $\times 2,0$) étire seulement l'**affichage** en Z pour mieux voir les couches fines ; le bouton **1:1** le remet à zéro. Les mesures en μm ignorent cet étirement.

ATTENTION

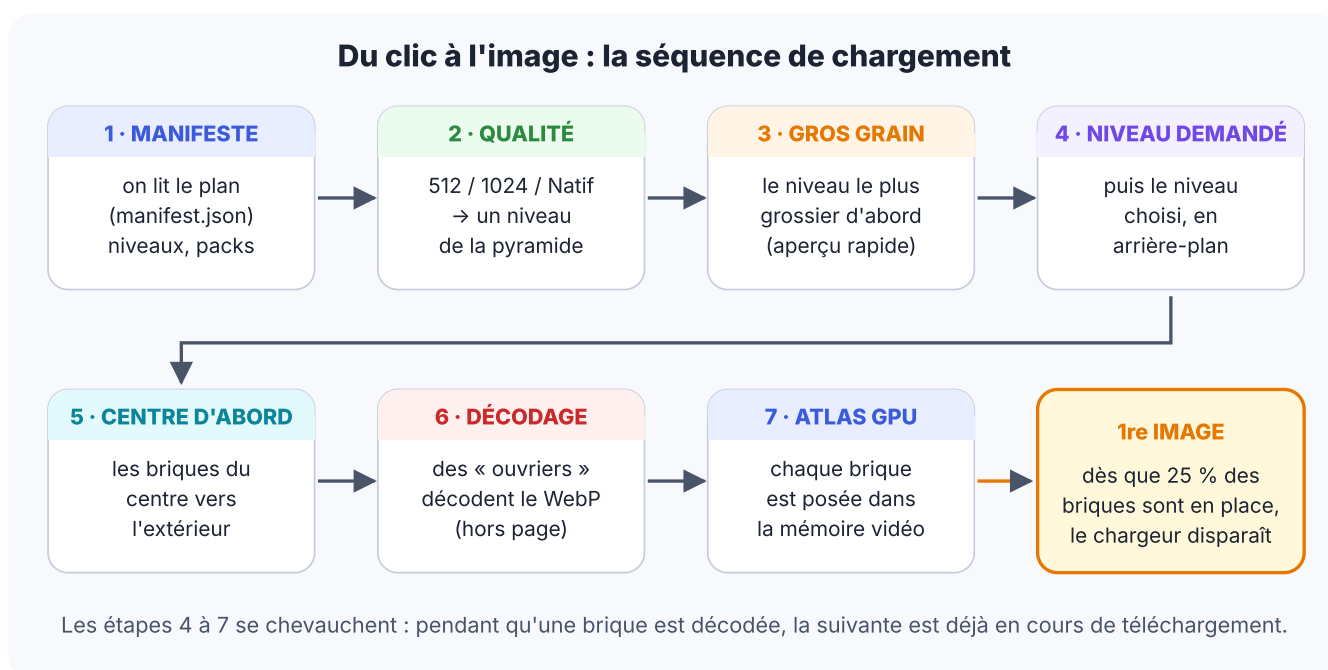
La barre d'échelle 3D est exacte pour ce qui se trouve **à la profondeur du centre de l'échantillon**. La vue est en perspective : ce qui est plus près paraît plus grand, ce qui est plus loin paraît plus petit.

10. Afficher des gigaoctets : le streaming

- Le navigateur ne charge **jamais** tout le volume : il télécharge des **briques** (petits cubes de 64^3 voxels) du centre vers l'extérieur et les range dans la mémoire de la carte graphique.
- Une première image apparaît dès que **25 %** des briques sont en place ; la qualité monte ensuite sans bloquer l'écran.
- Si la mémoire graphique est insuffisante, le viewer choisit tout seul un **niveau plus grossier** et vous le dit.

10.1 La séquence de chargement

Quand vous ouvrez un jeu de données, sept étapes s'enchaînent. Vous ne les voyez pas toutes, mais elles expliquent pourquoi l'image devient nette **progressivement**.



▲ Du clic à l'image : sept étapes qui se chevauchent.

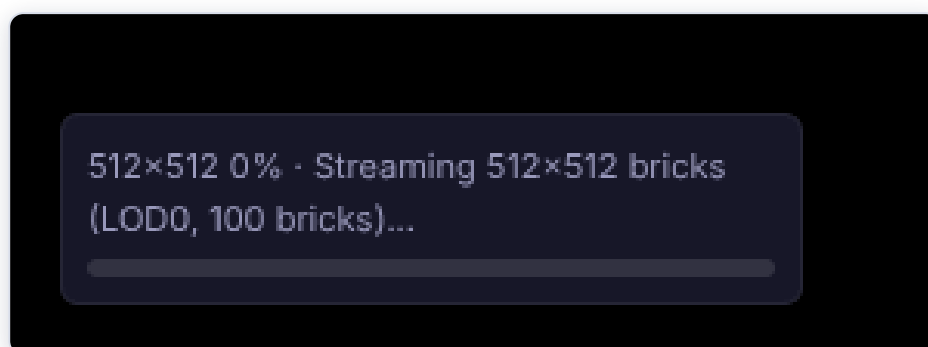
- 1 Manifeste.** Le viewer lit le « plan » du jeu de données (`bricks/manifest.json`) : niveaux de la pyramide, taille des briques, fichiers de paquets.
- 2 Qualité.** Le menu `Qualité de Rendu` se traduit en un niveau de la pyramide (section 10.2).
- 3 Gros grain d'abord.** À la première ouverture, le niveau **le plus grossier** est chargé en premier : quelques briques, une image en un clin d'œil.
- 4 Niveau demandé.** Le niveau choisi se charge ensuite en arrière-plan.

- 5 **Du centre vers l'extérieur.** Les briques sont classées par distance au centre, **mesurée en micromètres** (pour que la forte épaisseur des voxels en Z ne fausse pas le « centre »).
- 6 **Décodage.** Des « ouvriers » (workers) décodent chaque brique hors de la page (section 10.5).
- 7 **Atlas.** La brique décodée est copiée à sa place dans la mémoire de la carte graphique (section 10.3).

📌 À RETENIR

La **première image** s'affiche quand **25 %** des briques du niveau en cours sont sur la carte graphique ; la carte « chargement » disparaît alors. Les redessins dus aux briques qui arrivent sont groupés (au plus un toutes les 250 ms).

Où voir la progression



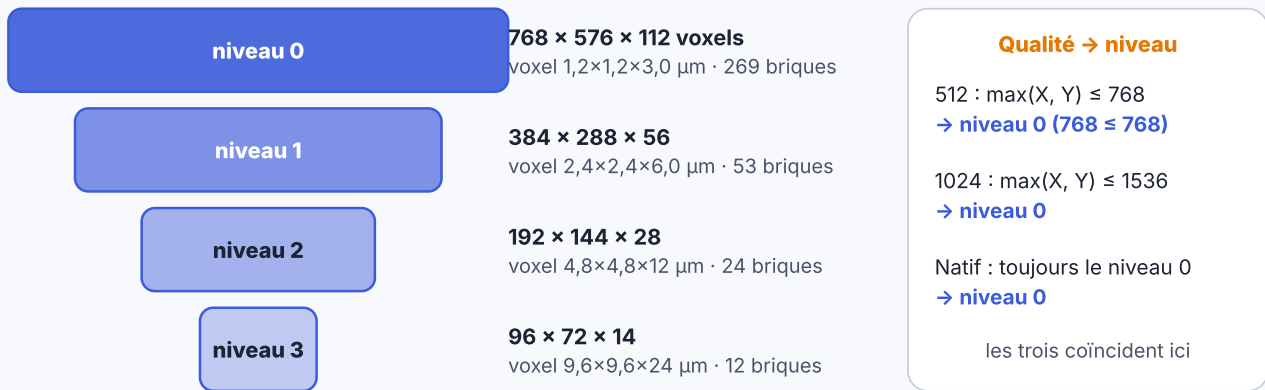
▲ La ligne de progression en bas à gauche du viewer : qualité, pourcentage de briques traitées et étape en cours (le texte de cette ligne est en anglais, même quand l'interface est en français).

Le pourcentage est la part des briques **déjà traitées** (déposées ou abandonnées), pas des octets téléchargés. La ligne disparaît quand le niveau est complet.

10.2 Le menu de qualité : 512, 1024, Natif

La pyramide de niveaux a été fabriquée par le pipeline (chapitre 6) : chaque niveau a **4 fois moins de voxels en $X \times Y$** que le précédent. Le menu choisit **le plus fin des niveaux qui reste raisonnable** pour le réglage demandé.

La pyramide de niveaux du jeu de démonstration E9.5



Chaque niveau a 4 fois moins de voxels en $X \times Y$ que le précédent (et la moitié en Z).

▲ La pyramide réelle du jeu de démonstration E9.5 et la règle des trois qualités.

RÉGLAGE	NIVEAU RETENU (FORMAT 4)
512 (défaut)	le niveau le plus fin dont le plus grand côté XY est $\leq$ 768 voxels
1024	le niveau le plus fin dont le plus grand côté XY est $\leq$ 1 536 voxels
Natif	toujours le niveau 0 (pleine résolution)

MICRO-EXEMPLE

Jeu **fictif** de $3\,072 \times 2\,304$ voxels en XY, niveaux à 3 072, 1 536, 768 et 384 :

- **512** → niveau à 768 (≤ 768) ; **1024** → niveau à 1 536 ($\leq 1\,536$) ; **Natif** → 3 072.

Sur le jeu de démonstration E9.5, le plus grand côté est 768 : les trois réglages tombent sur le niveau 0. Le menu **n'affiche alors qu'une seule option**, avec les vraies dimensions :

Natif (768×576×112) .



▲ Le panneau **Qualité de Rendu** : une seule ligne par niveau réellement distinct (ici une seule, avec ses dimensions), la ligne d'état (« 512×512 actif ») et, dessous, le sélecteur **Détail au zoom** (section 10.8).

i À SAVOIR

Le viewer s'ouvre en qualité **512**. Un changement de qualité qui échoue laisse le volume précédent à l'écran et remet le menu en place.

10.3 L'atlas GPU et la table des pages

Une brique de 64^3 voxels est trop petite pour mériter sa propre texture ; en créer des centaines serait ingérable. Le viewer réserve donc en mémoire vidéo **de grands blocs d'emplacements identiques** : l'**atlas**.

💡 UNE IMAGE POUR COMPRENDRE

L'atlas est un **entrepôt** dont toutes les étagères ont la même taille. La **table des pages** est la **fiche d'index** à l'entrée : « la brique n° 17 est sur l'étagère 4 ». Quand le rayon a besoin d'un voxel, il consulte la fiche, puis va directement à la bonne étagère. Si la fiche dit « vide », il ne se déplace même pas.

L'atlas et la table des pages : un entrepôt et sa fiche d'index

Table des pages (la fiche)

une case par brique du volume

p0 · 3	p0 · 4	vide	p0 · 7
p0 · 1	p1 · 0	p0 · 5	vide
vide	p0 · 2	vide	vide

« page · emplacement » ou « vide »
(vide : le rayon saute la brique)

Atlas (les étagères)

des emplacements identiques, en mémoire vidéo

0	1	2	3	4
5	libre	7	8	libre

Une brique = un emplacement de $66 \times 66 \times 66$ voxels (format 4)
1 à 4 canaux par voxel (1 octet chacun) · jusqu'à 8 « pages »
Plus d'emplacement libre ? On recycle le moins récemment utilisé.

▲ L'atlas et sa table des pages.

- Une case de la table par brique du volume : numéro de page + emplacement, ou « vide ».
- Jusqu'à 8 pages d'atlas, de 4, 8 ou 16 emplacements de côté.
- 1 à 4 canaux par voxel, un octet chacun : 1 canal = 1 octet par voxel, 2 canaux = 2, 3 ou 4 canaux = 4.
- Quand tous les emplacements sont occupés, on **recycle le moins récemment utilisé**.
- L'atlas est **dimensionné avant d'être alloué** : jamais de texture créée au hasard par brique.

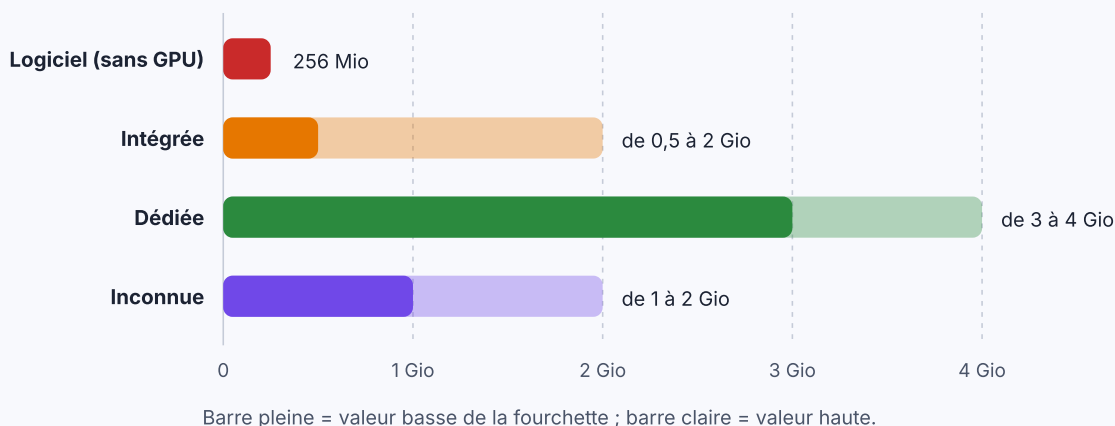
POUR LES CURIEUX

Un petit volume reste d'un seul tenant : si le niveau tient dans moins de **512 Mio** (en 4 octets par voxel) et dans les limites de la carte, il est stocké dans **une seule texture 3D** dense. Le jeu E9.5 au niveau 0 en est un exemple : $768 \times 576 \times 112 \times 4$ octets $\approx$ **189 Mio**. L'atlas à pages ne sert que pour les gros niveaux.

10.4 Le budget de mémoire vidéo

Aucun navigateur ne dit combien de mémoire vidéo il reste. Le viewer **l'estime** d'après le type de carte graphique et, si disponible, la mémoire de l'appareil.

Le budget de mémoire vidéo estimé par le viewer



▲ Le budget estimé selon le type de carte graphique (1 Gio = 1 024 Mio).

TYPE DE GPU	BUDGET ESTIMÉ
Logiciel (pas de vraie carte graphique)	256 Mio
Intégrée (Intel, Apple...)	de 0,5 à 2 Gio selon la mémoire de l'appareil
Dédiée (NVIDIA, Radeon...)	4 Gio (3 Gio si l'appareil déclare moins de 8 Go)
Inconnue	1 ou 2 Gio

Chaque **perte de contexte** graphique de la dernière semaine **divise le budget par deux** (plancher : 256 Mio). L'administrateur peut le fixer à la main si besoin.

Quand ça ne rentre pas

Avant d'allouer quoi que ce soit, le viewer calcule la taille du niveau demandé. Si elle dépasse le budget, il essaie le niveau **immédiatement plus grossier**, et ainsi de suite.

MICRO-EXEMPLE

Exemple **fictif** : vous demandez **Natif** et le niveau 0 réclame 1,9 Gio pour un budget de 1 Gio : le viewer passe au niveau 1 (environ 4 fois moins), qui rentre. Un avis s'affiche :

« La résolution {demandée} dépasse la mémoire GPU disponible — affichage en {obtenue}.
({nécessaire} Mo nécessaires, {disponible} Mo disponibles pour le volume.) »

Si la carte **refuse** une allocation malgré le calcul, le viewer retombe aussi sur un niveau plus grossier ; l'avis le précise : « (Le GPU a refusé l'allocation.) ». Le volume reste **toujours visible**, jamais un onglet planté.

10.5 Télécharger et décoder sans bloquer la page

Décoder une brique (une image WebP à défaire en voxels) est un calcul lourd. Il est confié à un **groupe d'ouvriers** (workers) qui travaillent **en parallèle de la page** : l'interface reste fluide.

- **Nombre d'ouvriers** : `min(8, nombre de cœurs - 1)` , au moins 1.

- **Décodage fidèle** : aucune gestion des couleurs du navigateur n'est appliquée ; les octets sont des **intensités mesurées**, pas des couleurs.
- **Mosaïque** : une brique de 66^3 voxels est une image WebP de 594×528 pixels (9×8 tuiles de 66×66) ; une image d'une autre taille est refusée comme corrompue.
- **Pas de copie inutile** : les octets décodés sont transférés tels quels à la page, puis envoyés à la carte graphique.
- Un ouvrier qui plante ou se fige (30 s) est **remplacé** ; la brique est retentée jusqu'à 3 fois, puis comptée comme manquante.

⚠ ATTENTION

Une brique corrompue n'est **jamais** affichée : elle reste vide et le viewer le signale (par exemple : « N briques n'ont pas pu être chargées »). Un flux incomplet n'est pas gardé en mémoire : rechargez pour réessayer.

10.6 Les paquets : un téléchargement ordonné

Les briques sont rangées dans des **paquets** (fichiers `.bin`). Télécharger un paquet entier est bien plus efficace que des milliers de petites requêtes.

- **Ordre « paquet par paquet »** : le viewer part de la brique centrale et progresse paquet après paquet ; chaque paquet est téléchargé **une seule fois** et libéré dès que ses briques sont découpées.
- **4 téléchargements de paquets au plus en même temps**, car un navigateur ouvre 6 connexions par serveur : les appels de la page ne doivent pas attendre derrière les gros fichiers.
- **Coupe précise** : pour une coupe ou une image du Studio, on demande seulement les **plages d'octets** utiles.

CACHE (OCTETS COMPRESSÉS)	TAILLE
Paquets entiers, tant qu'un chargement en a besoin	192 Mio
Plages d'octets (coupes)	64 Mio
Préchargement (images voisines d'une série)	32 Mio

🔧 POUR LES CURIEUX

Les briques **décodées** ne sont jamais gardées côté page : l'atlas GPU est leur seul domicile. Chaque adresse de paquet porte un cachet `?v=` (une empreinte des paquets du manifeste) ; le serveur répond alors « à garder un an » : si le jeu de données change, l'empreinte change et le navigateur retélécharge.

10.7 Les séries temporelles

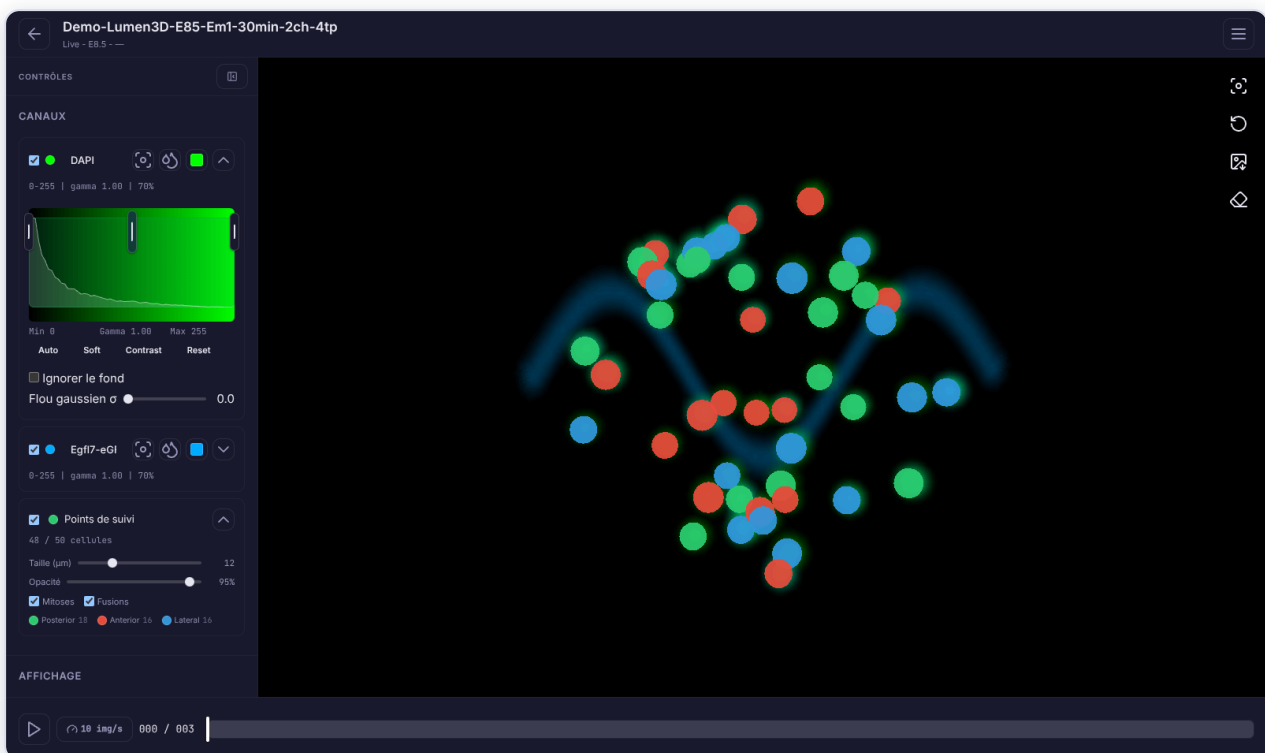
Un jeu **live** (série temporelle) contient **un arbre de briques par pas de temps**. Chaque pas de temps est chargé comme un petit volume à part entière.

Une série temporelle : un arbre de briques par pas de temps



Les images intermédiaires sont abandonnées plutôt que mises en file : la lecture ne prend jamais de retard sur le curseur.

▲ Chaque pas de temps a son arbre ; le tampon montre les images déjà en mémoire.



▲ La frise du jeu de démonstration Demo-Lumen3D-E85-Em1-30min-2ch-4tp : lecture, vitesse, curseur de temps et barre de tampon.

- **Lecture** : vitesses **0,5 – 1 – 2 – 5 – 10 – 20 images par seconde** (10 par défaut, mémorisée dans votre navigateur).
- **Une demande à la fois** : si vous déplacez le curseur vite, seule la **dernière** demande est servie.
- **La lecture attend l'image** : l'horloge est suspendue pendant qu'une image charge ; la lecture va à la vitesse réelle d'arrivée.
- **Tampon** : le viewer préserve en mémoire les images voisines (jusqu'à `min(768 Mio, max(128 Mio, 40 % du budget))`). La barre de la frise montre celles **réellement présentes** à la qualité courante. Un texte précise si la série ne tient pas entièrement : « Tampon n/total · qualité — la série entière ne tient pas en mémoire (... images max) ».
- **Préchargement** : pendant la lecture, les paquets des images suivantes (t+1, t+2...) sont préchargés en temps libre.

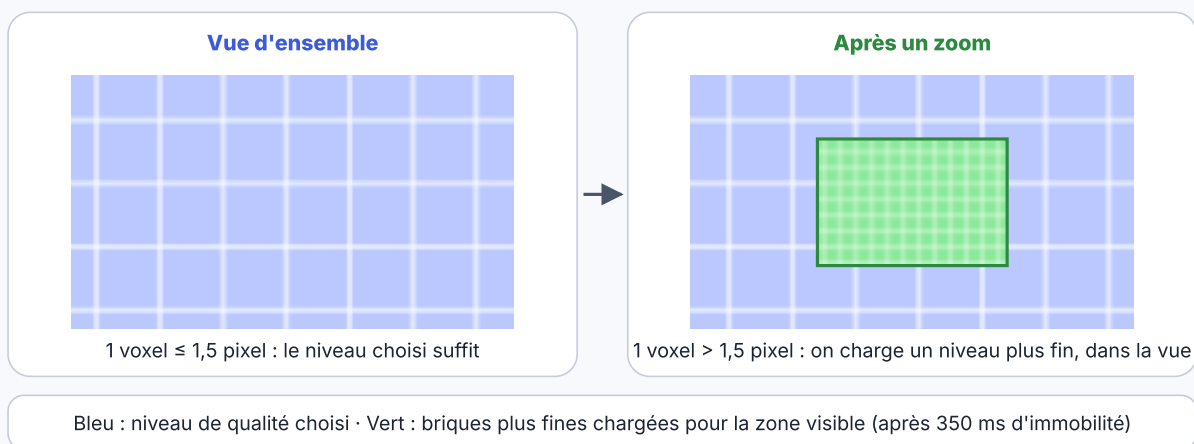
i À SAVOIR

Le préchargement complet n'est possible que pour les niveaux stockés en **texture dense** : un niveau en atlas à pages n'est pas préchargé.

10.8 Le mode « Détail au zoom »

Vous zoomez fort sur une zone : les voxels du niveau choisi deviennent plus gros que l'écran n'en a besoin, l'image paraît floue. Le mode **Détail au zoom** charge alors **des briques plus fines, seulement dans la zone visible**.

Mode « Détail au zoom » : du fin seulement là où vous regardez



▲ Le niveau choisi partout (bleu), et des briques plus fines uniquement dans la vue (vert).

OPTION	EFFET
Automatique	actif si le budget mémoire dépasse 1 Gio
Activé	actif même sous 1 Gio
Désactivé	jamais

- **Déclencheur** : un voxel du niveau courant couvre **plus de 1,5 pixel d'écran**.
- Le niveau retenu est le **plus grossier** des niveaux plus fins qui descend sous 1,5 pixel par voxel (le moins d'octets possible).
- Le chargement démarre **350 ms** après que la caméra s'est arrêtée, 8 briques à la fois, **les plus grosses et les plus centrales d'abord**. Il utilise au plus 768 Mio et 75 % de la mémoire restante.
- À la frontière de la zone fine, l'image passe d'un niveau à l'autre **à une face de brique**, sans fondu.

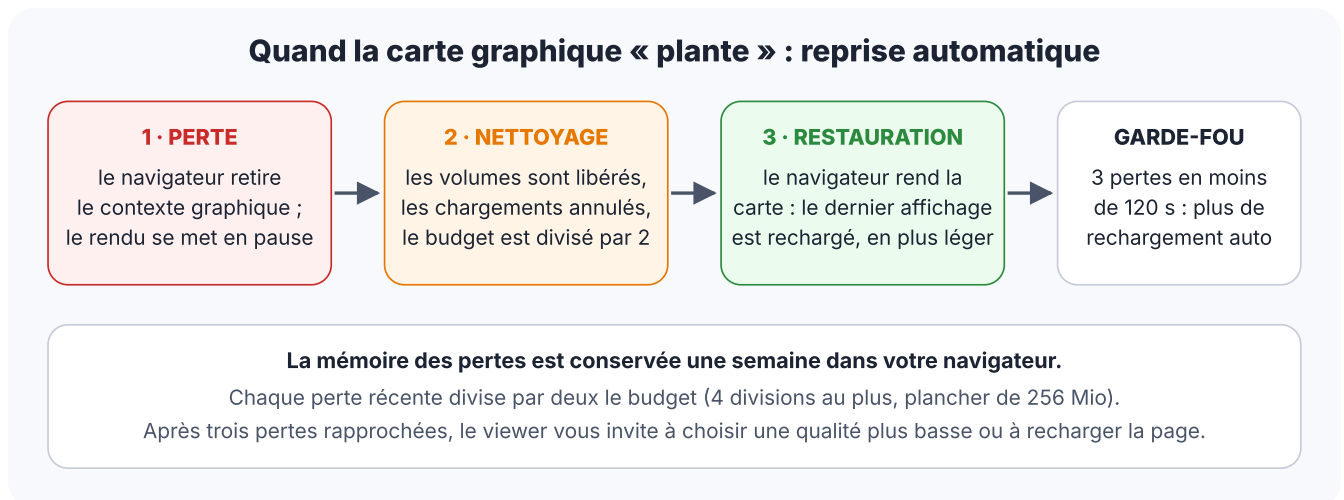
⚠ ATTENTION

Le mode n'existe que pour le **format 4** (briques à bordure). Il est suspendu pendant qu'un volume se charge et dès que vous appliquez un **flou gaussien** à un canal (chapitre 11), car des briques brutes sur un canal flou donneraient une autre image.

Le texte d'état affiche, selon le cas : « Détail : N briques du niveau L visibles », « Détail : chargement de x sur y briques du niveau L », ou « Détail indisponible : mémoire GPU insuffisante ».

10.9 Quand la carte graphique plante

Un navigateur peut retirer sa carte graphique à une page (pilote en difficulté, trop de mémoire demandée). Le viewer sait **repartir seul**.



▲ La reprise après une perte de contexte graphique.

Vous voyez d'abord « Contexte GPU perdu : le rendu est en pause jusqu'à ce que le navigateur le rende. », puis le volume se recharge en plus léger. Après **trois pertes en moins de 120 secondes**, il s'arrête et vous invite à choisir une qualité plus basse ou à recharger la page.

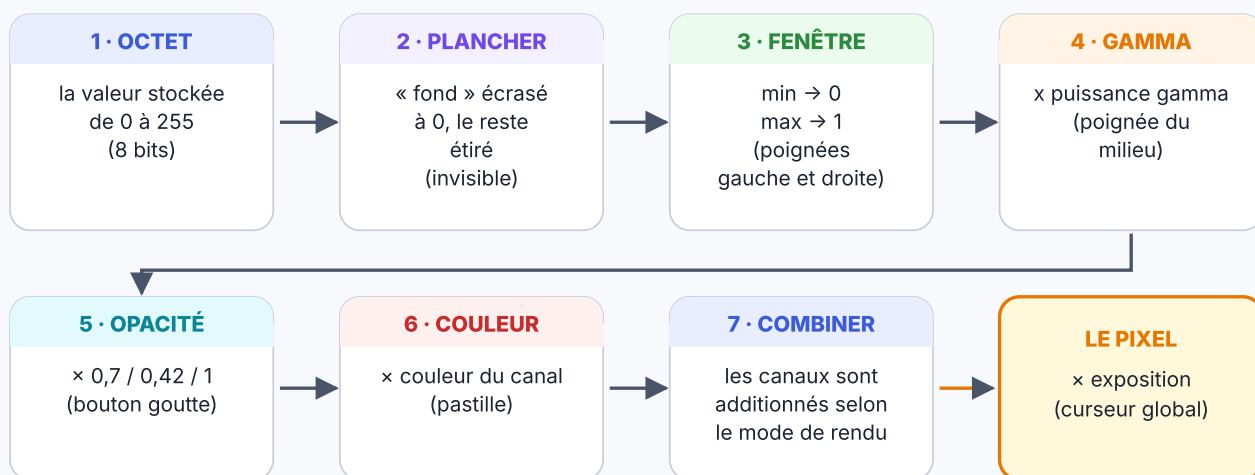
11. Couleurs, contrastes et histogrammes

- Chaque canal suit la même chaîne de réglages : **plancher invisible** → **fenêtre min / max** → **gamma** → **opacité** → **couleur**, puis les canaux sont combinés.
- L'**histogramme** montre comment les 256 niveaux de gris sont répartis ; ses trois poignées règlent la fenêtre et le gamma.
- Les niveaux de gris sont **relatifs** à un jeu de données : ce ne sont pas des intensités de fluorescence calibrées.

11.1 La chaîne de traitement d'un canal

Chaque voxel est stocké sous la forme d'un **octet** (un nombre de 0 à 255). Avant de devenir un pixel coloré, il traverse sept étapes. Cinq d'entre elles sont à votre portée ; une seule est cachée.

Le trajet d'une valeur, de l'octet à la couleur du pixel



Les étapes 3 à 5 se règlent canal par canal. L'étape 2 n'a aucun réglage : elle est décidée à partir de l'histogramme du jeu de données, au moment du décodage des briques.

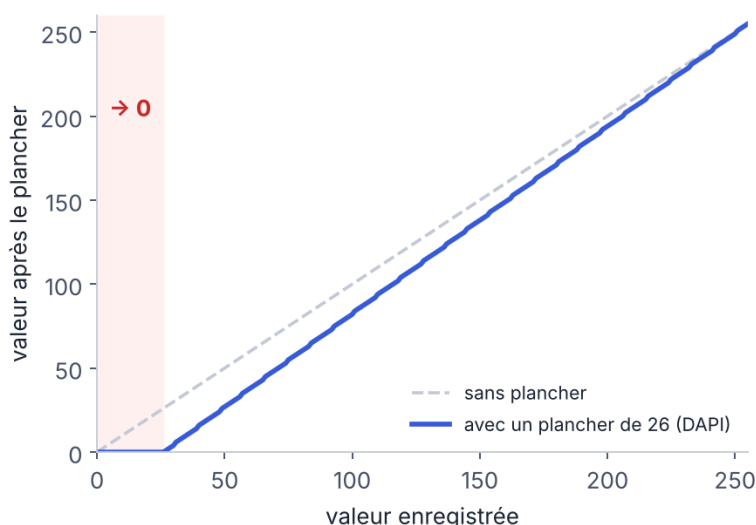
▲ Le trajet d'une valeur : de l'octet stocké à la couleur du pixel.

ÉTAPE	QUI DÉCIDE ?	OÙ LA RÉGLER
Plancher	automatique (histogramme)	nulle part
Fenêtre min / max	vous	poignées gauche et droite
Gamma	vous	poignée du milieu
Opacité	vous	bouton goutte (3 valeurs)
Couleur	vous	pastille de couleur

ÉTAPE	QUI DÉCIDE ?	OÙ LA RÉGLER
Exposition	vous	curseur Visibilité (exposition) , global

11.2 L'étape cachée : le plancher du fond

Avant même que vous touchiez à un réglage, le viewer **écrase le fond** : toutes les valeurs inférieures ou égales à un seuil, le **plancher**, deviennent **0**, et les valeurs restantes sont **étirées** pour occuper toute l'échelle 0–255.



▲ L'effet d'un plancher de 26 (valeur réelle estimée pour le canal DAPI du jeu E9.5).

POUR LES CURIEUX

Le plancher f est estimé à partir de l'histogramme du jeu de données : on cherche le premier niveau où le cumul atteint la case **0 + 20 % des voxels non nuls**, on prend le bord supérieur de cette case, on ajoute 2 et on borne le résultat entre **6 et 48**. Ensuite : $\text{valeur}' = 0$ si $\text{valeur} \leq f$, sinon $\text{arrondi}((\text{valeur} - f) \times 255 / (255 - f))$.

MICRO-EXEMPLE

Plancher de 26 (DAPI) : un voxel stocké à **120** devient $(120 - 26) \times 255 / 229 \approx \mathbf{105}$; un voxel à 20 devient **0**. Pour le jeu E9.5 les planchers estimés sont 26 (DAPI), 10 (Pecam1) et 22 (Sox2).

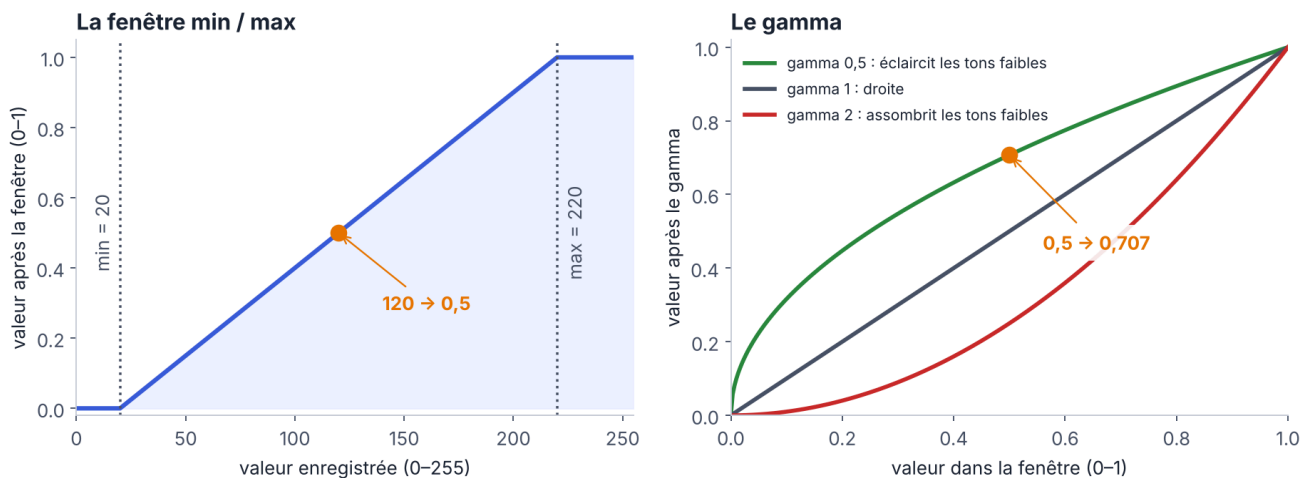
À SAVOIR

Les valeurs que vous lisez sur les poignées (Min 0, Max 255...) sont déjà **après** ce plancher. Vous ne le voyez jamais, mais il explique pourquoi le fond est si propre.

11.3 La fenêtre min / max et le gamma

La **fenêtre** choisit quelle portion de l'échelle devient « du noir au plein éclat ». Les valeurs sous **Min** deviennent noires, celles au-dessus de **Max** saturent, entre les deux le passage est **linéaire**.

Le **gamma** courbe ensuite cette échelle : un gamma inférieur à 1 **éclaircit** les tons faibles (utile pour des signaux ténus), un gamma supérieur à 1 les **assombrit**.



▲ À gauche, la fenêtre min 20 / max 220 ; à droite, l'effet du gamma 0,5 / 1 / 2.

1 2 3 4 MICRO-EXEMPLE

Un voxel de valeur **120**, avec Min = 20, Max = 220, gamma = 0,5 et opacité = 70 % :

1. Fenêtre : $(120 - 20) / (220 - 20) = \mathbf{0,5}$
2. Gamma : 0,5 puissance 0,5 = **0,707**
3. Opacité : $0,707 \times 0,7 = \mathbf{0,495}$
4. Couleur : sur un canal vert (0 ; 1 ; 0), le voxel contribue (0 ; **0,495** ; 0), avant l'exposition.

La poignée du milieu

La poignée du milieu ne règle pas le gamma directement : elle désigne le niveau (**dans la fenêtre**) qui doit s'afficher à **50 %**. Le viewer en déduit le gamma, borné entre **0,18 et 5,5** :

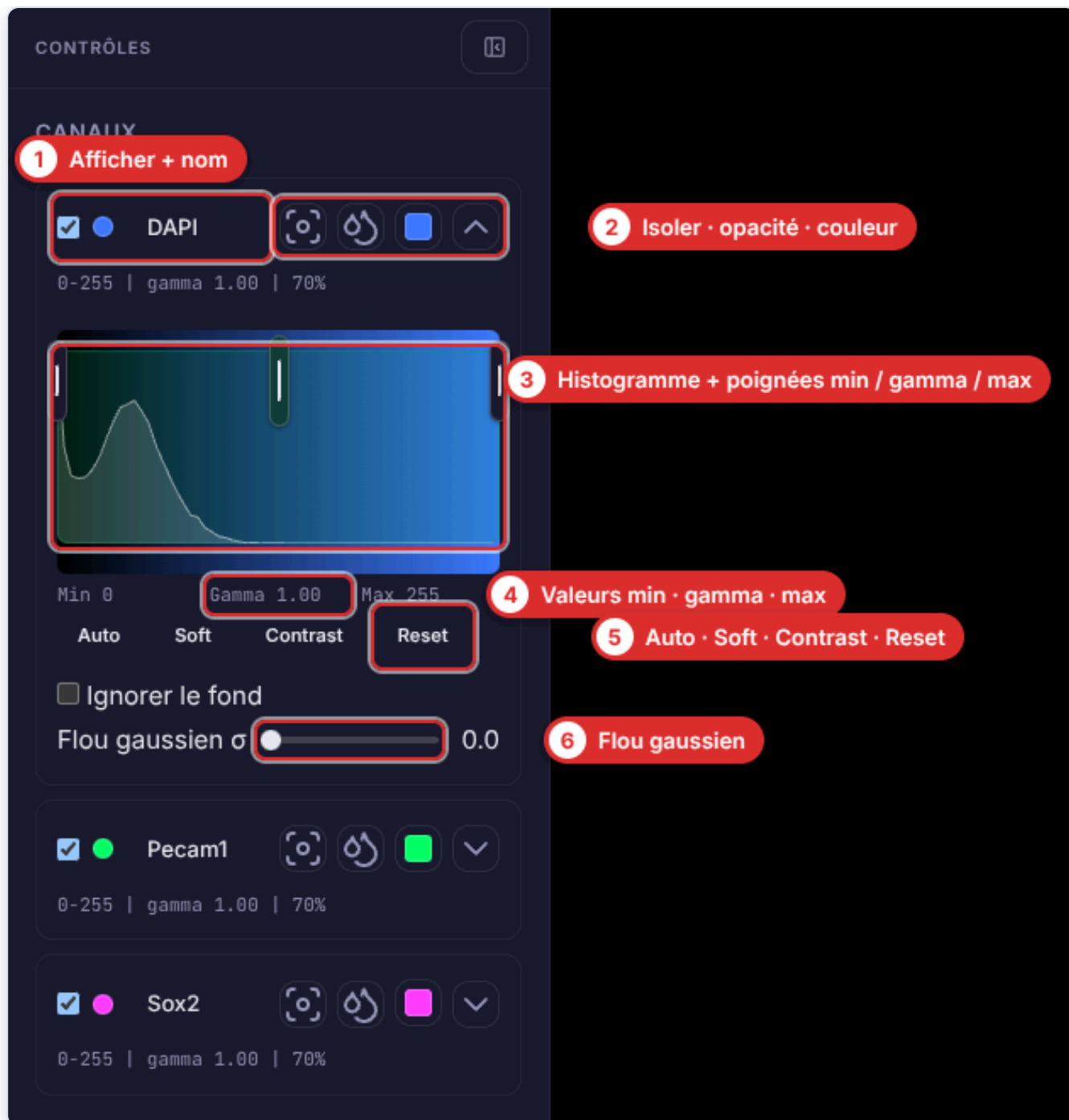
$\text{gamma} = \ln 0,5 / \ln m$, où m est la position relative de la poignée entre Min et Max.

1 2 3 4 MICRO-EXEMPLE

Poignée au quart de la fenêtre ($m = 0,25$) : $\text{gamma} = \ln 0,5 / \ln 0,25 = \mathbf{0,5}$. Avec Min = 20 et Max = 220, ce quart correspond à la valeur $20 + 0,25 \times 200 = \mathbf{70}$: une valeur de 70 s'affiche à 50 %.
Poignée au milieu ($m = 0,5$) : gamma = 1.

11.4 Le panneau d'un canal

Chaque canal a sa carte dans le panneau **Canaux**. La ligne de résumé (par exemple 0-255 | gamma 1.00 | 70%) rappelle la fenêtre, le gamma et l'opacité même quand la carte est repliée.



▲ La carte d'un canal dépliée (DAPI, jeu de démonstration E9.5).

(numéros : voir la légende ci-dessous.)

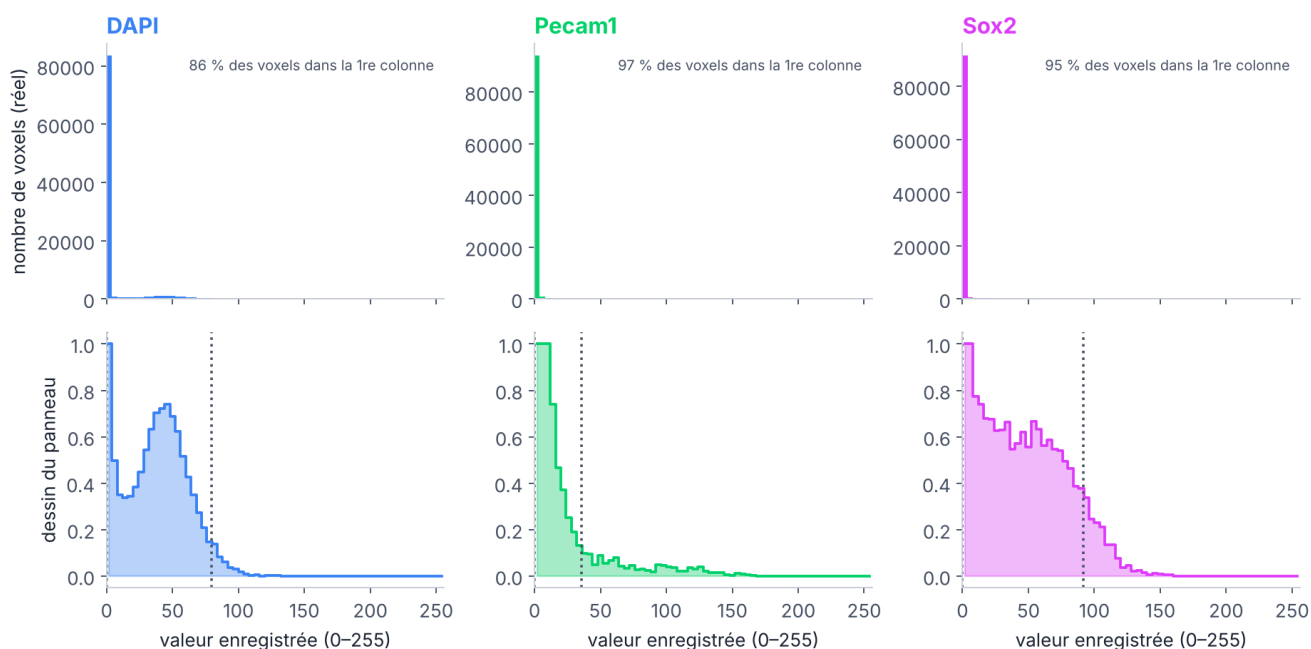
N	CE QUE C'EST
1	case à cocher (afficher / masquer le canal) et nom du canal, modifiable
2	boutons : Isoler le canal (Solo), opacité (goutte), couleur (pastille)
3	histogramme, avec les trois poignées : Min (gauche), gamma (milieu), Max (droite)
4	valeurs lues : Min, Gamma, Max

N	CE QUE C'EST
5	boutons de réglage rapide : Auto, Soft, Contrast, Reset (voir ci-dessous)
6	curseur Flou gaussien σ (section 11.7)

11.5 Les histogrammes

Un **histogramme** compte combien de voxels ont chaque valeur. Celui du panneau a **64 colonnes** (4 niveaux de gris par colonne).

- Il est calculé **par le pipeline**, sur le **niveau le plus grossier** de la pyramide, et enregistré dans le manifeste. Le navigateur ne le recalcule pas.
- Il porte sur **tous** les voxels, fond compris : c'est pourquoi la première colonne est énorme (86 à 97 % des voxels du jeu E9.5).
- Dans une **série temporelle**, chaque pas de temps a son propre histogramme ; le panneau le suit quand vous changez d'image.
- Il représente les **octets stockés** (avant le plancher et la fenêtre).



▲ Les histogrammes réels des trois canaux du jeu de démonstration E9.5. En haut : les comptes bruts. En bas : ce que dessine le panneau. Les traits pointillés marquent la plage choisie par **Auto**.

Comment le panneau dessine l'histogramme

Une colonne géante au niveau 0 écraserait tout le reste. Le panneau applique donc trois règles :

- échelle **linéaire**, normalisée par la colonne la plus haute ;
- les **4 % de colonnes les plus basses** (les 3 premières sur 64) sont **plafonnées à 1,35 fois** la plus haute colonne qui les suit ;
- les « trous en peigne » (une colonne sous 20 % de ses deux voisines) sont comblés par la moyenne de ses voisines, pour masquer un artefact de décodage du WebP.

À SAVOIR

La case **Ignorer le fond** ne change que le **dessin** : elle multiplie les colonnes affichées par $(i / 63)^2$ pour aplatir la bosse du fond. Elle **ne modifie pas l'image**.

11.6 Les boutons de réglage rapide

Les boutons sous l'histogramme posent d'un coup Min, Max et le milieu. Ils s'appellent **Auto**, **Soft**, **Contrast** et **Reset** dans le panneau (texte anglais affiché tel quel, même en interface française).

BOUTON	MIN	MAX	GAMMA OBTENU
Auto	percentile 0,5 %	percentile 99,5 %	1 (milieu de la fenêtre)
Soft	5	240	0,83
Contrast	20	209	1,22
Reset	0	255	1

POUR LES CURIEUX

Auto parcourt l'histogramme (64 colonnes) : Min = dernière colonne dont le cumul reste $\leq 0,5$ % des voxels, Max = fin de la dernière colonne dont le cumul reste $\leq 99,5$ %. Sans histogramme : Min 0,01 et Max 0,98 (sur 1). Les presets sont exprimés sur l'échelle 0–1 : Soft = 0,02 / 0,94 avec milieu à 0,42 ; Contrast = 0,08 / 0,82 avec milieu à 0,5 ; Reset = 0 / 1 / 0,5.

MICRO-EXEMPLE

Auto sur le jeu E9.5 donne Max = 80 pour DAPI, 36 pour Pecam1 et 92 pour Sox2, et Min = 0 : comme 86 à 97 % des voxels sont du fond, le 0,5 % inférieur est entièrement dans la première colonne. **Reset** remet 0 / 255.

ATTENTION

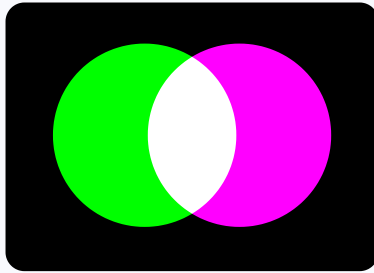
Auto reflète la **distribution de tous les voxels**, fond compris. Sur un signal rare, il peut placer Max très bas : l'image sature alors volontairement. Gardez un œil sur l'histogramme et ajustez les poignées à la main si besoin.

11.7 Couleurs, opacité, exposition, solo

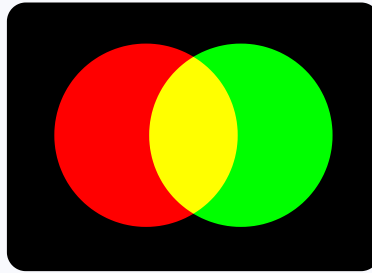
Couleur : une addition de lumières

La couleur d'un canal est un simple **multiplicateur RVB**. Les canaux sont **additionnés**, comme des projecteurs : là où deux canaux sont forts, les couleurs se mélangent, puis tout dépassement de 1 est coupé à l'écran.

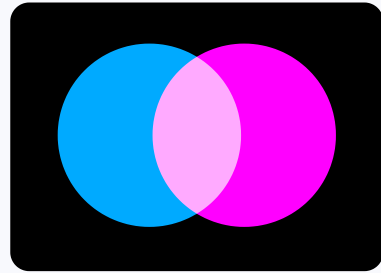
Les couleurs des canaux s'additionnent (comme des projecteurs de lumière)



vert + magenta = blanc
là où les deux sont forts



rouge + vert = jaune
le recouvrement devient jaune



bleu clair + magenta
→ un lilas très clair

▲ Mélanges additifs : vert + magenta donne du blanc.

- Le sélecteur propose **27 couleurs** : trois lignes (vives, claires, foncées) de neuf teintes.
- Par défaut (sans couleur dans les métadonnées) : vert, bleu clair, magenta, rouge.
- Le menu des **simulations de déficiences de la vision des couleurs** de l'en-tête applique un filtre à l'écran entier : il sert à vérifier qu'une figure reste lisible, il ne change pas les données.

👉 ASTUCE

Évitez le rouge et le vert côte à côte pour des figures destinées à un public large : choisissez plutôt vert et magenta, qui restent distincts pour la plupart des déficiences.

Opacité, exposition, solo

- **Opacité** : le bouton goutte fait défiler **70 %**, **42 %**, **100 %**. C'est un multiplicateur d'intensité (0,7 ; 0,42 ; 1), pas une absorption physique.
- **Exposition** : le curseur **Visibilité (exposition)** (de 20 à 500 %, soit $\times 0,2$ à $\times 5$) multiplie la couleur finale ; dans le mode naturel, il multiplie l'émission.
- **Solo** : **Isoler le canal** n'affiche que ce canal ; un deuxième clic (**Afficher tous les canaux**) restaure l'état précédent.
- Un jeu à **plus de 4 canaux** n'en montre que **4** (limite de la texture graphique).

11.8 Le flou gaussien

Le curseur **Flou gaussien σ** (de 0 à 5, par pas de 0,1) lisse le bruit **d'un canal**. L'effet est appliqué **au relâchement** du curseur ; ramener σ à 0 restaure le canal d'origine. Le réglage ne modifie que l'affichage.

Le flou gaussien : plan par plan, jamais entre les plans



Deux méthodes, même résultat

$\sigma < 1,8$: noyau gaussien exact
rayon = 3σ voxels (au plus 64)

$\sigma \geq 1,8$: 3 passes « boîte » + 1 passe exacte
rapide, et la variance totale vaut exactement σ^2

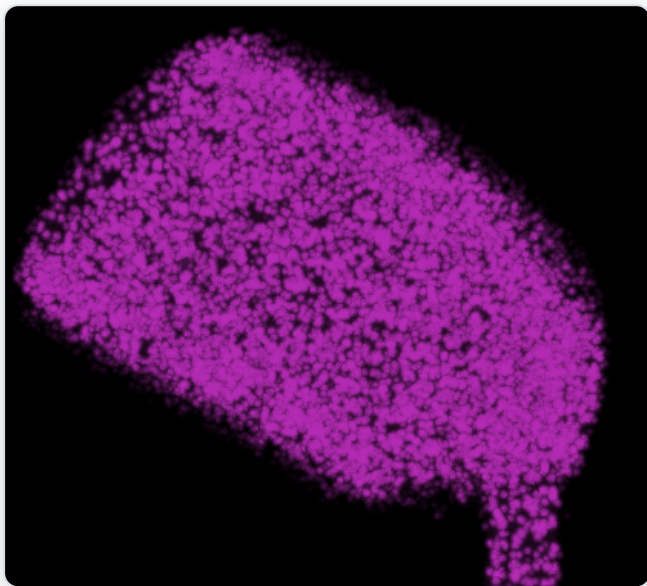
Pourquoi pas de flou en Z ?

L'écart entre deux plans est souvent plusieurs fois plus grand que la taille d'un voxel dans le plan (ici $3,0 \mu\text{m}$ contre $1,2 \mu\text{m}$). Un même σ en voxels ne serait donc pas le même en micromètres.
 σ est exprimé en voxels du niveau affiché : le même σ floute une plus grande surface réelle à un niveau grossier.

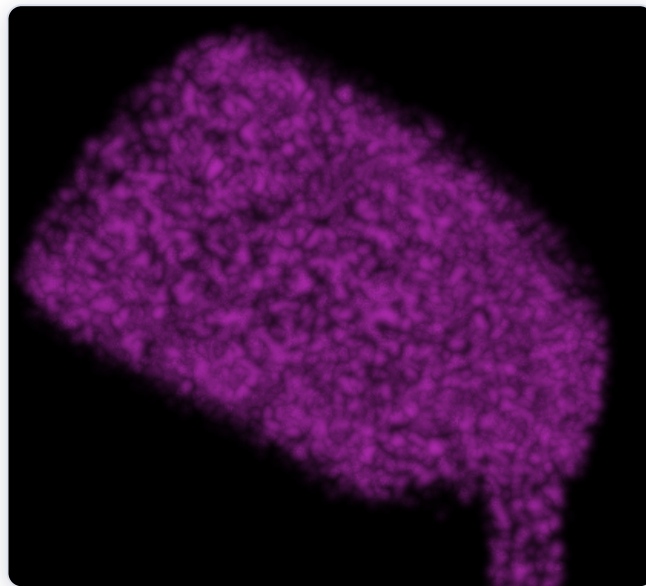
▲ Plan par plan, avec plusieurs ouvriers ; deux méthodes de calcul selon σ .

📌 À RETENIR

- **2D dans le plan** : chaque plan Z est flouté séparément, jamais d'un plan à l'autre.
- σ est en **voxels du niveau affiché**.
- Calcul par un groupe de **jusqu'à 4 ouvriers**, hors de la page ; une bulle indique que le calcul est en cours.



▲ Canal Sox2 seul, $\sigma = 0$.



▲ Même zone, $\sigma = 2,0$.

⚠ ATTENTION

Le flou n'est disponible que si le niveau affiché tient dans **une seule texture dense**. Pour un gros niveau stocké en atlas, le curseur est grisé avec le message : « Indisponible à cette qualité (atlas de briques creux) : choisissez une qualité inférieure pour utiliser le flou. » Utiliser le flou désactive aussi le mode [Détail au zoom](#) (chapitre 10).

11.9 Garde-fou : ce que les niveaux de gris veulent dire

⚠ ATTENTION

Les niveaux de gris affichés sont relatifs, pas calibrés. Le pipeline a ramené les intensités d'origine (12 ou 16 bits) sur 0–255, entre un fond estimé (99e percentile des coins du volume) et un signal maximal (99,9e percentile) ; le viewer y ajoute son plancher. Une valeur de 120 ne correspond donc à **aucun nombre de photons**.

- Comparer **deux canaux** d'un même jeu est délicat : chacun est étiré sur sa propre échelle.
- Comparer **deux jeux de données** par leurs niveaux de gris n'est pas valable, même avec les mêmes réglages.
- Dans une **série temporelle**, la même fenêtre est partagée par tous les pas de temps d'un canal : les variations d'intensité dans le temps restent comparables entre elles.
- Pour une **mesure quantitative**, retournez aux données d'origine (fichier Imaris).

13. Le panneau d'administration en bref

- Le panneau d'administration est la **salle des machines** de Lumen3D : c'est là que l'on met des données en ligne, que l'on règle l'apparence du site et que l'on met la plateforme à jour.
- Il compte **15 onglets en 4 groupes**. Vous n'en utiliserez que quelques-uns au quotidien.
- Ce chapitre est un **survol**. Le mode d'emploi pas à pas est le « **Guide de l'administrateur** », publié dans l'onglet **Documentation** du panneau lui-même.

13.1 À quoi sert le panneau, et comment y accéder

Le site public est la vitrine : les visiteurs regardent des embryons. Le panneau d'administration est l'**arrière-boutique**. Il sert à :

- **mettre des datasets en ligne** et les décrire (nom, stade, couleurs, orientation) ;
- **choisir ce que le public voit** (visible ou masqué) ;
- **personnaliser le site** : nom, couleurs, pages, mentions légales ;
- **installer des plugins** et **mettre la plateforme à jour**.

i À SAVOIR

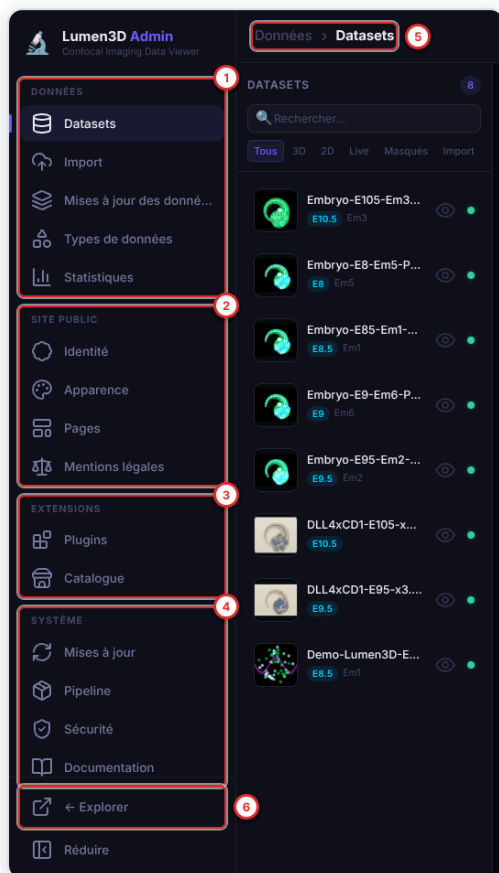
Le panneau n'a **aucun lien public**. On l'ouvre en tapant l'adresse `admpan.html` à la suite de celle du site (par exemple `https://votre-site/admpan.html`). Il demande un identifiant et un mot de passe. Le mot de passe est créé lors de la toute première visite, grâce à un assistant en 5 étapes (compte, identité, thème, textes, plugins).

13.2 Les 15 onglets, en 4 groupes

La barre latérale de gauche range les onglets en quatre groupes. Le fil d'Ariane en haut rappelle où vous êtes (par exemple **Données > Datasets**).

ONGLET	CE QU'IL FAIT	FRÉQUENCE
Données		
Datasets	Liste, aperçu, réglages	souvent
Import	Envoyer un dossier du pipeline	à chaque nouveau dataset
Mises à jour des données	Remettre au format 4	rarement
Types de données	Renommer 3D, 2D, Live	une fois
Statistiques	Visites, vues, téléchargements	à l'occasion
Site public		

ONGLET	CE QU'IL FAIT	FRÉQUENCE
Identité	Nom, vocabulaire, SEO, menu	à l'installation
Apparence	Couleurs, police, arrondis	à l'installation
Pages	Éditeur visuel de pages	parfois
Mentions légales	Textes légaux, par langue	une fois
Extensions		
Plugins	Activer, approuver	parfois
Catalogue	Installer des plugins signés	parfois
Système		
Mises à jour	Mettre la plateforme à jour	régulièrement
Pipeline	Télécharger le pack (ch. 4 à 8)	à chaque nouveau pack
Sécurité	Mot de passe, permissions	rarement
Documentation	Lire les guides publiés	à la demande



- **1** Groupe **Données** : les jeux de données
- **2** Groupe **Site public** : apparence et textes vus par les visiteurs
- **3** Groupe **Extensions** : les plugins
- **4** Groupe **Système** : mises à jour, sécurité, documentation
- **5** Fil d'Ariane : groupe puis onglet courant
- **6** Lien vers le site public (nouvel onglet)

En haut à droite : thème clair/sombre du panneau, langue (français, anglais, espagnol, néerlandais) et déconnexion.

▲ Barre latérale du panneau (jeu de démonstration).

📌 À RETENIR

Deux onglets commencent par « Mises à jour » : **Mises à jour des données** (groupe Données : remettre **vos datasets** au format actuel) et **Mises à jour** (groupe Système : mettre à jour **la plateforme**). Ne les confondez pas.

👉 ASTUCE

Une pastille orange « Modifications non sauvegardées » s'allume en haut dès qu'un onglet contient des changements pas encore enregistrés. **Ctrl + S** enregistre l'onglet visible (Datasets, Types de données, Identité, Apparence, Mentions légales).

13.4 Mettre un dataset en ligne

Le pipeline (chapitres 4 à 8) produit **un dossier**. Il y a deux façons de le déposer sur le serveur.

Mettre un dataset en ligne : deux chemins

VOIE A · Copier à la main (FTP / SFTP)



VOIE B · Onglet Import (depuis le navigateur, sans FTP)



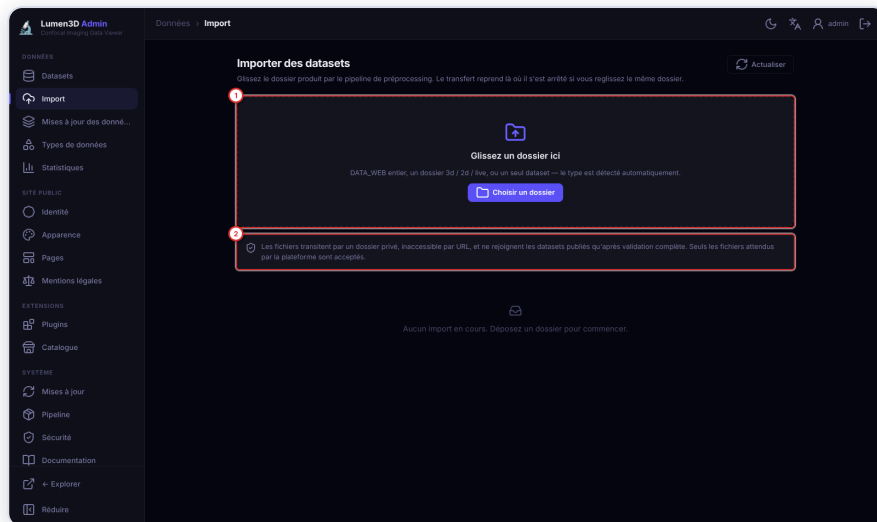
Pendant tout le transfert, les fichiers restent dans une zone privée (uploads/staging/), jamais accessible par URL. Un dataset publié par l'Import est masqué par défaut.

▲ Deux chemins pour mettre un dataset en ligne.

1 2 3 4 MICRO-EXEMPLE

Voie A : vous copiez le dossier dans `DATA_WEB/3d/` par FTP. Le dataset apparaît tout de suite : le catalogue est recalculé à chaque requête à partir des fichiers `metadata.json`, il n'y a rien à régénérer.

L'onglet Import, pas à pas



▲ L'onglet Import, avec un envoi en cours (jeu de démonstration).

1 Zone de dépôt : glissez le dossier ou cliquez sur **Choisir un dossier** · 2 Rappel : transit par un dossier privé, seuls les fichiers attendus sont acceptés · 3 Progression globale : pourcentage, volume transféré, vitesse, temps restant · 4 État du dataset (voir ci-dessous) · 5 Boutons de la carte : **Éditer**, **Vérifier**, **Publier**, **Supprimer** selon l'état

Les états d'un dataset en cours d'import :

ÉTAT AFFICHÉ	CE QUE CELA VEUT DIRE
Envoi — non éditable	les fichiers indispensables à l'ouverture ne sont pas tous là
Envoi — éditable	ouvrable en basse résolution : renommez, réglez les canaux pendant que le reste arrive
Envoyé — à publier	transfert complet et vérifié ; il reste à publier
Interrompu	reglissez le même dossier pour reprendre ; sans reprise, purge au bout de 7 jours

💡 UNE IMAGE POUR COMPRENDRE

Un déménagement par camion-navette. Le dossier part en petits cartons numérotés (blocs de 8 Mio) vers un entrepôt privé. Si le camion tombe en panne, on repart au dernier carton livré. Les cartons les plus utiles (fiche, plan, vue grossière) partent en premier : on peut commencer à décorer avant que tout soit arrivé.

⚠️ ATTENTION

Un dataset publié par l'Import est **masqué par défaut**. Allez dans **Datasets** puis activez l'œil (ou l'interrupteur **Visibilité**) pour le montrer au public.

13.5 Éditer un dataset

Cliquez un dataset dans la liste de l'onglet **Datasets** : l'aperçu s'ouvre au centre, les réglages à droite. Voici ce que vous pouvez y régler :

SECTION	CE QUE VOUS Y DÉCIDEZ
Visibilité	visible ou masqué dans l'explorateur (appliqué immédiatement)
Identification	nom d'affichage, stade, embryon, description
Galerie d'images	jusqu'à 40 images (8 Mo max) avec légendes, affichées dans le viewer
Calibration physique	taille du voxel X, Y, Z en μm
Paramètres d'affichage	exposition par défaut
Orientation	sens de l'échantillon, repère de référence, axes affichés, vue par défaut
Canaux (dans l'aperçu)	nom, couleur, min, max, gamma de chaque canal

À RETENIR

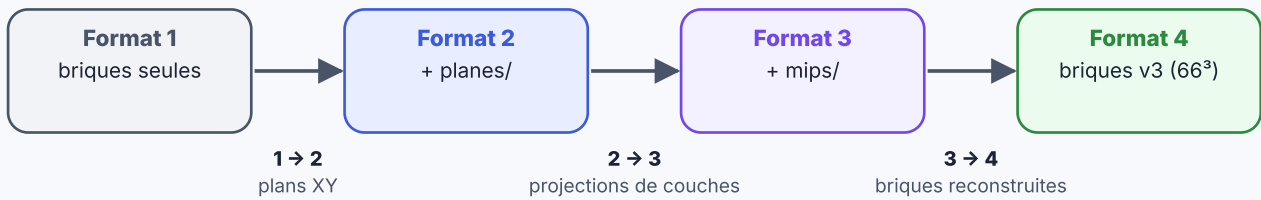
Ce que l'éditeur enregistre est un **réglage** : il est fusionné dans le `metadata.json` du dataset. Le panneau ne touche jamais aux pixels du niveau natif. La position de la caméra, le mode de rendu ou le plan de coupe ne sont pas conservés : ils appartiennent au visiteur.

Le bouton **Redéfinir la preview** (sous l'aperçu) fait de la vue actuelle la vignette de l'explorateur. L'orientation 3D (axes, vue par défaut) est expliquée au chapitre 12.

13.6 Mises à jour des données

La façon dont les datasets sont stockés a évolué quatre fois (**formats 1 à 4**, voir chapitre 7). Un ancien dataset reste lisible, mais il peut être **mis au format actuel sans repasser par le pipeline**, comme on met à jour un logiciel.

Mises à jour des données : formats 1 → 4



La nouvelle version se calcule à côté de l'ancienne ; elle ne la remplace qu'à la toute fin (version du dataset changée en dernier).

Qui fait le travail ? Vous choisissez, dataset par dataset :

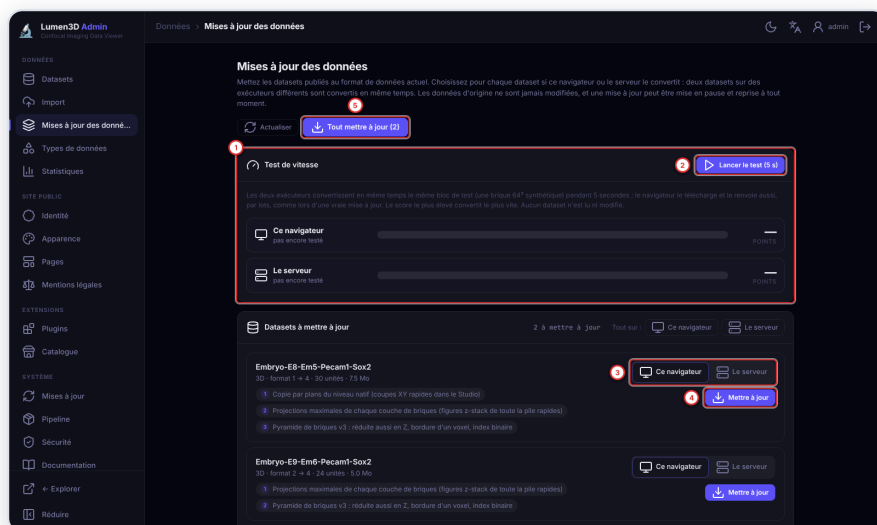


▲ Les trois mises à jour, les deux exécutants, le test de vitesse.

Les trois étapes ajoutent : **planes/** (un fichier par plan, coupes XY natives rapides dans le Studio), **mips/** (projection max de chaque couche de 64 plans, figures z-stack rapides), puis une pyramide de briques v3 (66³, bordure d'un voxel, **index.bin**).

? POURQUOI CE CHOIX ?

Le niveau natif est conservé voxel pour voxel : rien n'est perdu. L'ancien arbre **bricks/** n'est supprimé qu'après le changement de version du dataset. Un dataset produit par le pipeline 0.21.0 est **déjà** au format 4 : il n'y a rien à faire.



▲ L'onglet Mises à jour des données (jeu de démonstration, avant lancement).

1 2 Test de vitesse : **Lancer le test (5 s)** compare navigateur et serveur · **3** Choix de l'exécutant, dataset par dataset (verrouillé pendant l'exécution) · **4** **Mettre à jour** ce dataset · **5**

Tout mettre à jour : traite tous les datasets prêts

⚠ ATTENTION

Gardez l'onglet ouvert pendant une mise à jour. Le quitter la met en pause ; rien n'est perdu, vous reprenez plus tard. N'ouvrez pas plusieurs onglets à la fois : un hébergeur a déjà bloqué l'adresse d'un opérateur qui lançait trop de requêtes. Le régulateur réseau de l'onglet limite le débit pour éviter cela.

Ces mises à jour ne sont **pas obligatoires** : le viewer lit les formats 1 à 4. Elles accélèrent certaines opérations du Studio et améliorent l'affichage.

13.7 Plugins et catalogue signé

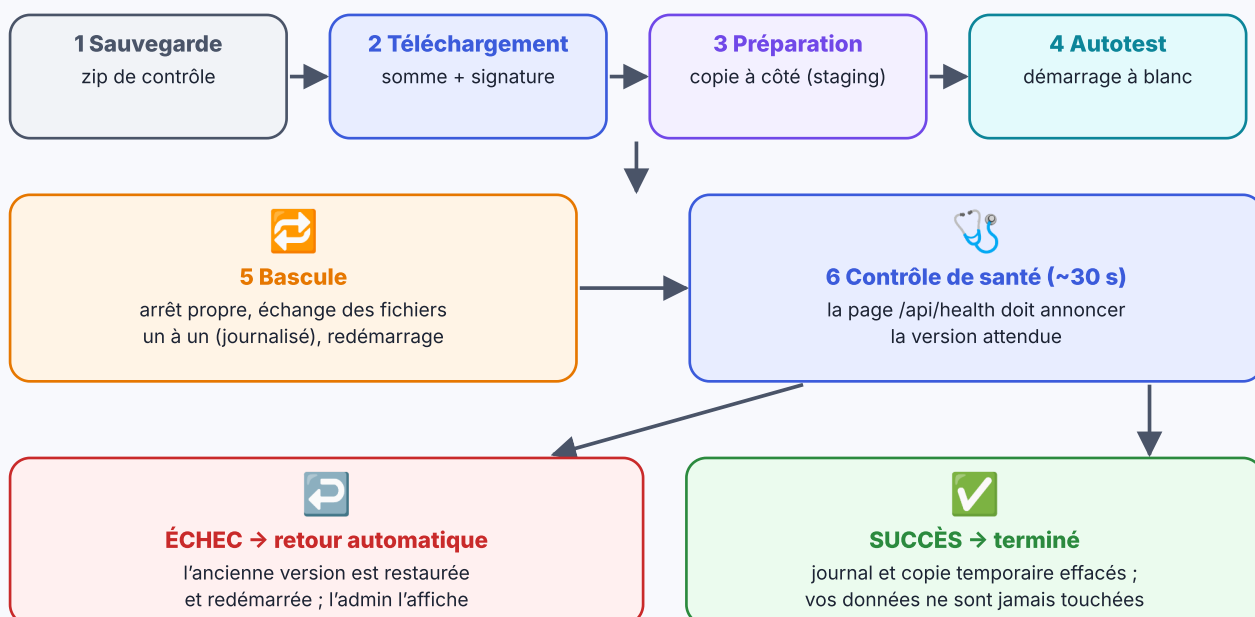
Les plugins sont les outils du viewer (chapitre 12). Ils **ne sont pas livrés avec la plateforme** : on les installe à la demande depuis le **Catalogue**, comme dans une boutique d'applications.

- Le catalogue est **signé** : un plugin non vérifié n'est jamais installé (chapitre 14).
- L'installation demande votre **mot de passe**, puis le plugin est installé et approuvé.
- L'onglet **Plugins** affiche trois cartes (Outils, Canaux, Modes de rendu) : un interrupteur par plugin et une étiquette de confiance (**intégré**, **approuvé**, **sandbox**, **non fiable** ...).
- Certains plugins n'apparaissent pas dans l'aperçu de l'onglet Datasets, ni dans la page Comparer : ils ne se déclarent compatibles qu'avec la page complète. Ce n'est pas une panne.

13.8 Mettre à jour la plateforme

Depuis **Système > Mises à jour**, un bouton lance la mise à jour. Ce qui se passe derrière a été pensé pour qu'un **site ne reste jamais cassé**.

Mettre à jour la plateforme : bascule avec filet de sécurité



Protégés pendant la mise à jour : DATA_WEB, config/, uploads/, identifiants, statistiques, plugins installés.

▲ La bascule de mise à jour et son filet de sécurité.

🔦 UNE IMAGE POUR COMPRENDRE

Le déménagement bleu-vert. On monte le nouvel appartement à côté de l'ancien, on vérifie que l'eau et l'électricité marchent, et seulement alors on y emménage. Si quelque chose cloche, on retourne dans l'ancien, resté intact.

- L'onglet affiche les **notes de version** de chaque version qui sera installée, et vérifie vos plugins avant : ceux qui seraient incompatibles sont mis de côté, puis réactivés plus tard.
- Il montre aussi la version du **pack Pipeline** (onglet **Pipeline**) : c'est un autre logiciel, qui s'installe sur le poste de traitement, pas sur le serveur.

13.9 Personnaliser le site (marque blanche)

Lumen3D n'est pas lié à un laboratoire : tout ce que le public voit se règle sans écrire de code.

ONGLET	CE QUE VOUS Y PERSONNALISEZ
Identité	nom de l'instance, nom du « spécimen » (embryon, organe...), accroche, pied de page, liens du menu
Apparence	couleur de marque, police, arrondis, avec aperçu en direct
Pages	éditeur visuel : sections, colonnes, widgets ; sauvegarde automatique du brouillon
Mentions légales	sections de texte, une langue à la fois

ONGLET	CE QUE VOUS Y PERSONNALISEZ
Types de données	le nom donné à « 3d », « 2d » et « live » dans les badges et les filtres

À SAVOIR

Renommer un type de données ne change que le **texte affiché** : jamais les dossiers, les identifiants ni les adresses. Le nom du spécimen et des types s'adapte automatiquement dans tout le site.

POUR ALLER PLUS LOIN

Pour le détail de chaque écran, des messages d'erreur et des cas particuliers, consultez le « **Guide de l'administrateur** » (onglet [Documentation](#)). Pour la sécurité du panneau, voir le chapitre 14.

14. Sécurité et fiabilité

- Lumen3D protège le panneau d'administration, le code exécuté dans les pages et les fichiers déposés. Chaque protection a une **raison simple**.
- Aucune donnée d'étude n'est envoyée à un tiers et la plateforme fonctionne **hors ligne**.
- La fiabilité repose sur des publications **tout ou rien** et sur une suite d'environ **200 fichiers de tests** qui bloque toute version défectueuse.

14.1 Le mot de passe administrateur

Il n'existe **aucun mot de passe par défaut** : le premier visiteur du panneau crée le compte (8 caractères minimum), et cette création ne peut jamais écraser un compte existant.

Le mot de passe administrateur



On ne stocke jamais le mot de passe : seulement son « empreinte digitale ». On ne remonte pas d'une empreinte à un mot de passe.

Contre les essais répétés



▲ Du mot de passe à l'empreinte, et les protections contre les essais répétés.

UNE IMAGE POUR COMPRENDRE

Une empreinte digitale. Le serveur garde l'empreinte de votre mot de passe, pas le mot de passe. Avec une empreinte on peut reconnaître la bonne personne, mais on ne peut pas refabriquer le doigt.

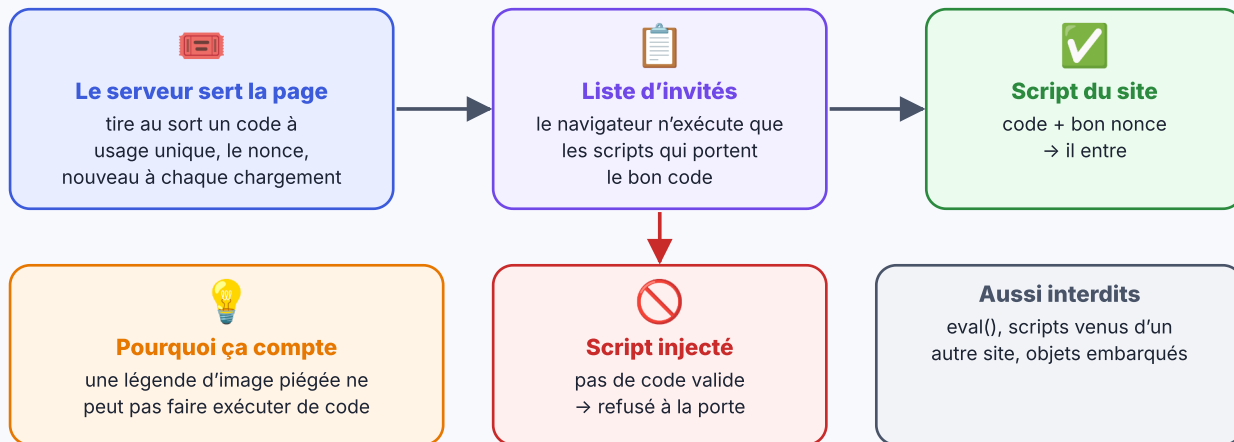
- L'empreinte est calculée par **PBKDF2-HMAC-SHA256**, avec un sel propre au compte et 600 000 tours : volontairement lent pour décourager les essais en masse.
- Après **10 échecs** en 15 minutes depuis une même adresse, l'accès est bloqué 15 minutes. Les échecs sont comptés **avant** la vérification, et un mauvais identifiant coûte le même temps qu'un mauvais mot de passe.
- Une session dure **8 heures**. Changer le mot de passe ferme toutes les autres sessions.
- Toute modification passe aussi par un jeton anti-falsification (CSRF).

14.2 La politique de sécurité du contenu (CSP)

💡 UNE IMAGE POUR COMPRENDRE

Une liste d'invités à l'entrée. À chaque chargement de page, le serveur tire un code secret (le « nonce ») et le remet aux scripts du site. Le navigateur ne laisse entrer que ceux qui le présentent.

La politique de sécurité du contenu (CSP)

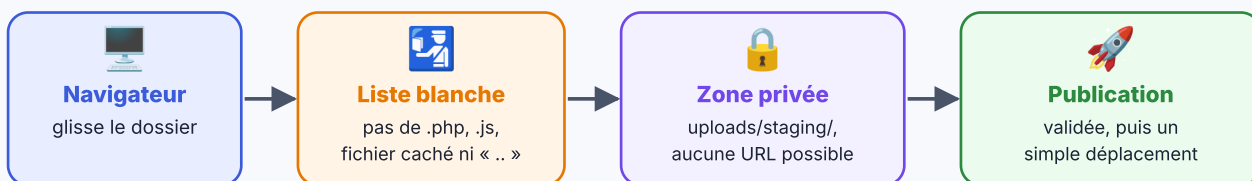


▲ Le nonce, une liste d'invités renouvelée à chaque chargement.

La politique est **appliquée** (et non simplement observée). Les bibliothèques (Three.js, Lucide, Plotly) sont hébergées par le site lui-même : aucun script ne vient d'ailleurs.

14.3 Les fichiers importés ne sont jamais servis

Les fichiers importés ne sont jamais servis par URL



Chaque bloc est vérifié (SHA-256) avant d'être écrit ; l'aperçu administrateur lit la zone privée par un relais protégé par la session.

▲ Le trajet d'un fichier importé jusqu'à sa publication.

- Les fichiers arrivent dans `uploads/`, un dossier **inaccessible par URL** (quatre verrous indépendants).
- Une **liste blanche** refuse avant écriture tout ce que le pipeline ne produit pas : pas de `.php`, de `.js`, de fichier caché, de chemin remontant (`..`).
- Seule une **publication explicite**, après validation, déplace le dataset vers `DATA_WEB/`. Ce dossier interdit l'exécution de scripts.

14.4 Les plugins : confiance et cage

Confiance accordée aux plugins

Les plugins : par défaut, on n'ouvre pas

Intégré / catalogue

signé, vérifié

Approuvé

par vous, lié à une empreinte

Développement

avec --dev-trust-local

Non fiable

non chargé



Approbation liée à une empreinte

Vous approuvez ce contenu précis. Si un seul fichier change, l'empreinte change : le plugin redevient non fiable. Votre mot de passe est demandé pour approuver.



Bac à sable (iframe isolée)

Un plugin tiers peut tourner dans une cage : sans accès à la page, il ne parle que par messages, avec des permissions déclarées et un débit limité.

▲ Niveaux de confiance, approbation liée à une empreinte, bac à sable.

? POURQUOI CE CHOIX ?

Un plugin est du code. Le laisser entrer sans contrôle serait comme donner un double des clés à un inconnu. Par défaut, un plugin non reconnu **n'est pas chargé**.

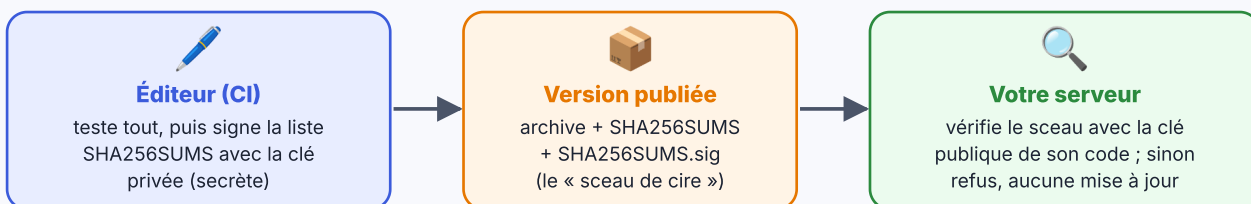
- Votre approbation est liée au **contenu exact** du plugin (empreinte) : modifié, il perd sa confiance.
- Un plugin déclare aussi avec quelles versions de la plateforme il est compatible : en cas de doute, il est écarté.

14.5 Versions et catalogue signés

💡 UNE IMAGE POUR COMPRENDRE

Un sceau de cire. Seul l'éditeur possède le sceau (la clé privée). Tout le monde peut vérifier qu'une lettre le porte (clé publique), mais personne ne peut en fabriquer un faux.

Versions et catalogue signés (Ed25519)



Seconde clé, distincte : elle signe le catalogue de plugins (avec un numéro de série qui empêche de revenir à un catalogue plus ancien).

▲ La chaîne de confiance d'une mise à jour : signature, vérification, refus.

- La signature est une **Ed25519**. La clé publique est **intégrée au code** du serveur : une clé fournie avec le téléchargement ne serait pas fiable.

- Une version sans signature valable est **refusée** : le programme de mise à jour n'applique que l'archive nommée d'après la version et listée dans la liste signée.
- Le catalogue de plugins a **sa propre clé** et un numéro de série croissant, pour qu'on ne puisse pas réinstaller une ancienne version corrigée depuis.

14.6 Vos données restent chez vous

- **Aucune donnée d'étude** n'est envoyée à un tiers.
- **Hors ligne** : tout le code est hébergé sur le serveur. Seules les polices Google Fonts sont chargées à distance. GitHub n'est contacté que par l'administrateur, pour vérifier les mises à jour.
- Les statistiques sont de simples compteurs (visites, vues, téléchargements), limités en débit côté serveur.

14.7 Fiabilité

RISQUE	PROTECTION
Une mise à jour casse le site	bascule vérifiée, retour automatique, sauvegarde préalable
Une publication s'interrompt	tout ou rien : le pipeline construit tout à l'écart et publie par un renommage ; une interruption est rattrapée au lancement suivant
Une coupure réseau pendant un import	reprise au bloc près
Une brique corrompue	signalée et ignorée, jamais envoyée à la carte graphique
Une régression dans le code	environ 200 fichiers de tests doivent passer avant chaque version

À RETENIR

Chaque version est **testée en entier** avant d'être signée et publiée. Une version qui échoue aux tests ne peut tout simplement pas sortir.

15. Glossaire

- Les termes sont classés **par ordre alphabétique**. Chaque définition tient en une ou deux phrases.
- « → chapitre N » vous envoie là où le mot est expliqué en détail.
- Les noms entre crochets, comme **Studio**, sont ceux que vous voyez à l'écran.

Atlas

Grande texture 3D en mémoire graphique où l'on range les briques à afficher, comme les cases d'un casier. → chapitre 10

Bordure (apron)

Voxel supplémentaire autour d'une brique, copié depuis ses voisins, pour que le filtrage ne laisse pas de couture entre deux briques. Les briques du format 4 font 66^3 pour 64^3 utiles. → chapitre 7

Brique

Petit cube de $64 \times 64 \times 64$ voxels : l'unité de découpage et de chargement d'un volume. → chapitre 7

Bruit de fond

Signal parasite présent même là où il n'y a rien à voir (lumière diffuse, électronique du capteur). → chapitre 5

Cache

Mémoire où l'on garde ce qui a déjà été téléchargé pour ne pas le redemander. → chapitre 10

Canal

Une image du même objet pour une couleur de fluorescence donnée (par exemple DAPI, Pecam1, Sox2). → chapitre 4

Exposition

Réglage de luminosité globale appliqué à l'affichage ; il n'altère pas les données. → chapitre 11

Compression avec perte, sans perte

Avec perte, le fichier est plus petit mais les valeurs sont légèrement modifiées ; sans perte, on retrouve exactement les valeurs d'origine. Les briques sont stockées sans perte. → chapitre 6

CSP (politique de sécurité du contenu)

Règle donnée au navigateur : n'exécuter que les scripts portant un code à usage unique (nonce). → chapitre 14

Dilatation

Opération qui épaissit les zones d'un masque d'un ou plusieurs pixels. → chapitre 5

Échantillon (rendu)

Valeur lue en un point du volume pendant le lancer de rayons. Un rayon en prend des centaines. → chapitre 9

Ed25519 (signature)

Méthode de signature numérique : un « sceau de cire » que seul l'éditeur peut apposer et que tout le monde peut vérifier. → chapitre 14

ESS (Empty Space Skipping, saut de l'espace vide)

Les briques presque entièrement vides ne sont pas stockées ni parcourues, ce qui économise octets et calcul. → chapitres 7 et 9

GPU

Processeur de la carte graphique, spécialisé dans le calcul parallèle ; il exécute le lancer de rayons. → chapitre 9

Fenêtrage

Choix des valeurs min et max affichées : en dessous, noir ; au-dessus, pleine intensité. → chapitre 11

Filtre médian

Remplace chaque valeur par la médiane de ses voisins : il supprime les points isolés sans brouiller les contours. → chapitre 5

Fonction de transfert

Règle qui convertit une valeur du volume en couleur et en opacité. → chapitre 11

Format de données

Façon dont un dataset est rangé sur le disque ; numéroté de 1 à 4. Le format courant est le 4. → chapitres 7 et 13

Gamma

Réglage qui éclaircit ou assombrit les tons moyens sans toucher aux extrêmes. → chapitre 11

Kabsch (algorithme)

Méthode qui trouve la rotation et le décalage alignant au mieux deux nuages de points ; utilisée pour stabiliser un timelapse. → chapitre 8

Lancer de rayons (ray marching)

Pour chaque pixel de l'écran, on suit un rayon à travers le volume en cumulant couleur et opacité. → chapitre 9

LOD (niveau de détail) / niveau

Version du volume à une résolution donnée ; le niveau 0 est le plus fin. → chapitre 6

Manifeste

Fichier qui décrit le contenu d'un dataset (niveaux, briques, packs). → chapitre 7

Masque

Image en noir et blanc qui dit quels pixels comptent (blanc) et lesquels sont écartés (noir). → chapitre 5

HDF5

Format de fichier scientifique qui range de gros tableaux de nombres, comme un classeur à tiroirs. Les fichiers Imaris en sont un. → chapitre 4

Histogramme

Graphique qui compte combien de voxels ont chaque valeur d'intensité ; il aide à régler min et max. → chapitre 11

.ims

Fichier Imaris : un conteneur HDF5 qui contient les canaux, les niveaux de résolution et les métadonnées d'une acquisition. → chapitre 4

index.bin

Fichier binaire du format 4 qui dit dans quel pack se trouve chaque brique ; il remplace le gros manifeste JSON. → chapitre 7

Interpolation trilinéaire

Estimation d'une valeur entre voxels en mélangeant les huit voisins, ce qui donne une image lisse. → chapitre 9

MIP (projection d'intensité maximale)

Image où chaque pixel prend la valeur la plus forte rencontrée le long d'un axe. → chapitres 7 et 12

Mosaïque

Plusieurs petites images collées en une grande ; une brique est stockée comme une mosaïque de ses coupes. → chapitre 7

Nonce

Code aléatoire à usage unique, tiré à chaque chargement de page, qui autorise les scripts du site. → chapitre 14

Opacité

Degré d'absorption de la lumière par une structure : 0 est transparent, 1 est opaque. → chapitre 11

Ouverture morphologique

Érosion suivie d'une dilatation : elle efface les petits points isolés d'un masque et garde les grandes formes. → chapitre 5

Migration

Mise à jour d'un dataset déjà publié vers un format plus récent, sans repasser par le pipeline. → chapitre 13

Percentile

Valeur sous laquelle se trouve un pourcentage donné des mesures : le 99e percentile est dépassé par 1 % des valeurs. → chapitre 5

Pixel

Plus petit élément d'une image 2D. → chapitre 4

Plans

Dossier `planes/` du format 2 : un fichier par plan z du niveau natif, pour lire une coupe XY sans charger 64 plans. → chapitre 13

Plugin

Outil optionnel qui s'ajoute au viewer (mesure, capture, filtre...) ; il est installé depuis le catalogue signé. → chapitres 12 et 13

PNG

Format d'image sans perte ; les plans du format 2 sont des tuiles PNG de 512×512. → chapitre 13

Profondeur de bits

Nombre de bits par valeur : 8 bits donnent 256 niveaux, 16 bits en donnent 65 536. → chapitre 4

Studio

Outil de figures : recadrage, annotations, barres d'échelle et export d'images pour publication. → chapitre 12

Suivi cellulaire

Suivi de chaque cellule d'image en image dans un timelapse (trajectoires, lignées, divisions). → chapitre 8

Super-bloc

Groupe de 4×4×4 briques voisines qu'un pack range en bloc pour qu'une région se lise d'un seul tenant. → chapitre 7

Pack

Fichier `.bin` qui regroupe de nombreuses briques pour limiter le nombre de téléchargements. → chapitre 7

Pyramide

Ensemble de versions du volume à des résolutions de plus en plus grossières, empilées comme les étages d'une pyramide. → chapitre 6

Quaternion

Quatre nombres qui décrivent une rotation 3D sans blocage ; ils servent à mémoriser l'orientation d'un embryon. → chapitre 12

Shader

Petit programme exécuté par la carte graphique ; celui du viewer fait le lancer de rayons. → chapitre 9

Signature

Voir Ed25519. → chapitre 14

Staging

Zone privée où arrivent les fichiers importés avant publication ; elle n'est jamais accessible par URL. → chapitres 13 et 14

Streaming

Chargement progressif : on affiche ce qui est arrivé et on complète en continu, sans attendre tout le fichier. → chapitre 10

Voxel

Pixel en 3D : un petit cube de valeur d'intensité. Sa taille réelle (en μm) est la calibration. → chapitre 4

VRAM

Mémoire de la carte graphique. Son budget limite la taille d'atlas que le viewer peut utiliser. → chapitre 10

WebGL2

Technologie du navigateur qui donne accès à la carte graphique, notamment aux textures 3D. → chapitre 9

Table de pages

Tableau qui dit où chaque brique se trouve dans l'atlas, comme un plan de casiers. → chapitre 10

Three.js

Bibliothèque JavaScript qui simplifie l'affichage 3D dans le navigateur. → chapitre 2

Timelapse

Série d'acquisitions du même spécimen au fil du temps, chaque image étant un volume 3D. → chapitre 4

WebP sans perte

Format d'image compressé qui restitue exactement les valeurs d'origine ; il stocke les mosaïques de briques. → chapitre 6

Worker (Web Worker)

Processus d'arrière-plan du navigateur : il décode et calcule sans geler l'écran. → chapitre 10